

P vs NP Proof Attempts: Crux-First and Steelman Lean Notes on a Weakness-Based Route

May 18, 2026

Abstract

This note records the current Lean work around a claimed proof of $P \neq NP$ via “quantale weakness and geometric complexity.” The project now proceeds on two tracks in parallel.

The *blocker* track asks where the argument must fail unless an additional theorem is supplied. The *optimistic* track assumes the high-level strategy may work and builds the missing background theory from the ground up so that any genuine repair can plug into a machine-checked scaffold.

The current Lean work proves concrete obstructions. First, a decoder that is merely “local on a bounded-degree neighborhood of radius $\Theta(\log m)$ ” is *not* thereby polylogarithmic in size: the visible neighborhood already has polynomial size in m . Second, if one does not prove an additional compression theorem, then the class of all local rules on that neighborhood is doubly exponential in the neighborhood size, so it cannot be uniformly represented by a polylogarithmic code budget.

Before the switching story even starts, the sampler already carries another burden. The manuscript fixes the VV width at $k = c_1 \log m$, but the cited $\Omega(1/m)$ uniqueness probability comes from averaging over many scales of k . The new Lean obstruction packages the fixed-width variance bound: if the retained-solution count has mean $\mu > 1$ and variance at most μ , then exact isolation occurs on at most a $\mu/(\mu - 1)^2$ fraction of hash choices. So fixed logarithmic width can only isolate often when the pre-hash solution set is already essentially polynomial in size.

And even the manuscript’s repeated *pairwise-independent columns* language cannot shoulder the hashing argument by itself. A separate Lean countermodel shows that pairwise-independent columns do not imply universal hashing: one can have pairwise-uniform columns together with a deterministic three-column linear dependency, so a nonzero difference hashes to zero with probability 1.

Nor can the manuscript’s δ -biased right-hand side rescue the hash moments by itself. Another Lean module shows that once a candidate’s label map is genuinely uniform over the hash-label space, its average hit mass against *any* finite probability weight on labels is exactly the uniform value $1/|L|$; and if a pair of candidates has jointly uniform labels, then the weighted average joint-hit mass is exactly $1/|L|^2$. So any useful effect of the δ -biased right-hand side must come from failure of the underlying label maps to be genuinely uniform, not from the bias itself.

And even exact pre-conditioning uniformity is not enough by itself. A further Lean countermodel shows that conditioning on a label-dependent event can collapse a perfectly uniform label map to a point mass. So any appeal to pre-conditioning 2-universality or uniform labels must be supplemented by a separate theorem about what survives the manuscript’s later conditioning on uniqueness. The newest Lean strengthening records the corresponding independence failure in reusable form: inside any conditioning event, deterministic coupling $E \leftrightarrow F$ is conditionally independent exactly in the degenerate cases where E is empty or full on that conditioning event. The same exact boundary now also covers complement coding $E \leftrightarrow \neg F$. The four-point diagonal model is now an instance, not merely a one-off computation: two Boolean coordinates can be exactly independent before conditioning, while conditioning on the diagonal event makes them

coupled and nondegenerate. Thus the v11 uniqueness-conditioned route needs a real local-global decorrelation theorem, not just pre-conditioning independence of the VV layer.

Third, even one natural repair to the calibration step has a sharp failure mode: if an involution flips the target bit while preserving the retained local features, then every classifier using only those invariant features has exactly $1/2$ average accuracy. So one cannot repair the sign-flip calibration bug merely by dropping the involution-sensitive part of the local input and hoping marginalization will preserve a domination claim.

Even before that charitable repair, the manuscript’s exact local input formula already has a hard consistency problem: it defines the post-switch input as $u = (z, a_i, b)$ while the involution T_i toggles b by a_i . So full-input invariance can hold only on the zero-column case $a_i = 0$.

The new bridge module sharpens that fork: on the exact same T_i -action, the reduced projection (z, a_i) is invariant and therefore carries zero symmetry-respecting signed signal, while any advantage obtained by also retaining b is bounded by the mass on nonzero VV columns, since those are exactly the points where b resolves an involution pair.

Sharper still, the same involution hypotheses force every retained feature fiber to contain exactly half target-1 and half target-0 points. So even soft scores or conditional-mean estimators on those invariant features have zero margin on every feature value.

And this is not just a uniform-counting artifact: any involution-invariant finite weighting still assigns equal total mass to the target-1 and target-0 parts of every retained feature fiber. So non-uniform but symmetry-respecting sampling does not restore a usable margin.

In fact, any soft score computed only from those involution-invariant retained features has zero weighted signed correlation with the target. So even a richer invariant scoring statistic carries no usable directional signal before genuine symmetry breaking is introduced.

More sharply, even when the weighting is not exactly involution-invariant, the entire invariant-score signal is still carried only by the antisymmetric part of that weighting. So this rescue route now needs a real theorem that substantial useful mass lives in the weight asymmetry itself. The support-level version is formal too: if the weights are symmetric at every nonzero-score point, the signal is zero; any nonzero signal must expose a point where the score is nonzero and the weight changes across the involution.

Likewise, if one augments the invariant local view by a side channel, then any advantage gained by classifying on the pair (u, v) is still bounded by the total mass where the side channel actually separates an orbit point from its involution partner.

Equivalently, any claimed domination gap over chance now forces a matching amount of orbit-resolving mass. So one cannot even state a serious advantage theorem without simultaneously proving a serious resolution theorem.

More generally, keeping a controlled amount of non-invariant information only helps on the slice where it really breaks the symmetry. Any residual slice on which the retained features are still involution-invariant remains exactly at chance under symmetry-respecting weights.

Quantitatively, any excess over chance is bounded by the weight of the symmetry-broken region. So a proof that keeps only a little non-invariant information must also show that almost all relevant mass actually lies outside the unresolved symmetric slice. The support-level version is now formal too: if all positive-weight points remain in the unresolved slice, strict advantage is impossible; if strict advantage is claimed, Lean produces a positive-weight point outside that slice.

Fourth, the natural “short global program” rescue also fails on its own: if the switched input really has only $n = O(\log m)$ visible bits, then an arbitrary local rule on those bits can be hard-wired by a truth table of size $2^n = \text{poly}(m)$. So polynomial-size decoders do not by themselves force any small subclass inside the full local-rule space.

Fifth, the most creative metagraph-style rescue faces its own exactness barrier: if a quotient state must capture the full interface behavior of a k -bit local gadget, then there are 2^{k2^k} such boundary transducers. At binary-log width this already forces at least linear code size in m , so a small exact metagraph/message quotient cannot appear by default.

Sixth, even the tree-message route does not follow from tree-likeness alone: a fixed perfect binary tree architecture of depth n already realizes every Boolean rule on n visible bits, just by

changing its leaf labels.

The v13 crux ledger adds two smaller but sharper obligations. A $P = NP$ upper-bound decoder that uses an arbitrary SAT-search output cannot recover a hidden message unless every satisfying witness has the same message readout. The current Lean interface packages a SAT-search output as an arbitrary satisfying witness and proves that correctness for every valid SAT-search output is exactly the fixed-message readout theorem; in the satisfiable case, this is equivalent to single-message readout constancy, even after existentially quantifying over the claimed output message. The generic blocker is now theorem-level: any two satisfying witnesses with different readouts produce two valid SAT-search outputs with different readouts and rule out every fixed-message decoder correct for all valid outputs. Bare satisfiability plus a readout map is not enough; the two-point Boolean model is just the smallest instance of this general obstruction. The newest Lean strengthening separates this from the weaker “selected witness” contract:

CorrectForSomeSatSearchOutput

is only existence of one satisfying witness with the desired readout. Distinct satisfying readouts certify two incompatible selected-output messages, while still ruling out any message correct for all valid SAT-search outputs:

`exists_two_messages_correctForSomeSatSearchOutput_of_distinct_satisfying_readouts,`

`exists_correctForSomeSatSearchOutput_and_not_exists_correctForAll_of_distinct_satisfying_readouts.`

Thus a repair that silently chooses a favorable satisfying witness is not the same theorem as a decoder robust to arbitrary SAT-search output. `PNPv13ConcreteSATReadoutObstruction.lean` now instantiates the same point with ordinary CNF syntax rather than an abstract witness type. The concrete formula is the one-variable tautological clause $x \vee \neg x$. Lean proves that both the false and true assignments satisfy the CNF, that reading out the value of x gives distinct messages, and therefore

`oneVarTautologyCNF_not_exists_correctForAllSatSearchOutputs.`

It also proves selected-output correctness for both messages. Thus the manuscript-facing missing assumption is not “the realization is satisfiable” but a genuine theorem that all satisfying assignments of the actual CNF realization have the same message readout, or that SAT search is constrained by a stronger output contract. The positive side is now formal at the same concrete CNF interface:

`cnf_exists_correctForAllSatSearchOutputs_iff_singleMessageReadout_of_satisfying_assignment.`

For any satisfiable concrete CNF, an arbitrary-output SAT-search decoder message exists exactly when the satisfying assignments have single-message readout constancy. `PNPv13ManuscriptCNFReadout.lean` then formalizes the manuscript’s own Appendix I.23 bridge rather than only the abstract or one-variable surfaces. Its interface records the supported public instance Y , the CNF F_Y , the extracted witness $\text{Wit}_Y(\alpha)$, the semantic witness relation $R(Y, W)$, the witness message $M(W)$, and the fixed projection $\pi_M(\alpha)$. Lean proves

`ManuscriptCNFReadoutData.singleMessageReadout_of_singleMessagePromise,`

namely: if satisfying CNF assignments extract to valid witnesses, if the projection agrees with the extracted witness message, and if valid witnesses for the supported Y satisfy the v13 single-message promise, then all satisfying CNF assignments have the same projected message. With one satisfying assignment this yields

`ManuscriptCNFReadoutData.exists_correctForAllSatSearchOutputs_of_singleMessagePromise.`

The same module now proves the converse under the natural completeness condition for the manuscript extraction layer. `ManuscriptCNFReadoutData.CNFExtractionComplete` says every

supported valid witness is represented by some satisfying CNF assignment with the same message projection, and `ManuscriptCNFReadoutData.SupportedCNFSatisfiable` records that the supported CNF has at least one satisfying assignment. Under these two hypotheses Lean proves

`ManuscriptCNFReadoutData.supportedArbitraryOutputSATSearchCorrect_iff_singleMessagePromise.`

Thus replacing the fixed-message theorem by an arbitrary-output SAT-search procedure does not weaken the semantic obligation: for complete supported CNF realizations, SAT-search correctness is exactly the single-message promise. The one-variable manuscript countermodel is also marked complete and satisfiable, via

`ambiguousOneVarManuscriptCNFReadoutData_cnfExtractionComplete`

and

`ambiguousOneVarManuscriptCNFReadoutData_not_supportedArbitraryOutputSATSearchCorrect.`

A complementary boundary now proves that the completeness hypothesis is not cosmetic. The `unrepresentedOneVarManuscriptCNFReadoutData` model has a constant CNF projection, so

`unrepresentedOneVarManuscriptCNFReadoutData_supportedArbitraryOutputSATSearchCorrect`

holds, but it also has an unrepresented valid semantic witness with the opposite message:

`unrepresentedOneVarManuscriptCNFReadoutData_satSearchCorrect_and_not_singleMessagePromise,` `unrep`

Thus CNF-side arbitrary-output correctness can only be promoted back to the semantic single-message promise after proving that every supported valid witness is represented by a satisfying CNF assignment. The same file also proves the boundary is real: a one-variable CNF countermodel satisfies the compiler/extraction/projection shape while failing the single-message promise and while admitting no message correct for all SAT-search outputs. Thus the current minimal manuscript obligation is not the local CNF compiler, but Proposition 4.10 / Lemma I.23's premise: locked-layer rigidity must be instantiated for the actual gauge-buffered locked construction so that every valid extracted witness has the same message. The same Lean module now pushes one step further into Proposition 4.10 itself. The `LockedLayerCNFReadoutData` refinement exposes a locked-layer view of each semantic witness, the locked-layer acceptance predicate, and the message carried by that locked view. Lean proves

`LockedLayerCNFReadoutData.singleMessagePromise_of_lockedLayerRigidity,`

which is the formal Proposition 4.10 argument: if every valid witness satisfies the locked layer and its witness message is the locked-layer message, then Hypothesis 4.9's rigidity of accepted locked views implies the witness-level single-message promise. Composed with Appendix I.23, this gives

`LockedLayerCNFReadoutData.exists_correctForAllSatSearchOutputs_of_lockedLayerRigidity.`

The same surface can be instantiated either with the raw Section 4 predicate Lock_Y , matching Hypothesis 4.9, or with the Appendix D locked-completion predicate $\text{Lock}_Y \wedge \text{Ready}_Y$, matching Hypothesis D.8. Lean now records the exact contrapositive certificate:

`LockedLayerCNFReadoutData.not_lockedLayerRigidity_of_distinct_accepted_lockMessages.`

Thus an attempted concrete formalization of the v13 locked core has a crisp pass/fail target: prove that no two accepted lock completions over the same supported public syntax carry different

messages, or exhibit such a pair and the readout-constancy route fails at Proposition 4.10. At the witness level the analogous obstruction is

`ManuscriptCNFReadoutData.not_singleMessagePromise_of_distinct_valid_witnessMessages.`

The remaining unproved manuscript burden has therefore been localized to the actual locked-layer construction: prove Hypothesis 4.9 for the concrete Lock_Y predicates without smuggling the message as public data. A matching one-variable locked-layer countermodel shows that the locked-layer-shaped interface without rigidity still permits two accepted locked views with different messages and no arbitrary-output SAT-search decoder message. `PNPv13AppendixDLockedCore.lean` now formalizes the Appendix D version directly. A locked completion is the manuscript’s triple $(z, \xi_{\text{lock}}, M)$ satisfying both $\text{Lock}_Y(z, \xi_{\text{lock}}, M)$ and $\text{Read}_Y(z, \xi_{\text{lock}}, M)$. Appendix D.8 is represented as `AppendixDLockedCore.LockedMessageRigidity`; D.7 plus D.8 gives an existential, choice-free locked-message specification via

`AppendixDLockedCore.exists_lockedMessageSpec_of_lockSatisfiable_of_lockedMessageRigidity.`

The Appendix D.41/D.42 witness layer and Appendix I semantic CNF layer then prove, respectively,

`AppendixDWitnessData.singleMessagePromise_of_lockedMessageRigidity,`

`AppendixICNFReadoutData.singleMessageReadout_of_lockedMessageRigidity,` `AppendixICNFReadoutData.`

Thus the actual Appendix D/I interface guarantees fixed CNF readout only because D.8 is supplied. It is not a consequence of the finite locked-core syntax itself. Lean proves this with `ambiguousAppendixDLockedCore`: one public lock syntax, two quotient states, trivial Lock , and deterministic $\text{Read}(z, M) \equiv (M = z)$. This core satisfies D.7 and deterministic readout, but

`ambiguousAppendixDLockedCore_not_lockedMessageRigidity`

and its Appendix I realization still has no message correct for arbitrary satisfying SAT-search outputs:

`ambiguousAppendixICNFReadoutData_not_exists_correctForAll.`

`PNPv13AppendixDLocalLockedCore.lean` then strengthens this no-go to the bounded-local predicate language of Appendix D.5. It defines `BoolLocalConstraint` as a finite-scope Boolean predicate, packages finite conjunctions with a shared arity bound as `BoundedBoolLocalPredicate`, and forgets the local presentation back to the Appendix D semantic core through `AppendixDLocalBoolLockedCore.toAppendixDLockedCore`. The one-bit model `ambiguousAppendixDLocalBoolLockedCore` has empty local Lock clauses and a single arity-two local Read clause enforcing $M = z$. Lean proves that this bounded-local core satisfies D.7 and fixed-state readout determinism, but still refutes D.8:

`ambiguousAppendixDLocalBoolLockedCore_not_lockedMessageRigidity.`

Thus bounded arity and locality in D.5 do not generate cross-completion message constancy. Appendix D.8 remains the load-bearing global rigidity theorem that must be proved for the manuscript’s concrete locked construction. `PNPv13AppendixDPublicSyntaxDiscipline.lean` adds the public-syntax discipline from Appendix D.29–D.31 on top of this local core. A public syntax record has template and surface atoms, both normalizing through a neutral alphabet; Lean proves the D.31 core statement that every listed public template or surface atom has a neutral normal form. The same one-bit local core can be equipped with singleton neutral public syntax:

`ambiguousPublicSyntaxDisciplined_not_lockedMessageRigidity.`

It still satisfies D.7 and deterministic readout. Therefore the no-public-tag discipline is necessary hygiene, but it is not a substitute for the semantic locked-message rigidity theorem. `PNPv13AppendixDProductConstruction.lean` then starts on the concrete route suggested by

Appendix D.36–D.41. It formalizes the product witness shape $(x, g, z, \xi_{\text{lock}}, \xi_{\text{buf}}, \xi_{\text{aux}}, M)$ and the D.38 product relation as locked acceptance, readout consistency, gauge, buffer, and auxiliary clauses. Lean proves the D.41 bridge

`AppendixDProductConstruction.singleMessagePromise_of_lockedMessageRigidity,`

extracting a locked completion from every valid product witness and applying locked-core rigidity. It also proves the exact contrapositive:

`AppendixDProductConstruction.not_lockedMessageRigidity_of_distinct_valid_witnessMessages.`

The neutral-public one-bit core lifts through a product construction with trivial gauge, buffer, and auxiliary clauses:

`ambiguousAppendixDProductConstruction_not_lockedMessageRigidity.`

This matches Appendix D.56’s suggested record split: the product construction uses a `LockCore` component carrying a rigidity proof. The remaining non-interface burden is still a proof of that component theorem for the actual locked predicates. `PNPv13AppendixDProductCNFBridge.lean` now connects this product relation to Appendix I’s CNF extraction/projection layer. The positive theorem

`AppendixDProductCNFReadoutData.singleMessageReadout_of_lockedMessageRigidity`

shows that the product-CNF route closes exactly when the underlying locked core has D.8 rigidity. The matched contrapositive is sharper for crux work:

`AppendixDProductCNFReadoutData.not_lockedMessageRigidity_of_distinct_satisfying_projections.`

Thus two satisfying assignments of the realized CNF with different fixed product-message projections refute the locked-core theorem needed by the whole product route. The same file instantiates this with the one-variable tautological CNF: both satisfying assignments extract to valid Appendix D product witnesses, their singleton-block messages differ, and therefore

`ambiguousAppendixDProductCNFReadoutData_not_lockedMessageRigidity,      ambiguousAppendixDProductCNF`

This is the current steelman boundary for the Appendix D product repair: the CNF compiler and product witness clauses are not enough; the author must prove that the concrete locked predicates exclude every such pair of satisfying assignments with distinct product messages. `PNPv13AppendixDProductExactCNFBridge.lean` strengthens the same test against a more charitable repair: suppose the CNF realization is exact enough that every valid product witness has a satisfying CNF encoding with the same fixed projected message. Lean packages this as `AppendixDProductExactCNFReadoutData`. Even then, exactness only transfers ambiguity back to the CNF:

`AppendixDProductExactCNFReadoutData.not_singleMessageReadout_of_distinct_valid_witnessMessages,`

`AppendixDProductExactCNFReadoutData.not_exists_correctForAllSatSearchOutputs_of_distinct_valid_wit`

The exact one-variable product countermodel proves that every valid product witness encodes into the tautological one-variable CNF, yet the two explicit valid witnesses still have different messages:

`ambiguousAppendixDProductExactCNFReadoutData_not_singleMessageReadout,      ambiguousAppendixDProduct`

So exact CNF realization is not a repair theorem either. It only moves the burden to the same semantic locked-core claim: prove the actual product relation has no two valid witnesses over the same public syntax with different messages. `PNPv13AppendixDProductBijectionCNFBridge.lean`

tests the strongest natural version of this repair: require satisfying CNF assignments and valid product witnesses to round-trip through extraction and encoding. Lean proves that this stronger bijective interface still gives exactly the same boundary:

`AppendixDProductBijectionCNFReadoutData.singleMessageReadout_iff_validProductWitnessSingleMessage.`

Thus bijectivity does not manufacture readout constancy; it merely identifies CNF readout constancy with single-message constancy of valid product witnesses. The one-variable product model is made bijective by encoding a witness as its Boolean message and extracting the corresponding canonical witness. It still proves

`bijectiveAmbiguousAppendixDProduct_not_singleMessageReadout, bijectiveAmbiguousAppendixDProduct.`

So the remaining manuscript burden is now insensitive to this whole family of CNF-realization repairs: unrestricted, exact, and bijective CNF interfaces all reduce to the same missing theorem, namely global locked-core message rigidity for the actual Appendix D construction. The deterministic readout predicate in Definition 4.4 is now separated as well. The refinement `DeterministicReadoutLockedLayerData` records a readout state, a predicate $\text{Ready}(\text{state}, M)$, and determinism of that predicate for each fixed state. Lean proves

`DeterministicReadoutLockedLayerData.lockedLayerRigidity_of_acceptedReadStateConstancy,`

so deterministic readout plus constancy of the accepted readout state implies Hypothesis 4.9 and therefore the arbitrary-output SAT-search theorem. But Lean also proves a one-variable countermodel:

`ambiguousOneVarDeterministicReadoutLockedLayerData_not_acceptedReadStateConstancy.`

In that model every readout state has a unique message, but there are two accepted readout states with different messages. Thus deterministic readout is not the missing theorem; the manuscript must prove a global cross-completion rigidity statement for the actual locked layer. The reusable negative criterion is now explicit as

`DeterministicReadoutLockedLayerData.not_acceptedReadStateConstancy_of_distinct_accepted_readStates`

Separately, unconditional one-coordinate half-success estimates do not imply a product small-success bound. Two identical half-mass events have half-mass intersection, not quarter-mass intersection. Thus the product theorem needs a genuinely sequential conditional hypothesis over admissible histories. The same Lean file now records this repair target positively on the four-point Boolean square: a first-step half-success bound plus a second-step conditional half-success bound implies the two-step product bound. The correlated model fails exactly the second, conditional premise. The separate finite switching module generalizes this to arbitrary finite histories: if every successive event halves the current admissible history class, then the whole tower has the expected product bound. What remains is not product arithmetic, but an instantiation theorem for the actual switched histories. The same generic layer also proves the corresponding failure principle: one failed prefix half-step blocks sequential admissibility for the entire remaining history. The newest monotonicity lemmas also rule out a spurious edge case: if the current history class is already empty, the next prefix half-step is automatic. Hence real blockers must occur at positive-mass conditioning prefixes. The generic layer now also records the global contrapositive: if the final intersection is too large for the claimed tower, then no sequential half-admissibility proof can exist at all. The newest localization theorem strengthens this from a global audit to a specific missing premise: any such product-bound violation yields a decomposition `pre ++ E :: suffix` whose accumulated prefix fails the conditional half-step. The v13 layer lifts the same statement from event lists to switched-coordinate lists, so a failed product count identifies an exact switched coordinate whose induced success event does not halve the current success history. The v13 layer also specializes the product check to abstract atom

ledgers, semantic concrete-cell certificates, and ordered fixed-field certificates. Thus a proposed switched list must first pass the tower-product count check before any finer cell structure can rescue it. The v13 instantiation layer now sharpens this further. Lean first proves a useful negative interface fact: an abstract atom ledger with arbitrary per-cell counts is equivalent to the generic prefix half-step, because a successful half-step can always be represented by one summary cell. It then proves the same collapse for an existential “concrete” cell-map certificate: the constant cell map is enough whenever the prefix half-step already holds. The latest Lean theorem lifts this prefix collapse through the whole ordered switching list: both the abstract atom-ledger switching interface and the existential concrete-cell switching interface are equivalent to generic sequential half-admissibility of the induced success events. So a real manuscript instantiation cannot be an existential over cell maps. It must instead name the actual finite history field before each switched coordinate and prove cellwise half-success for that fixed field. The newest atom-ledger structure theorem clarifies the role of the concrete cell map: once a finite `cellOf` is supplied, the prefix and next-success decomposition equations are automatic finite partition identities. Thus decomposition is not an independent escape hatch; the remaining mathematical burden is the cellwise half-success inequality for the actual cells. The fixed-field interface records this obligation: the supplied field’s cells must decompose both the current success prefix and the prefix plus the next success event, and the next success event must occupy at most half of each cell. This has now been rewritten in Lean in the more diagnostic form `v13FieldPrefixInstantiation_iff_success_le_failure`: after the finite cell is fixed, a certificate exists exactly when every cell has at least as many next-failure prefix points as next-success prefix points. The supporting theorem `v13ConcretePrefixCount_eq_success_add_failure` proves that this is not a new assumption but the exact split of each cell’s prefix mass into success and failure parts. This fixed-field condition is strictly stronger than the global prefix half-step: on the Boolean square, first-coordinate success is globally half-admissible from the empty history, but the fixed field that reveals that same first coordinate has a ‘true’ cell where success has conditional mass 1, so no fixed-field certificate exists. Lean now also lifts this fixed-field premise to an ordered fielded switching list: each step carries its own supplied field, and the full fixed-field instantiation is equivalent to the recursive list of these cellwise half-success inequalities. Thus the fixed-field decomposition equalities are no longer open proof obligations; the remaining fielded burden is exactly the half-success ledger over the named cells. A full list of fixed-field certificates implies the generic sequential half-admissibility and hence the finite tower product bound. Conversely, a single failed fixed-field prefix certificate blocks the whole remaining fielded suffix, and the same Boolean first-coordinate one-step model instantiates that blocker. The newest fixed-field theorem abstracts the reason: any positive-mass cell on which the next success event is total violates the half-success inequality. In particular, if the supplied field reveals the next success event itself, then a fixed-field certificate can exist only when that next success prefix has zero mass. The latest necessary-condition theorem gives the same fact in positive form: under any valid fixed-field certificate, every cell containing a successful point must also contain an unsuccessful prefix point. A singleton successful atom is therefore already fatal. The newest strengthening generalizes the success-revealing-field blocker: `V13HistoryFieldDeterminesSuccessOn` says that every field cell is homogeneous for the next success event on the current history support, and the corresponding positive-success blocker proves that any such field is certifiable only when the next-success prefix has zero mass. The older global predicate `V13HistoryFieldDeterminesSuccess` is now just a special case. Thus a proposed v13 history field cannot already decide the next success bit after conditioning on any positive-mass prefix; it must leave enough same-cell counterexamples to satisfy the success-versus-failure balance. Lean also checks the repair-resistant distinction: on the Boolean square, the second-coordinate field does *not* globally determine first-coordinate success, but after conditioning on the diagonal history event it does determine that success bit, and the positive diagonal success prefix has no fixed-field certificate or same-cell failure matching. The Boolean first-coordinate field now fails by the global special case, not only by an arithmetic calculation on one named cell. The blocker has also been lifted from the head of a fielded suffix to an arbitrary later position: a failed fixed-field prefix certificate after any preceding fielded list blocks the whole original list, with the current history equal to the old history extended

by the preceding success events. In the Boolean square, the one-cell field certifies the first half-success cut, but the first-coordinate field is still impossible as the second repeated-coordinate field because it determines the repeated success event on a positive-mass prefix. The newest interface exposes the same burden constructively: `V13FieldFailureMatching` requires, for every supplied field cell, an injection from successful prefix points into unsuccessful prefix points in that same cell. Lean proves this matching interface equivalent to the fixed-field certificate, and the recursive fielded version is equivalent to the whole fixed-field switching instantiation. Thus a proposed repair can now be asked for an explicit same-cell matching at each switched position, not merely a global counting slogan. The matching interface also gives the contrapositive in the form needed for audit: if the accumulated next-success prefix has positive mass and the recursive matching proof exists, then the supplied field at that position cannot determine the next success bit on the accumulated history support. The newest pointwise extraction theorem makes the same obligation more operational: for every successful prefix point, a recursive matching proof yields an actual failed prefix point in the same supplied field cell. Consequently, one successful point whose accumulated-history cell contains only successes already blocks both the matching formulation and the fixed-field certificate. The diagonal Boolean-square example now also fails by this pointwise criterion, not only by count arithmetic. The same `v13`-specific module also checks the smallest positive-mass failure mode for sequential repetition, and this is now a general finite theorem rather than only a Boolean-square calculation: if an event has positive mass after the current history and the next switched step repeats that same exact-success event, then the second prefix half-step is false. In the Boolean-square instance the first success prefix has count 2, the repeated-success prefix still has count 2, and the second prefix half-step is false. Thus no `v13` switching certificate can exist for any positive-mass repeated two-step switching claim. The Boolean-square example also fails the global product-bound audit: two identical half-mass success events still have half-mass intersection, so the required quarter-mass tower bound is false. The product-bound contrapositive now reaches the operational fixed-field matching layer as well: a tower-product violation rules out the recursive same-cell matching proof directly. In particular, even using the non-revealing one-cell field for both repeated Boolean-square cuts cannot instantiate the fixed-field route; the fielded product bound and the recursive matching obligation both fail. The newest localization theorem also turns either a failed recursive matching proof or a tower-product violation into a concrete failed fielded position $pre + +item :: suffix$, with the accumulated history computed from pre . For the repeated one-cell Boolean-square model, Lean identifies the exact obstruction: after the first success event, the one-cell field's only accumulated-history cell has next-failure count 0 and next-success count 2, so it is the concrete bad cell for the second repeated success test.

The route synthesis module now separates this blocker from the stronger claim that every possible `v13` repair is dead. It names the exact route-class assumptions: `V13RepeatedPositivePivotFrom` for an abstract switched list, `V13RepeatedPositiveFieldedPivotFrom` for a fixed-field list, and `V13FieldedBadCellFrom` for a localized same-cell balance failure. Under those hypotheses Lean proves that the corresponding abstract, fixed-field, and recursive matching formulations are blocked. Conversely, if a proposed manuscript instantiation avoids the repeated-positive-pivot pattern, Lean now names the repair theorem it owes: prove the recursive `V13FieldSwitchingFailureMatchingFrom` obligation for the actual ordered fielded list. That theorem entails both `V13NoRepeatedPositiveFieldedPivotFrom` and `V13NoFieldedBadCellFrom`, and it also supplies the finite tower product bound. Thus the current conclusion is a decision tree: either the construction satisfies the repeated-pivot or bad-cell hypotheses and is blocked, or it must provide the explicit cellwise same-cell matching repair theorem.

Lean now also closes the most direct renaming repair for repeated pivots. The module `PNPv13ForcedStepObstruction.lean` proves the finite-count fact behind it: if an accumulated history has positive mass and pointwise implies the next success event, then adding that event leaves the history count unchanged, so the next event cannot be a conditional half-step. This is not limited to literal event repetition. The fielded `v13` theorem packages the condition as `V13ForcedPositiveFieldedStepFrom` and proves that it blocks both `V13FieldSwitchingInstantiatedFrom` and the equivalent recursive same-cell matching formu-

lation. The Boolean-square anchor makes the distinction concrete: first-coordinate success and second-coordinate success are both valid one-cell half-cuts, but after both are imposed the diagonal success event is forced. The diagonal event is globally half-mass, and it is not the same success event as either coordinate event, yet it cannot serve as the third conditional cut.

One natural refinement repair is now tested separately. The module `PNPv13FieldRefinement.lean` defines a genuine finite field refinement as a projection from fine cells to coarse cells commuting with the pointwise cell maps. It proves that a coarse bad cell cannot be hidden by such a refinement: if the coarse cell has fewer next-failure points than next-success points, then some fine cell over it has the same strict imbalance. Consequently, subdividing a bad supplied cell cannot produce a fixed-field prefix certificate for the same history and switched step. This does not rule out changing to a non-refining adaptive field, but that move is now exactly the stronger repair theorem already named above: prove the recursive same-cell matching obligation for the actual new fielded list.

There is now a second refinement-side obstruction, aimed at a different escape hatch. `PNPv13FieldRefinementDeterminesSuccessObstruction.lean` proves that support-relative success determination is monotone under refinement: if a coarse field already determines the next success event on the current history support, then every finer field does too, because each fine cell lies inside one coarse cell. So refinement cannot rescue a field that was already revealing too much. The Boolean-square regression makes this concrete by starting from the diagonal history event: the second-coordinate field already determines repeated first-coordinate success on that conditioned support, and refining it all the way to the identity field still blocks both the fixed-field certificate route and the equivalent recursive same-cell matching route.

The next refinement-side file removes the remaining ambiguity about what a refinement repair would have to accomplish. The module `PNPv13FieldRefinementFailedPrefixObstruction.lean` proves the abstract failure-monotonicity theorem: once the coarse field already fails the fixed-field prefix obligation at a given accumulated history, every genuine refinement of that field fails there as well. So refinement does not merely preserve the two previously isolated causes of failure; it preserves failed prefixes as such. The Boolean-square regression again uses the diagonal history followed by repeated first-coordinate success: the second-coordinate field already fails at that conditioned prefix, and refining it to the identity field still blocks both fixed-field route formulations.

Assessment after the v13 switching packets: the formal work does not prove that no conceivable v13-style switching strategy can work, because one residual positive target remains—name the actual manuscript history fields and prove their recursive cellwise half-success bounds. But it does close the main bookkeeping escape hatches. Abstract ledgers and existential concrete cell maps add no strength beyond generic sequential half-admissibility; decomposition by finite cells is automatic; and fixed fields reduce exactly to the cellwise half-success inequalities, with positive-mass total-success cells, repeated success events, and forced later success events blocked by small finite models.

At the same time, the optimistic Lean work now isolates the precise positive interface one would need for the switching lemma to go through: a *small explicitly encoded hypothesis family* whose realized classifier class is bounded by its code budget, together with a perspective-relative notion of accessible input surface inspired by the existing Hyperseed formalization.

These results do not by themselves disprove the whole paper. They do isolate an essential gap: the switching/locality step must be strengthened by a real theorem showing that the specific switched predictors lie in an exceptionally small subclass, not merely in the class of all local functions.

Contents

1	Context	13
2	Current Lean Files	13

3	Two Parallel Programs	15
3.1	The Blocker Program	15
3.2	The Optimistic Program	16
3.3	Hyperseed Perspective Surface	17
3.4	Meredith/HML Observable Surface	17
4	Optimistic Background Theorem	17
5	Obstruction I: Log-Radius Locality Is Already Polynomial	18
5.1	Interpretation	18
6	Obstruction Ib: Fixed-Width VV Isolation Does Not Follow from the Cited Lemma	18
6.1	Lean theorem	19
6.2	Why this hits the manuscript	19
6.3	Consequence	20
7	Obstruction Ic: Pairwise-Independent Columns Do Not Imply Universal Hashing	20
7.1	Lean theorem	20
7.2	Why this matters	21
8	Obstruction Id: RHS Bias Is Irrelevant Once The Label Maps Are Uniform	21
8.1	Lean theorems	21
8.2	Why this matters	21
9	Obstruction II: The Full Local Rule Class Is Enormous	38
9.1	Interpretation	39
10	What These Results Break	39
11	Obstruction III: Zero-Margin Symmetrization Cannot Concentrate	39
11.1	Lean counting lemma	40
11.2	Application to the Current Symmetrization Wrapper	40
11.3	Consequence	40
12	Obstruction IIIb: The Exact Post-Switch Input Is Not T_i-Invariant	41
12.1	Lean theorem	41
12.2	Why this matters	41
12.3	Consequence	42
13	Obstruction IIIc: The Exact Manuscript Fork Already Lands in the Symmetry Budget	42
13.1	Lean theorems	42
13.2	Why this matters	43
14	Obstruction IV: Involution-Invariant Feature Collapse Forces Exact Neutrality	43
14.1	Lean theorem	43
14.2	Why this hits the charitable calibration patch	44
14.3	Consequence	44

15 Obstruction IVb: Invariant Feature Fibers Have Zero Margin	45
15.1 Lean theorem	45
15.2 Why this hits the soft-score repair	45
15.3 Consequence	46
16 Obstruction IVc: Symmetry-Respecting Weights Still Have Zero Margin	46
16.1 Lean theorem	46
16.2 Why this closes the non-uniform averaging reply	46
16.3 Consequence	47
17 Obstruction IVd: Residual Symmetric Slices Still Stay At Chance	47
17.1 Lean theorem	47
17.2 Why this hits the “controlled non-invariant info” reply	48
17.3 Consequence	48
18 Obstruction IVe: Advantage Is Bounded By Symmetry-Broken Mass	48
18.1 Lean theorem	48
18.2 Why this hits the last soft escape	49
18.3 Consequence	49
19 Obstruction IVf: Invariant Soft Scores Have Zero Signed Signal	50
19.1 Lean theorem	50
19.2 Why this hits the richer invariant-scoring repair	50
19.3 Consequence	51
20 Obstruction IVg: Invariant-Score Signal Comes Only From Weight Asymmetry	51
20.1 Lean theorem	51
20.2 Why this narrows the weighting rescue	52
20.3 Consequence	52
21 Obstruction IVh: Side Channels Help Only On Orbit-Resolving Mass	52
21.1 Lean theorem	53
21.2 Why this hits the controlled-side-channel rescue	53
21.3 Consequence	53
22 Obstruction IVi: Any Domination Gap Forces Matching Resolution	54
22.1 Lean theorem	54
22.2 Why this matters	58
22.3 Consequence	58
23 Obstruction V: Short Programs Still Cover the Full Log-Width Local Rule Class	58
23.1 Lean theorem	59
23.2 Binary-log consequence	59
23.3 Consequence	59
24 Obstruction VI: Exact Metagraph Quotients Have Interface-State Explosion	59
24.1 Lean theorem	60
24.2 Why this matters	60
24.3 Consequence	61

25 Obstruction VII: Tree-Likeness Alone Does Not Yield a Small Message Class	61
25.1 Lean theorem	61
25.2 Why this matters	62
25.3 Consequence	62
26 What These Results Do Not Yet Prove	62
27 Stop-Grade Synthesis Ledger	62
28 Next Attack Surface	81
29 Lean Excerpts	135
30 Conclusion	135

1 Context

The proof strategy under review is the October 2025 manuscript “ *$P \neq NP$: A Non-Relativizing Proof via Quantale Weakness and Geometric Complexity.*” At a high level, the paper proposes the following chain:

1. Construct a masked random Unique-SAT ensemble.
2. Show that every short polynomial-time decoder can be wrapped into a predictor that is local on a constant fraction of blocks.
3. Use neutrality and sparsification to show local predictors succeed only with exponentially small probability across many independent blocks.
4. Convert that small-success statement into a lower bound on time-bounded Kolmogorov complexity.
5. Contradict the $P = NP$ upper bound obtained from Unique-SAT self-reduction.

The current crux-first effort does not attempt the full chain at once. Instead it attacks the most fragile inference in the middle:

“Switched predictor is local at logarithmic radius” $\implies$ “switched predictor belongs to a polylogarithmic hypothesis class.”

That implication is not automatic. It needs either a structural theorem about the switched predictor or a direct circuit/description-length bound. The Lean work below shows why.

2 Current Lean Files

The current Lean modules under `Mettapedia/Computability/PNP/`:

File	Role
HypothesisClass.lean	optimistic code-indexed hypothesis
InfinitaryHMLObstruction.lean	HML over infinitely many minimizers
InfinitaryHMLObstructionRegression.lean	regression checks for finite-decision
LocalityObstruction.lean	Log-radius locality is larger than
CompressionObstruction.lean	Arbitrary local rules do not admit
FixedWidthIsolationObstruction.lean	fixed-width hashing isolates only
PairwiseSurvivorMoments.lean	pairwise first and factorial-second
PairwiseCandidateBridge.lean	candidate-wise 0/1 hits plus polynomial
PairwiseColumnsObstruction.lean	pairwise-independent columns
RhsBiasIrrelevance.lean	once label maps are uniform, a
TwoUniversalRhsIrrelevance.lean	exact finite 2-universality of the
ConditioningObstruction.lean	conditioning can collapse uniform
ApproximateDecorrelationObstruction.lean	distinguishing conditioned regression
ApproximateDecorrelationObstructionRegression.lean	regression checks for coupled,
SymmetrizationObstruction.lean	unbiased majority does not beat
PostSwitchInputObstruction.lean	exact $u = (z, a_i, b)$ is T_i -invariant
PostSwitchForkObstruction.lean	exact T_i -fork: invariant (z, a_i)
PostSwitchForkObstructionRegression.lean	regression checks for the exact
VisiblePostSwitchSurface.lean	isolates the exact visible surface
ExactSwitchedFamily.lean	isolates the exact switched-family
ExactVisibleCompressionObstruction.lean	if the exact family spans all ex-
OrbitNeutralityObstruction.lean	involution-invariant feature com-
FiberNeutralityObstruction.lean	invariant feature fibers have ex-
WeightedFiberNeutralityObstruction.lean	symmetry-respecting weights s-
InvariantScoreObstruction.lean	invariant soft scores have zero
WeightAsymmetryObstruction.lean	invariant-score signal comes on
WeightAsymmetryObstructionRegression.lean	regression checks for invariant-
SideChannelResolutionObstruction.lean	side channels help only on orb-
ResolutionDemandObstruction.lean	any doubled or strict half-accu-
ResolutionDemandObstructionRegression.lean	regression checks for positive-a-
ResidualSideInformationResolutionCost.lean	source-determined residual sid-
RandomizedResidualObstruction.lean	randomized residuals reduce to
RandomizedResidualObstructionRegression.lean	regression checks for the rand-
PostSwitchRandomizedResidualObstruction.lean	exact post-switch support blo-
PostSwitchRandomizedResidualObstructionRegression.lean	regression checks for the post-
PostSwitchResidualWitness.lean	concrete active-orbit witness o-
PostSwitchResidualWitnessRegression.lean	regression checks for the exact
ResidualSymmetryObstruction.lean	any unresolved symmetric slic-
AsymmetryBudgetObstruction.lean	advantage is bounded by the s-
AsymmetryBudgetObstructionRegression.lean	regression checks for the asym-
GlobalWeaknessObstruction.lean	a family decoded only from on-
GlobalWeaknessObstructionRegression.lean	regression checks for distinctio-
ShortProgramObstruction.lean	polynomial-size programs alrea-
MetagraphObstruction.lean	exact metagraph/interface qu-
MetagraphObstructionRegression.lean	regression checks for finite qu-
TreeMessageObstruction.lean	fixed tree architectures already
TreeMessageObstructionRegression.lean	regression checks for tree evalu-
ClockedKpolyBridge.lean	finite-counting kernel for clock-
ClockedKpolyBridgeRegression.lean	regression checks for the clock-
ClockedKpolyTargetInterface.lean	exact-surface bit-budget target
ClockedKpolyTargetInterfaceRegression.lean	regression checks for the clock-

These compile in the current `mettapedia` workspace via `lake env lean` (with selected files also built to `.olean` artifacts).

3 Two Parallel Programs

3.1 The Blocker Program

The blocker program is adversarial. It asks:

1. Is the visible radius already too large for any naive `poly(log m)` claim?
2. Is the full class of local rules too large for a short description?
3. Can the manuscript’s fixed $k = c_1 \log m$ VV layer really achieve $\Omega(1/m)$ uniqueness probability on large solution sets, or does fixed-width isolation already collapse once the expected survivor count is bigger than 1?
4. Is the manuscript’s weaker *pairwise-independent columns* language alone enough to justify universal hashing, or can higher-order linear dependencies still make a nonzero difference hash to zero deterministically?
5. If the hash labels are already uniform or pairwise-uniform, can the δ -biased right-hand side still change the first and pair hit moments at all?
6. Even if the pre-conditioning label map is uniform, can later conditioning on uniqueness still collapse the local label distribution completely?
7. Even if two coordinates are independent before conditioning, can a uniqueness-style conditioning event make them dependent unless a separate decorrelation theorem is supplied?
8. Does majority symmetrization fail whenever the local conditional mean has zero margin?
9. Is the exact post-switch input $u = (z, a_i, b)$ really preserved by the paper’s involution, or only in the zero-column case $a_i = 0$?
10. Once that exact fork is made explicit, does the charitable repair reduce either to zero invariant-feature signal on (z, a_i) or to advantage bounded by the mass on nonzero VV columns?
11. Does the obvious “drop the bad coordinate” calibration patch collapse any feature-only classifier back to exact neutrality?
12. Does the softer “estimate the conditional mean on invariant features” repair still have exact zero margin on every retained feature fiber?
13. Does non-uniform but involution-symmetric weighting avoid that zero-margin collapse?
14. Can an arbitrary invariant soft score recover directional signal, or does it also have zero signed correlation with the target?
15. If the weighting is not exactly symmetric, does invariant-score signal come only from the antisymmetric part of that weighting?
16. If one keeps a non-invariant side channel, is total advantage still bounded by the mass where that side channel actually resolves involution pairs?

17. Conversely, does any claimed domination advantage force a comparably large orbit-resolving mass for that side channel?
18. If one keeps some controlled non-invariant information, do residual unresolved slices still remain exact-chance unless their mass is proved negligible?
19. Can a small symmetry-breaking slice still yield a large global advantage, or is the excess over chance quantitatively bounded by the symmetry-broken mass?
20. Does “short global decoder” alone still allow the full local-rule class at log-width?
21. Does an exact finite-state metagraph/message quotient on a log-width interface already require too many states?
22. Does tree-likeness alone force a small exact message/interpreter class?
23. Does arbitrary SAT-search output recover a unique hidden message without a theorem that all satisfying witnesses have the same readout?
24. Do unconditional half-success marginals imply a product small-success bound without sequential conditional admissibility?

The current Lean answer to all listed questions is “yes.” Therefore the published proof cannot be accepted *as written* unless it supplies additional structural theorems. The short-global-decoder item is now sharpened in a weakness-specific form: `GlobalWeaknessObstruction.lean` proves that if every member of an indexed predictor family factors through the single global `distinctionWeakness` scalar, then the family is not surjective onto all Boolean classifiers on any nontrivial state space:

`not_surjective_familyFactorsThroughDistinctionWeakness_of_nontrivial.`

The same statement is proved for `nonDistinctionWeakness`. Thus the formalized GSLT weakness bridge cannot be used as a full encoded-family readout unless the route supplies a genuinely per-instance or per-code observable rather than only a global evidence summary.

3.2 The Optimistic Program

The optimistic program asks what would be enough to save the route. The current working answer is:

locality alone is not enough, but *locality plus explicit compressibility* may be.

Concretely, the switching theorem would need to produce not merely a local rule on a radius- $O(\log m)$ neighborhood, but a family of predictors realized by a code space of size at most $\text{polylog}(m)$ bits. In that case the realized hypothesis class is small, and standard finite-class learning bounds become plausible again.

The new module `HypothesisClass.lean` formalizes exactly this interface:

1. an *encoded family* is a decoder from a finite code space into classifiers;
2. the realized class has cardinality at most the size of the code space;
3. therefore an s -bit family realizes at most 2^s classifiers;
4. no such family can surject onto the full local-rule class once $s < 2^n$.

This is the correct positive abstraction: if this switching argument is salvageable, it must land in one of these small encoded subclasses rather than in the full local-rule space.

3.3 Hyperseed Perspective Surface

The Hyperseed formalization in `Mettapedia/Hyperseed/` already provides a useful observer-relative interface:

- a *perspective* determines what is observable,
- a budget determines what is affordable,
- a guard determines what is admitted.

The optimistic PNP background layer reuses this by treating a decoder’s *available input surface* as a subtype of the Hyperseed `availableRegion`. This does not solve switching. It does provide a clean ground-up semantics for what a bounded observer can even use as input, without baking in a bespoke notion of locality for every later argument.

3.4 Meredith/HML Observable Surface

The Meredith/HML side of the route has a separate finite-presentability obligation. The module `InfinitaryHMLObstruction.lean` shows that if the type of certified minimal contexts is infinite, then even the depth-1 HML fragment is infinite. The rho-calculus instance already has an infinite ladder of such minimal contexts, so its depth-1 fragment and full HML formula type are both infinite.

The original formal obstruction was one-sided: no finite code type can decode surjectively onto those formula surfaces. The newest strengthening records the dual finite-encoding boundary:

`not_injective_depthFragment_one_to_finite_code,      not_injective_hmlFormula_to_finite_code.`

The concrete rho versions,

`not_injective_rhoDepthFragment_one_to_finite_code,      not_injective_rhoHMLFormula_to_finite_cod`

block the repair move of replacing an infinite observable surface by a finite injective code without first proving a genuine finite quotient theorem for the actual contexts used by the route.

4 Optimistic Background Theorem

The basic positive theorem is elementary but indispensable:

Theorem 4.1 (Code budget bounds realized hypothesis class). *Let H be a family of Boolean classifiers realized by decoding an s -bit code:*

$$\text{decode} : \{0, 1\}^s \rightarrow \mathcal{F}.$$

Then the realized class satisfies

$$|\text{range}(\text{decode})| \leq 2^s.$$

This is formalized in `HypothesisClass.lean` as `BitEncodedClassifierFamily.card_realized_le`.

The contrapositive is equally important:

Theorem 4.2 (Short codes cannot cover the full local-rule class). *If $s < 2^n$, then no s -bit decoder can surject onto the full set of Boolean rules on n visible bits.*

This is formalized as `not_surjective_decode_to_fullLocalRule_of_lt`.

These statements are not the final theorem. They are the right *background theory* for the optimistic route: they tell us exactly what shape of compression theorem the switching step must deliver.

5 Obstruction I: Log-Radius Locality Is Already Polynomial

Suppose a factor graph has bounded degree $d \geq 2$, and a predictor is allowed to inspect a radius- r neighborhood around one target bit. A rough but unavoidable upper/lower size model is

$$\text{visible nodes} \asymp d^r.$$

If $r = \Theta(\log m)$, then this is

$$d^{c \log m} = m^{c \log d},$$

which is polynomial in m , not polylogarithmic in m .

The Lean module `LocalityObstruction.lean` records this in several exact forms.

Theorem 5.1 (Binary-log lower bound). *For every $d, m \in \mathbb{N}$ with $d \geq 2$,*

$$m \leq d^{\log_2 m + 1}.$$

This is formalized as the theorem `inputSize_le_neighborhoodSize_at_binaryLogRadius`.

Theorem 5.2 (Power-of-two specialization). *For every $d, n \in \mathbb{N}$ with $d \geq 2$,*

$$2^n \leq d^{\log_2(2^n)} = d^n.$$

This appears as `powerOfTwo_le_neighborhoodSize_at_exactBinaryLogRadius`.

Theorem 5.3 (Asymptotic dominance). *Fix $k \in \mathbb{N}$ and real constants $d > 1$, $c > 0$. Then*

$$(\log x)^k = o(d^{c \log x})$$

as $x \rightarrow \infty$.

This is formalized as `polylog_isLittleO_logRadiusNeighborhood`, and the corresponding negative Big-O statement appears as `logRadiusNeighborhood_not_isBigO_polylog`.

5.1 Interpretation

This does *not* say that every local rule requires polynomial-size circuits. It says something weaker but still decisive: locality at logarithmic radius, by itself, does not justify a $\text{poly}(\log m)$ complexity claim. The radius already exposes a polynomial-sized input window.

So if the paper wants a tiny post-switch hypothesis class, it must prove a *compression theorem* about the specific local rules that arise after switching. That is a separate and nontrivial burden.

6 Obstruction Ib: Fixed-Width VV Isolation Does Not Follow from the Cited Lemma

The manuscript's main ensemble fixes

$$k = c_1 \log m$$

in the VV layer. But the cited isolation lemma proving uniqueness probability $\Omega(1/m)$ chooses k *uniformly across all scales* $0, \dots, m-1$. Those are not the same claim.

The new Lean module `FixedWidthIsolationObstruction.lean` packages the exact finite bound one needs at fixed width.

6.1 Lean theorem

Theorem 6.1 (Variance-budget bound for exact isolation). *Let Y be the number of candidates retained by a random hash choice, on a finite hash space. If*

$$\mathbb{E}[Y] = \mu > 1 \quad \text{and} \quad \text{Var}(Y) \leq \mu,$$

then

$$\Pr[Y = 1] \leq \frac{\mu}{(\mu - 1)^2}.$$

This is formalized as `exactOneProb_le_of_variance_budget`.

The new companion module `PairwiseSurvivorMoments.lean` formalizes the abstract bridge from pairwise-survivor moments to this variance budget. In that file, a finite model satisfying

$$\mathbb{E}[Y] = \mu \quad \text{and} \quad \mathbb{E}[Y(Y - 1)] \leq \mu^2$$

is shown to obey $\text{Var}(Y) \leq \mu$, and therefore the same exact-isolation upper bound. This separates the remaining manuscript obligation cleanly: one still has to prove that the actual fixed-width VV sampler instantiates those pairwise moments.

The newer bridge module `PairwiseCandidateBridge.lean` takes one more step toward the actual VV picture. It proves that if the retained-candidate indicators are 0/1-valued, all have the same marginal hit rate p , and distinct candidates satisfy the pairwise upper bound

$$\mathbb{E}[X_a X_b] \leq p^2, \quad a \neq b,$$

then the total survivor count satisfies the exact abstract interface of `PairwiseSurvivorMomentModel`. Concretely:

- `candidateHitCount_sum` proves the first moment formula;
- `candidateHitCount_factorialSecondMoment_le` proves the factorial second-moment bound;
- `pairwiseCandidateMomentModel` packages these into the survivor-moment model used by the fixed-width isolation theorem.

So the remaining gap has narrowed again. To recover the manuscript’s fixed-width uniqueness claim, one now needs an end-to-end theorem that the actual VV hash family satisfies these candidate-wise marginal and pairwise bounds at the fixed logarithmic width used later in the paper.

6.2 Why this hits the manuscript

In the source text, the supporting VV lemma states the classical bound

$$\Pr[|S \cap h^{-1}(0^k)| = 1] \geq c/m$$

only after choosing k uniformly from $0, \dots, m - 1$. But the actual ensemble later fixes $k = c_1 \log m$ and then still says isolation succeeds with $\Omega(1/m)$ probability.

That gap is not cosmetic. Under the standard pairwise-independent VV heuristic, if a formula has N satisfying assignments and the fixed hash width is k , then the retained-count random variable has mean

$$\mu = N/2^k$$

and variance at most μ . The Lean theorem above therefore yields

$$\Pr[\text{exact isolation}] \lesssim \frac{1}{\mu} = \frac{2^k}{N},$$

up to a universal constant once μ is bounded away from 1.

So with fixed $k = O(\log m)$, the sampler can isolate with noticeable probability only when the pre-hash solution set is already $N = O(2^k) = \text{poly}(m)$. The manuscript, however, only cites a much weaker upper bound of the form $N \leq 2^{\alpha m}$, which still permits an exponential solution set. On such instances, fixed logarithmic width gives exponentially small exact-isolation probability rather than $\Omega(1/m)$.

6.3 Consequence

This means the paper now needs a real theorem of one of the following forms:

1. on the relevant random-3CNF mass, the satisfiable formulas entering the VV stage already have only $\text{poly}(m)$ satisfying assignments; or
2. the ensemble does not actually use fixed $k = c_1 \log m$, and the later local-input and finite-alphabet claims must be rewritten accordingly.

Without one of those repairs, the promised block distribution D_m is not supported by the cited VV argument.

7 Obstruction Ic: Pairwise-Independent Columns Do Not Imply Universal Hashing

The ensemble definition samples A from “any 2-universal distribution with pairwise-independent columns,” and the later robustness discussion repeatedly foregrounds the *pairwise-independent columns* part. The new Lean module `PairwiseColumnsObstruction.lean` shows that this weaker condition is not enough on its own.

7.1 Lean theorem

Theorem 7.1 (Pairwise columns need not be universal). *There exists a one-bit linear hash family with three columns such that:*

1. *each pair of columns is jointly uniform over $\mathbb{F}_2^2$; but*
2. *some nonzero input difference hashes to 0 on every seed.*

Hence pairwise-independent columns do not by themselves imply the universal-hashing bound required for VV isolation.

This is formalized by the bijectivity theorems `col12_bijective`, `col13_bijective`, and `col23_bijective`, together with `threeColumnHash_badDifference` and `threeColumnHash_not_universal`.

7.2 Why this matters

This does *not* refute a hash family that is genuinely 2-universal. It does refute a common informal shortcut: one cannot replace real universal hashing by mere pairwise independence of the columns. Higher-order linear dependencies can survive that weaker condition and completely destroy the needed collision bound.

So the manuscript now owes a sharper statement here too:

1. either prove that the actual fixed-width VV family is genuinely 2-universal in the sense needed for the candidate-wise moment bridge; or
2. stop appealing to pairwise-independent columns as if that weaker property alone carried the isolation argument.

8 Obstruction Id: RHS Bias Is Irrelevant Once The Label Maps Are Uniform

The manuscript samples the VV right-hand side b from a δ -biased source and suggests that this robustness can be folded into the collision/isolation analysis. The new Lean module `RhsBiasIrrelevance.lean` isolates the exact point where that can and cannot matter.

8.1 Lean theorems

Theorem 8.1 (Uniform labels erase single-hit bias). *Let L be a finite label space with any probability weight w on L , and suppose a finite seed space S splits as $S \cong \{1, \dots, d\} \times L$ with $d > 0$, so that a candidate's label map is exactly the projection onto L . Then*

$$\frac{1}{|S|} \sum_{s \in S} w(f(s)) = \frac{1}{|L|}.$$

This is formalized by `average_mass_of_uniform_labels`.

Theorem 8.2 (Jointly uniform labels erase pair-hit bias). *Under the same hypotheses, if a pair of candidates has jointly uniform labels $S \cong \{1, \dots, d\} \times L \times L$ with $d > 0$, then*

$$\frac{1}{|S|} \sum_{s \in S} \mathbf{1}_{f(s)=g(s)} w(f(s)) = \frac{1}{|L|^2}.$$

This is formalized by `average_joint_mass_of_uniform_label_pairs`.

8.2 Why this matters

This does not prove that the manuscript's actual fixed-width VV family has perfectly uniform label maps. It proves the sharper conditional statement: *if* the candidate-wise and pairwise label maps are genuinely uniform, then the choice of a δ -biased right-hand-side distribution is irrelevant to the first and pair-hit moments. The averaged values are exactly the same as under a uniform right-hand side.

So the manuscript cannot get extra collision-control power from δ -bias alone. Any real effect of the right-hand side must come from a failure of the underlying label maps to be uniform in the first place. This narrows the burden again:

1. either prove that the actual fixed-width family's label maps are *not* uniform in a way that helps the needed moment bounds; or
2. stop treating δ -bias itself as if it repaired the missing universal-hashing step.

The new bridge module `TwoUniversalRhsIrrelevance.lean` sharpens this one step further in the manuscript's own language: if the actual A -family is genuinely finite 2-universal, then the weighted single-hit and pair-hit averages are exactly the uniform values for *every* finite right-hand-side distribution w . So on the first two VV moments, the entire burden sits on the A -family, not on the δ -biased choice of b .

But even this sharper bridge is only a *pre-conditioning* statement. The new module `ConditioningObstruction.1` shows that this is the end of what one can get for free: conditioning a perfectly uniform label map on a single label fiber makes the conditional label distribution a point mass. So no argument may pass from pre-conditioning uniformity to the distribution conditioned on uniqueness without an additional theorem controlling that conditioning step.

The same file now formalizes the sharper v11-style warning using only four points. Under uniform counting on $\{0, 1\}^2$, the first-coordinate and second-coordinate truth events satisfy the cross-multiplied independence equation:

`conditioningBoolPair_unconditioned_first_second_independent.`

But after conditioning on the diagonal event $x_1 = x_2$, the conditional independence equation fails:

`not_conditioningBoolPair_conditioned_on_diagonal_first_second_independent.`

The reason is now isolated as a general finite-count theorem:

`countIndependentWithin_coupled_iff_degenerate_on_condition.`

If E and F are pointwise equivalent on the conditioning event C , then the cross-multiplied conditional independence equation holds if and only if either $C \cap E$ is empty or $C \cap \neg E$ is empty. The older nonconstant blocker is now just the positive/negative corollary:

`not_countIndependentWithin_of_coupled_nonconstant_on_condition.`

If E and F are pointwise equivalent on the conditioning event C , and both $C \cap E$ and $C \cap \neg E$ have positive finite count, then the cross-multiplied conditional independence equation cannot hold. The diagonal Boolean square supplies the smallest witness to the hypotheses, but the theorem applies to any uniqueness-style conditioning event that makes two formerly separate predicates determine one another while both truth values remain present. The complement-coded variant is now formalized by

`countIndependentWithin_anticoupled_iff_degenerate_on_condition`

and its positive/negative corollary

`not_countIndependentWithin_of_anticoupled_nonconstant_on_condition.`

Thus replacing a copied conditioned bit by its negation does not evade the obstruction: if $E \leftrightarrow \neg F$ throughout C , conditional independence again forces E to be empty or full on C . Any repair must show genuine residual randomness after conditioning, not just change the encoding of the determined

bit. The newest Lean refinement closes the corresponding arbitrary Boolean recoding escape. For any Boolean map $r : \{0, 1\} \rightarrow \{0, 1\}$, the theorem

`countIndependentWithin_boolRecoding_iff_degenerate_on_condition`

states that if F is exactly the conditioned recoding $r(E)$ on C , and r is nonconstant, then conditional independence again holds only in the same degenerate empty/full cases. Its contrapositive,

`boolRecoding_constant_of_countIndependentWithin_of_nonconstant_on_condition,`

says that if E has both truth values inside C and $F = r(E)$ remains conditionally independent of E , then r must be constant. So the proof route cannot repair the conditioning gap by applying a different deterministic Boolean post-processing to the copied conditioned bit; it needs a real theorem that the post-conditioned variable still contains residual randomness not determined by E . The classification is now exact rather than only one-sided:

`countIndependentWithin_boolRecoding_iff_constant_or_degenerate_on_condition.`

For deterministic Boolean recodings of a conditioned source bit, conditional independence holds precisely when the recoding is constant or the source bit is empty/full on the conditioning event. Equivalently, under positive mass on both $C \cap E$ and $C \cap \neg E$,

`countIndependentWithin_boolRecoding_iff_constant_of_nonconstant_on_condition.`

Thus the only deterministic recoding repair that survives the finite-count audit is the information-erasing constant one, which is useless for a route that needs a discriminating post-conditioned predictor. This does not refute a future proof of the required v11 decorrelation estimate. It does block the naive inference from pre-conditioning independence of F , A , or b to independence after conditioning on the USAT/uniqueness promise.

The latest Lean strengthening closes the immediate “richer deterministic label” variant of the same repair. For any decidable codomain α and any deterministic recoding $r : \{0, 1\} \rightarrow \alpha$, every output fiber

$$\{\omega : r(\mathbf{1}_{E(\omega)}) = y\}$$

is conditionally independent of the source bit E , on a nondegenerate conditioning event, if and only if that fiber does not distinguish the two source-bit values:

$$r(1) = y \iff r(0) = y.$$

This is formalized as

`countIndependentWithin_boolToAnyRecoding_fiber_iff_fiber_constant_of_nonconstant_on_condition.`

Thus changing the output type of a deterministic recoding of the same conditioned bit does not evade the audit: every output value that separates E -true states from E -false states yields a fiber still coupled or anti-coupled to the source.

The quantitative extension, `ApproximateDecorrelationObstruction.lean`, closes one immediate “make it approximate” variant on the same finite-count surface. It defines the two oriented cross-multiplied defects

$$\Delta_+(C, E, F) = |C| |C \cap E \cap F| - |C \cap E| |C \cap F|,$$

and

$$\Delta_-(C, E, F) = |C \cap E| |C \cap F| - |C| |C \cap E \cap F|.$$

A symmetric tolerance- τ approximate independence claim must bound both defects by τ . Lean proves that under coupling $E \leftrightarrow F$ inside C ,

$$\text{countIndependentWithinForwardGap_coupled} \implies \Delta_+(C, E, F) = |C \cap E| |C \cap \neg E|,$$

and under anti-coupling $E \leftrightarrow \neg F$ inside C ,

$$\text{countIndependentWithinBackwardGap_anticoupled} \implies \Delta_-(C, E, F) = |C \cap E| |C \cap \neg E|.$$

Consequently, the theorem

$$\text{not_countIndependentWithinTolerance_of_coupled_product_gt}$$

blocks any approximate repair whose tolerance is smaller than this explicit source-fiber product, with the analogous anti-coupled theorem for Δ_- . The same module now lifts this quantitative result to every decidable output fiber of an arbitrary deterministic recoding $r : \{0, 1\} \rightarrow \alpha$. If the fiber over y distinguishes the two source-bit values,

$$\neg(r(1) = y \iff r(0) = y),$$

then

$$\text{not_countIndependentWithinTolerance_of_boolToAnyRecoding_fiber_distinguishes_product_gt}$$

blocks any symmetric tolerance below the same product $|C \cap E| |C \cap \neg E|$. The obligation form is now also formalized:

$$\text{conditionedSourceFiberProduct_le_of_CountIndependentWithinTolerance_boolToAnyRecoding_fiber_di.}$$

It says that any successful approximate-independence certificate for an informative deterministic fiber forces

$$|C \cap E| |C \cap \neg E| \leq \tau.$$

The newest regression-pinned form removes one more escape hatch: if the two conditioned source-bit fibers are known only through lower bounds L_1 and L_0 , then

$$\text{lowerBounds_mul_le_of_CountIndependentWithinTolerance_boolToAnyRecoding_fiber_distinguishes}$$

forces $L_1 L_0 \leq \tau$. The direct contradiction form,

$$\text{not_countIndependentWithinTolerance_of_boolToAnyRecoding_fiber_distinguishes_lowerBounds_mul_gt}$$

rules out the same approximate-independence certificate whenever $\tau < L_1 L_0$. Thus a repair cannot separately claim substantial mass on both conditioned sides and a small approximate-independence tolerance for an informative deterministic fiber; those two claims are now a direct finite-count contradiction. The latest Lean strengthening packages the same audit at the whole-output level. The predicate

$$\text{CountIndependentWithinToleranceOutput}$$

requires every output fiber of a deterministic variable $Y : \Omega \rightarrow \alpha$ to satisfy the same tolerance bound. For a nonconstant recoding $Y(\omega) = r(\mathbf{1}_{E(\omega)})$, Lean proves

`conditionedSourceFiberProduct_le_of_CountIndependentWithinToleranceOutput_boolToAnyRecoding_non`

and its lower-bound and contradiction forms. Thus a repair cannot evade the fiber theorem by claiming approximate independence only for the recoded output variable as a whole: the $r(1)$ -fiber already forces the same $|C \cap E| |C \cap \neg E| \leq \tau$ obligation. The newest finite-coin refinement covers the first randomized recoding subcase. For $Y(\omega, c) = r(\mathbf{1}_{E(\omega)}, c)$, if a single output value y uniformly selects the true source side for every coin,

$$\forall c, r(1, c) = y \quad \text{and} \quad \forall c, r(0, c) \neq y,$$

or uniformly selects the false side with the inequalities reversed, then

`conditionedSourceFiberProduct_le_of_CountIndependentWithinToleranceOutput_coinwise_trueFiber`

and

`conditionedSourceFiberProduct_le_of_CountIndependentWithinToleranceOutput_coinwise_falseFiber`

force the same product/tolerance obligation on the lifted source/coin product space. The lower-bound, strict-tolerance contradiction, and zero-tolerance forms are regression-pinned as well. The follow-up source-count form uses

`finiteEventCount_fst_lift`

to factor source-only events on $\Omega \times \text{Coin}$:

$$|(C \cap E) \times \text{Coin}| = |C \cap E| |\text{Coin}|.$$

Consequently the coinwise true/false-fiber certificate must satisfy

$$(|C \cap E| |\text{Coin}|)(|C \cap \neg E| |\text{Coin}|) \leq \tau,$$

with matching lower-bound and strict-tolerance contradiction forms `sourceLowerBounds_coinProduct_le_of_Count` and `not_countIndependentWithinToleranceOutput_of_coinwise*_sourceLowerBounds_mul_gt`.

The zero-tolerance boundary is now source-level as well. The helper `finiteEventCount_fst_lift_pos_of_pos` proves that any positive source-only event remains positive on $\Omega \times \text{Coin}$ when the finite coin space is nonempty. Hence

`not_countIndependentWithinToleranceOutput_zero_of_coinwise_trueFiber_sourceNonconstant,`
`not_countIndependentWithinToleranceOutput_zero_of_coinwise_falseFiber_sourceNonconstant`

block uniform true-side or false-side distinguishing fibers from satisfying zero-tolerance output independence under the source-side hypotheses $|C \cap E| > 0$, $|C \cap \neg E| > 0$, and $\text{Coin} \neq \emptyset$. This removes the need to state the nondegeneracy premise directly on the lifted product space. The arbitrary finite-coin zero-tolerance boundary is now exact under the same source-side nondegeneracy hypotheses. Lean proves

`countIndependentWithinToleranceOutput_zero_iff_finiteCoinRecoding_fiberBalanced_of_sourceNoncon`

which says that zero-tolerance output independence on $\Omega \times \text{Coin}$ holds if and only if every output fiber is balanced between the true and false source sides. The proof uses the one-sided zero blockers

`not_countIndependentWithinToleranceOutput_zero_of_finiteCoinRecoding_trueMinusFalse_sourceNonc`
`not_countIndependentWithinToleranceOutput_zero_of_finiteCoinRecoding_falseMinusTrue_sourceNonc`

to rule out any positive imbalance in either direction. The next Lean refinement removes the uniform-fiber assumption for arbitrary finite-coin recodings of the same source bit. For each output value y , let

$$T_y = \#\{c : r(1, c) = y\}, \quad F_y = \#\{c : r(0, c) = y\}.$$

Lean proves the exact mixed-coin identities

$$\Delta_y^+ = |C \cap E| |C \cap \neg E| |\text{Coin}| (T_y - F_y),$$

$$\Delta_y^- = |C \cap E| |C \cap \neg E| |\text{Coin}| (F_y - T_y),$$

as

`countIndependentWithinForwardGap_finiteCoinRecoding_fiber`

and

`countIndependentWithinBackwardGap_finiteCoinRecoding_fiber.`

Thus a finite-coin randomized recoding can repair the same-source fiber only by balancing the true-side and false-side coin counts for every output fiber, or by making the source-fiber product small enough for the chosen tolerance. The strict blocker theorems

`not_countIndependentWithinToleranceOutput_of_finiteCoinRecoding_trueMinusFalse_gt`

and

`not_countIndependentWithinToleranceOutput_of_finiteCoinRecoding_falseMinusTrue_gt`

turn any unbalanced fiber into a direct contradiction when the claimed tolerance is below the corresponding scaled imbalance. The source-lower-bound forms replace the exact source counts by $L_1 \leq |C \cap E|$ and $L_0 \leq |C \cap \neg E|$, yielding the manuscript-facing obligations

$$L_1 L_0 |\text{Coin}| (T_y - F_y) \leq \tau \quad \text{and} \quad L_1 L_0 |\text{Coin}| (F_y - T_y) \leq \tau,$$

with matching strict contradiction forms. The current Lean packet packages this as a threshold quantization theorem: below one lower-bounded source/coin imbalance block, approximate output independence is already equivalent to exact balanced fibers. The main form is

`countIndependentWithinToleranceOutput_sourceLowerBounds_lt_coinProduct_iff_finiteCoinRecoding_`

assuming $L_1 \leq |C \cap E|$, $L_0 \leq |C \cap \neg E|$, and $\tau < L_1 L_0 |\text{Coin}|$. The exact source-count specialization is

`countIndependentWithinToleranceOutput_lt_sourceCoinProduct_iff_finiteCoinRecoding_fiberBalanced`

Thus small additive slack cannot hide a single finite-coin fiber imbalance; it must either reach the first source/coin defect block or force exact balance. This does not close randomized residuals that use information beyond a finite-coin recoding of the conditioned source bit, but it precisely closes the finite-coin cancellation loophole for the same-source recoding surface. Thus richer deterministic

labels do not evade the quantitative audit: every informative deterministic output fiber must either pay this finite-count defect or prove that the relevant product is small after the manuscript's normalization. In the finite-coin case the analogous obligation is the scaled coin-fiber imbalance, so exact coin-count balance is now the formal escape condition rather than an informal repair slogan. The balanced case is now formalized directly. Lean defines

$$\text{FiniteCoinRecodingFiberBalanced } r \equiv \forall y, T_y = F_y,$$

and proves

`countIndependentWithinToleranceOutput_zero_of_finiteCoinRecoding_fiberBalanced.`

Thus a perfectly balanced finite-coin recoding really does decorrelate the marginal output from the source bit at zero tolerance. This is not a blocker; it is the exact cancellation case. The companion retained-coin theorem records the price of using that cancellation while still preserving source information through the random seed. If the certified output is $(r(E(\omega), c), c)$ and for some retained coin c_0 one has $r(1, c_0) \neq r(0, c_0)$, then Lean proves

`countIndependentWithinForwardGap_coinRetained_trueFiber`

with defect

$$|C \cap E| |C \cap \neg E| |\text{Coin}|.$$

The corresponding obligation and strict-blocker forms

`sourceCoinProduct_le_of_CountIndependentWithinToleranceOutput_coinRetained_trueFiber`

and

`not_countIndependentWithinToleranceOutput_of_coinRetained_trueFiber_sourceProduct_gt`

show that retaining the coin revives the finite-count obstruction. The newest Lean strengthening turns this into an exact below-threshold boundary:

`finiteCoinRecodingFiberBalanced_coinRetained_iff_pointwise_eq`

states that balanced fibers for the retained output $(r(b, c), c)$ are equivalent to pointwise source-blindness, $\forall c, r(1, c) = r(0, c)$. Hence below one lower-bounded source/coin block,

`countIndependentWithinToleranceOutput_coinRetained_sourceLowerBounds_lt_coinProduct_iff_pointw`

says approximate output independence of the retained-coin output is possible exactly when every retained coin already carries no source-dependent output. The retained-side counterpart now makes the same boundary fiberwise rather than coinwise. Lean proves

`finiteCoinRecodingFiberBalanced_sideRetained_iff_sideFiber_count_eq,`

which says that balanced fibers for $(r(b, c), s(c))$ are equivalent to equality of true and false counts in every retained output/side fiber:

$$\forall y, t, \quad \#\{c : r(1, c) = y \wedge s(c) = t\} = \#\{c : r(0, c) = y \wedge s(c) = t\}.$$

Consequently, below one lower-bounded source/coin block,

`countIndependentWithinToleranceOutput_sideRetained_sourceLowerBounds_lt_coinProduct_iff_sideFi`

turns approximate independence of the retained-side output into this exact side-fiber multiset-balance condition. A side channel can still aggregate coins, but it cannot hide an imbalanced true/false multiset inside any observed side/output fiber below the first source/coin threshold. Therefore the balanced-fiber escape has a sharp fork: the marginal output can be exactly decorrelated only by erasing same-source evidence from output fibers. This is not limited to literally retaining the whole coin. Lean now proves the two-sided side-channel generalization

`countIndependentWithinForwardGap_sideRetained_trueFiber` and `countIndependentWithinBackwardGap`

For any retained side channel $s : \text{Coin} \rightarrow \text{Side}$, if the side fiber $s(c) = s(c_0)$ contains no false-side output equal to $r(1, c_0)$, the output $(r(E, c), s(c))$ has forward defect

$$|C \cap E| |C \cap \neg E| |\text{Coin}| \#\{c : r(1, c) = r(1, c_0) \wedge s(c) = s(c_0)\}.$$

Symmetrically, if the same kind of side fiber contains no true-side output equal to $r(0, c_0)$, the selected false-side fiber has the same scaled formula as a backward defect, with $r(0, c_0)$ in place of $r(1, c_0)$. The obligation and strict-blocker forms

`sideChannelProduct_le_of_CountIndependentWithinToleranceOutput_sideRetained_trueFiber` and `side`

and

`not_countIndependentWithinToleranceOutput_of_sideRetained_trueFiber_product_gt` and `not_count`

show that any side information capable of isolating such a source-bearing fiber must pay the corresponding scaled product in the tolerance. Lean now removes the literal-pair assumption as well. For any postprocessing $p : \alpha \times \text{Side} \rightarrow \beta$, if the observed value $p(r(1, c_0), s(c_0))$ is distinct from every false-side observed value, then

`countIndependentWithinForwardGap_postprocessedSide_trueFiber`

gives the same forward-defect formula with the selected observed-fiber count

$$\#\{c : p(r(1, c), s(c)) = p(r(1, c_0), s(c_0))\}.$$

The theorem

`countIndependentWithinBackwardGap_postprocessedSide_falseFiber`

is the symmetric false-side version. Thus hashing, compressing, or otherwise observing the retained side channel does not evade the audit whenever some observed fiber still separates the source side; the corresponding strict blockers

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_trueFiber_product_gt` and `not_c`

force the same scaled product into the tolerance. Lean now also records the exact below-threshold boundary for this hashed or otherwise postprocessed side-channel case. It proves

`finiteCoinRecodingFiberBalanced_postprocessedSide_iff_observedFiber_count_eq,`

which says that balanced fibers for $p(r(b, c), s(c))$ are exactly equality of true-side and false-side counts for every observed postprocessed value:

$$\forall z, \quad \#\{c : p(r(1, c), s(c)) = z\} = \#\{c : p(r(0, c), s(c)) = z\}.$$

Consequently,

`countIndependentWithinToleranceOutput_postprocessedSide_sourceLowerBounds_lt_coinProduct_iff_of`

says that below one lower-bounded source/coin block, approximate independence of the postprocessed retained-side output is possible exactly under this observed-fiber balance condition. Hashing can merge side/output fibers, but it must merge them into perfectly balanced observed fibers; otherwise the finite-count obstruction reappears. Lean also records the quantitative form behind this exact boundary:

`sourceLowerBounds_observedFiberImbalanceProducts_le_of_CountIndependentWithinToleranceOutput_p`

Writing

$$T_z = \#\{c : p(r(1, c), s(c)) = z\}, \quad F_z = \#\{c : p(r(0, c), s(c)) = z\},$$

the theorem says that any approximate-independence certificate with source lower bounds ℓ_1, ℓ_0 must satisfy, for every observed value z ,

$$\ell_1 \ell_0 |\text{Coin}|(T_z - F_z) \leq \tau \quad \text{and} \quad \ell_1 \ell_0 |\text{Coin}|(F_z - T_z) \leq \tau.$$

The strict blockers

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_observedFiber_trueMinusFalse_so`

and

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_observedFiber_falseMinusTrue_so`

therefore block any hashed retained-side repair whose merged observed fibers are merely less isolated but still imbalanced. Postprocessing has to buy actual cancellation, not just coarser observation. Lean now also exposes the predicate-level downstream consequence:

`postprocessedSide_outputPredicate_count_eq_of_countIndependentWithinToleranceOutput_sourceLower`

For every decidable property P of the postprocessed retained-side observation, below the same source/coin block,

$$\#\{c : P(p(r(1, c), s(c)))\} = \#\{c : P(p(r(0, c), s(c)))\}.$$

The strict blockers

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_outputPredicate_trueCount_gt_fa`

and

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_outputPredicate_falseCount_gt_fa`

rule out either direction of downstream predicate advantage. The downstream-decoder consequence is now explicit as well:

`postprocessedSide_outputDecoder_correctCount_eq_card_of_countIndependentWithinToleranceOutput_s`

For every Boolean decoder d of the postprocessed retained-side observation, below the same source/coin block Lean proves

$$\#\{c : d(p(r(1, c), s(c))) = 1\} + \#\{c : d(p(r(0, c), s(c))) = 0\} = |\text{Coin}|.$$

Thus any decoder that does better than half on the two coin fibers refutes the approximate-independence certificate. The strict blocker is

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_outputDecoder_correctCount_gt_c`

Lean now records the uniform-recovery special cases directly. A downstream predicate that contains every true-side observed value and no false-side observed value is blocked by

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_outputPredicate_separates_source`

A Boolean decoder that uniformly recovers the source bit after postprocessing is blocked by

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_outputDecoder_recovers_sourceLow`

Finally, if the postprocessed true-side and false-side observed supports are disjoint, then membership in the true-side support gives such a decoder, and Lean proves the corresponding support-separation blocker

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_sourceRangesSeparated_sourceLow`

The same result is also recorded in positive form as a required observed collision:

`exists_postprocessedSide_true_false_collision_of_countIndependentWithinToleranceOutput_sourceLow`

Thus a deterministic postprocessing repair that still claims below-threshold approximate independence must identify at least one true-side coin and one false-side coin whose observed postprocessed values coincide. It is not enough to hide the retained pair syntactically; the repair must create actual true/false support overlap in the finite coin observation. Lean then strengthens this from a single collision to pointwise support coverage:

`postprocessedSide_pointwise_true_false_collisions_of_countIndependentWithinToleranceOutput_sou`

Equivalently, every true-side coin has a false-side witness with the same postprocessed observation, and every false-side coin has a true-side witness. The two one-sided witness forms are

`exists_false_postprocessedSide_collision_of_countIndependentWithinToleranceOutput_sourceLowerBo`

`exists_true_postprocessedSide_collision_of_countIndependentWithinToleranceOutput_sourceLowerBo`

Thus a repair cannot satisfy the below-threshold certificate by arranging only one accidental overlap while leaving most finite-coin observations separated. Lean now also records why even this pointwise collision target is not enough:

`postprocessedSide_pointwise_collisions_do_not_imply_coinEquiv_preserving.`

The witness is a three-coin, two-observation example with true-side multiplicities 2, 1 and false-side multiplicities 1, 2. Every observed value appears on both source sides, so the pointwise collision obligations are satisfied in both directions, but no bijective coin matching can preserve the

postprocessed observation. This blocks the parallel repair route that accepts support overlap as a replacement for the exact finite multiplicity matching. Lean further isolates the remaining “choose a witness” repair. A one-sided true-to-false coin map that preserves the postprocessed observation is enough only if it is injective:

`exists_postprocessedSide_coinEquiv_preserving_of_injective_coinMap_preserving.`

Equivalently, when no preserving coin equivalence exists, every preserving one-sided witness map must be non-injective:

`not_injective_postprocessedSide_coinMap_preserving_of_no_coinEquiv.`

The same three-coin skew example realizes this obstruction in the strongest finite form:

`postprocessedSide_oneSided_coinMap_preserving_does_not_imply_coinEquiv_preserving.`

It has preserving one-sided maps, but every such map collapses two true-side coins onto one false-side witness. Thus witness choice alone cannot replace the multiplicity-preserving bijection demanded by the postprocessed-side route. The finite example is not the main theorem here. Lean now records the abstract repair equivalence:

`exists_injective_postprocessedSide_coinMap_preserving_iff_exists_coinEquiv_preserving.`

For any finite coin type, an injective one-sided witness map is exactly the original preserving coin-equivalence obligation. With decidable observed outputs this becomes the observed-fiber count condition:

`exists_injective_postprocessedSide_coinMap_preserving_iff_observedFiber_count_eq.`

Consequently, any observed-fiber count mismatch forces every preserving one-sided witness map to be non-injective:

`not_injective_postprocessedSide_coinMap_preserving_of_observedFiber_count_ne.`

This generic pigeonhole theorem is the robust blocker for the injective-witness repair route; the three-coin skew instance merely exhibits one small mismatch. The next Lean strengthening records the multiplicity-level obligation behind that support statement. Exact observed-fiber equality gives a fiberwise equivalence, and Lean assembles these fiberwise equivalences through `Equiv.sigmaFiberEquiv` into a global matching of coin choices:

`exists_postprocessedSide_coinEquiv_preserving_of_countIndependentWithinToleranceOutput_sourceL`

The matched equivalence $e : \text{Coin} \simeq \text{Coin}$ satisfies

$$\forall c, \quad \text{post}(r(\text{true}, c), \text{side}(c)) = \text{post}(r(\text{false}, e(c)), \text{side}(e(c))).$$

The direct contrapositive blocker is

`not_countIndependentWithinToleranceOutput_of_no_postprocessedSide_coinEquiv_preserving_sourceL`

Lean also proves the local cardinality obstruction to this matching repair. Any proposed preserving coin equivalence forces equality of the true-side and false-side finite counts in every observed postprocessed fiber:

`postprocessedSide_observedFiber_count_eq_of_coinEquiv_preserving.`

Consequently, a single observed value z with unequal true/false coin-fiber multiplicities rules out every preserving coin equivalence,

`not_exists_postprocessedSide_coinEquiv_preserving_of_observedFiber_count_ne,`

and below the source/coin threshold it directly refutes approximate output independence:

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_observedFiber_count_ne_sourceLower`

Thus the repair is not allowed to merely produce the same set of observed values on both source sides: every observed value must occur with the same finite multiplicity, enough to support a true/false coin bijection preserving the postprocessed observation. The same matching obstruction is now exposed at the downstream-predicate interface. Any preserving coin equivalence forces equality of the true-side and false-side counts for every decidable predicate of the postprocessed observation,

`postprocessedSide_outputPredicate_count_eq_of_coinEquiv_preserving.`

Hence a single predicate-count mismatch rules out every preserving coin equivalence,

`not_exists_postprocessedSide_coinEquiv_preserving_of_outputPredicate_count_ne,`

and below the source/coin threshold blocks approximate output independence:

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_outputPredicate_count_ne_sourceLower`

For decoder-only repair attempts, Lean records the corresponding half-accuracy coin-matching obligation:

`postprocessedSide_outputDecoder_correctCount_eq_card_of_coinEquiv_preserving,`

with contradiction forms

`not_exists_postprocessedSide_coinEquiv_preserving_of_outputDecoder_correctCount_ne_card`

and

`not_countIndependentWithinToleranceOutput_of_postprocessedSide_outputDecoder_correctCount_ne_card`

Thus a repair cannot hide behind a downstream test or decoder instead of exhibiting the full observed-fiber matching. The predicate-only route is now fully collapsed, not just blocked in one direction. Lean proves that downstream predicate neutrality is equivalent to observed-fiber balance,

`postprocessedSide_outputPredicate_count_eq_iff_observedFiber_count_eq,`

and therefore equivalent to the existence of the preserving coin equivalence,

`exists_postprocessedSide_coinEquiv_preserving_iff_outputPredicate_count_eq.`

Combining this with the source/coin threshold gives the exact predicate-only boundary

`countIndependentWithinToleranceOutput_postprocessedSide_sourceLowerBounds_lt_coinProduct_iff_observedFiber_count_eq`

Thus a proposed certificate that all downstream tests are source-neutral is not a weaker substitute for coin matching; it is precisely the same finite multiplicity condition in predicate form. The same equivalence has now been recorded directly at the independence layer:

`countIndependentWithinToleranceOutput_postprocessedSide_sourceLowerBounds_lt_coinProduct_iff_c`

Below the source/coin threshold, a postprocessed-side approximate independence certificate exists if and only if there is a true/false coin equivalence preserving every observed postprocessed value. This closes another natural repair attempt: hashing or postprocessing the retained side cannot leave a recoverable Boolean trace, or even disjoint observed supports, for the source bit unless the independence budget has already crossed the source/coin threshold. The finite-output postprocessing foundation also separates lossless recoding from genuine coarsening. It now proves

`finiteCoinRecodingFiberBalanced_postcompose_iff_of_injective`

and the corresponding predicate-erasure equivalence

`finiteCoinOutputPredicateErasing_postcompose_iff_of_injective.`

Thus an injective deterministic postprocessor cannot create the balanced finite-coin erasure needed by the repair; if postprocessing repairs the obstruction, the map must be many-to-one. The stronger observed-support version is

`finiteCoinRecodingFiberBalanced_postcompose_iff_of_observedLeftInverse,`

with the corresponding predicate-erasure form

`finiteCoinOutputPredicateErasing_postcompose_iff_of_observedLeftInverse.`

Even a many-to-one map cannot create erasure if the original output can be recovered from the postprocessed value on every observed true/false coin sample. Therefore a coarsening repair must actually lose original-output information on the finite support used by the argument. Lean now sharpens this to the collision level:

`finiteCoinRecodingFiberBalanced_postcompose_iff_of_observedInjective`

and

`exists_observedOutput_collision_of_postcompose_fiberBalanced_of_not_fiberBalanced.`

If the original finite-coin recoding is not balanced but the postprocessed recoding is balanced, then two distinct observed original outputs must have the same postprocessed value. Thus the many-to-one repair owes an actual observed collision, not merely a vague lossy postprocessing step. Lean now sharpens the same point quantitatively:

`countIndependentWithinToleranceOutput_finiteCoinOutput_postcompose_iff_of_observedInjective`

and

`countIndependentWithinToleranceOutput_finiteCoinOutput_postcompose_iff_of_observedLeftInverse.`

These prove that support-injective postprocessing, or more generally any postprocessing with a left inverse on the actually observed outputs, preserves the entire finite-coin output tolerance certificate exactly. The defect-level forms

`countIndependentWithinOutputDefect_finiteCoinOutput_postcompose_eq_of_observedInjective`

and

`countIndependentWithinOutputDefect_finiteCoinOutput_postcompose_eq_of_observedLeftInverse`

show that the maximum output defect itself is unchanged. Therefore a deterministic postprocessing repair cannot lower the approximate decorrelation budget unless it genuinely loses original-output information on the observed support. Lossless relabeling, or any supportwise recoverable coarsening, is quantitatively useless as a repair. The new follow-up file `FiniteCoinPostprocessingImprovementObstruction.lean` now records the strict contrapositive:

`exists_observedOutput_collision_of_postcompose_tolerance_of_not_tolerance`

and

`exists_observedOutput_collision_of_postcompose_outputDefect_lt`.

If postprocessing makes a fixed tolerance- τ certificate newly true, or strictly lowers the finite-coin output defect, then some two distinct observed original outputs must map to the same postprocessed value. The paired cardinality forms

`exists_postprocessedFiber_card_gt_one_of_postcompose_tolerance_of_not_tolerance`

and

`exists_postprocessedFiber_card_gt_one_of_postcompose_outputDefect_lt`

state the same repair obligation as a genuinely nontrivial merged fiber. Thus even approximate same-source improvement still owes a concrete observed many-to-one step; it cannot come from a supportwise lossless re-encoding. The next Lean strengthening is source-bias sensitive:

`exists_oppositeBias_observedOutput_collision_of_postcompose_fiberBalanced_of_not_fiberBalanced`.

It proves that a balancing postprocessor must merge, inside one postprocessed fiber, an original output with strictly more true-side coin mass and an original output with strictly more false-side coin mass. Same-bias collisions therefore cannot be used as the repair; the coarsening must perform a real opposite-bias cancellation on the observed finite support. The conservation form of the same obstruction is now explicit:

`finiteCoinSignedFiberBias_postcompose_eq_sum_preimage`

and

`finiteCoinRecodingFiberBalanced_postcompose_iff_forall_sum_signedFiberBias_preimage_eq_zero`.

Here `finiteCoinSignedFiberBias` is the integer true-minus-false fiber count. A successful deterministic coarsening is therefore equivalent to zero total signed original bias in every postprocessed

fiber. This blocks a parallel repair route in which the author supplies collisions but not exact cancellation certificates: the Lean statement asks for the full per-fiber integer identity. The degenerate endpoint is explicit as

`finiteCoinRecodingFiberBalanced_constant,`

which says a constant output is always balanced. That is source erasure, not evidence that the original finite-coin signal survived postprocessing. The new file `FiniteCoinPostprocessingSameSignObstruction.lean` now pushes the approximate story to the same cancellation boundary. It defines `finiteOutputFiberSignedBiasOneSided` and proves

`countIndependentWithinOutputDefect_finiteCoinOutput_le_of_signedBiasOneSided.`

If every postprocessed fiber merges only nonnegative signed biases, or only nonpositive signed biases, then deterministic postprocessing cannot strictly lower the finite-coin output defect at all. The contrapositive forms

`exists_oppositeSign_observedOutput_collision_of_postcompose_outputDefect_lt`

and

`exists_oppositeSign_observedOutput_collision_of_postcompose_tolerance_of_not_tolerance`

therefore show that strict approximate improvement requires a merged fiber that contains both a positive-bias original output and a negative-bias original output. Mere same-sign coarsening is not enough: the repair must perform genuine opposite-sign cancellation on the finite observed support. The new file `FiniteCoinSignedBiasQuantizerObstruction.lean` turns this into the manuscript-facing finite-quantization statement. It defines `finiteCoinSignedBiasQuantizerOneSided` for quantizers $q : \mathbb{Z} \rightarrow \beta$ of the signed true-minus-false boundary bias, and the induced recoding `finiteCoinSignedBiasQuantizedRecoding`. The core theorem

`countIndependentWithinOutputDefect_finiteCoinOutput_le_of_finiteCoinSignedBiasQuantizerOneSided`

says that if every quantizer cell lies entirely in the nonnegative biases or entirely in the nonpositive biases, then quantizing the boundary bias cannot strictly lower the finite-coin output defect. The contrapositive theorems

`not_finiteCoinSignedBiasQuantizerOneSided_of_quantizedOutput_outputDefect_lt`

and

`not_finiteCoinSignedBiasQuantizerOneSided_of_quantizedOutput_tolerance_of_not_tolerance`

therefore say that any successful finite quantization repair must use some bucket that crosses zero. The explicit collision forms

`exists_oppositeSign_quantizerCollision_of_quantizedOutput_outputDefect_lt`

and

`exists_oppositeSign_quantizerCollision_of_quantizedOutput_tolerance_of_not_tolerance`

state that one quantizer value must identify a positive-bias and negative-bias original output. So a side-preserving bucketization of the boundary field cannot be the repair; any helpful finite quantizer must perform genuine cross-zero cancellation. The new file `FiniteCoinSignedBiasDitheredQuantizerObstruction` lifts the same obstruction to the manuscript’s explicit finite-public-dither variant. Lean models the dithered quantizer by adjoining the seed to the observed output,

`finiteCoinOutputWithDither,`

and then applying a deterministic map of the pair (signed bias, seed). The theorem

`countIndependentWithinOutputDefect_finiteCoinOutputWithDither_le_of_finiteCoinSignedBiasDither`

says that if every dithered quantizer bucket contains only nonnegative signed biases or only nonpositive signed biases, uniformly across all seeds, then the dithered quantization still cannot strictly lower the finite-coin output defect. The tolerance form

`not_countIndependentWithinToleranceOutput_finiteCoinSignedBiasDitheredQuantizedRecoding_of_not`

shows that such a one-sided dithered quantizer also cannot make a failing tolerance certificate newly true. Consequently, any successful finite dithered quantizer must have a bucket that merges a positive-bias output/seed pair with a negative-bias output/seed pair, as recorded in

`exists_oppositeSign_ditheredQuantizerCollision_of_quantizedOutput_outputDefect_lt`

and

`exists_oppositeSign_ditheredQuantizerCollision_of_quantizedOutput_tolerance_of_not_tolerance`.

So “independent dither” does not create a new escape hatch here: it is still just deterministic post-processing on a larger finite observed support, and any helpful bucket must perform genuine cross-zero cancellation. The follow-up file `FiniteCoinSignedBiasDitheredQuantizerExactCancellation.lean` pushes the same dither lane to the exact zero-tolerance boundary. The theorem

`finiteCoinSignedFiberBias_finiteCoinSignedBiasDitheredQuantizedRecoding_eq_sum_preimage`

states that the signed bias of one dithered quantizer output is exactly the sum of the original signed biases over the corresponding (output, seed) preimage fiber. Therefore

`countIndependentWithinToleranceOutput_zero_finiteCoinSignedBiasDitheredQuantizedRecoding_iff_f`

shows that exact output balance after dithered quantization is equivalent to a literal integer cancellation law in every dithered bucket. The contrapositive

`not_countIndependentWithinToleranceOutput_zero_finiteCoinSignedBiasDitheredQuantizedRecoding_o`

blocks exact repair as soon as one dithered quantizer bucket has nonzero total signed bias on the enlarged support. So public dither does not merely require an opposite-sign collision somewhere; every surviving bucket must satisfy exact integer cancellation. Lean now also identifies this law with the older retained-side postprocessing boundary rather than treating it as a separate escape route: the theorem

`finiteCoinSignedBiasDitheredQuantizedRecoding_exactCancellation_iff_observedBucketCount_eq`

shows that exact signed-bias cancellation is exactly the statement that every observed dithered quantizer bucket contains equally many true-side and false-side witnesses, and the route-facing theorem

`countIndependentWithinToleranceOutput_zero_finiteCoinSignedBiasDitheredQuantizedRecoding_iff_of`

packages zero-tolerance success in precisely that bucketwise count-equality form. Lean now also proves this erasure claim first in predicate form,

`not_finiteCoinRecodingFiberBalanced_of_outputPredicate_separates,`

and then in decoder form:

`not_finiteCoinRecodingFiberBalanced_of_outputDecoder_recovers.`

For any nonempty coin space, no output-only property can hold on every true-side marginal output and fail on every false-side marginal output while the recoding remains fiber-balanced; the Boolean decoder result is just the special case where the property is “the decoder returns 1.” If a repair instead weakens this to a partial output-predicate advantage, the aggregate count theorem

`finiteCoinRecodingFiberBalanced_outputPredicate_count_eq`

still blocks it: for every decidable output predicate P ,

$$\#\{c : P(r(1, c))\} = \#\{c : P(r(0, c))\}.$$

The strict forms

`not_finiteCoinRecodingFiberBalanced_of_outputPredicate_trueCount_gt_falseCount`

and

`not_finiteCoinRecodingFiberBalanced_of_outputPredicate_falseCount_gt_trueCount`

turn either directional predicate-count advantage into a contradiction to fiber balance. Specializing to a Boolean decoder gives the sharper half-accuracy statement

`finiteCoinRecodingFiberBalanced_outputDecoder_correctCount_eq_card.`

If $D : \alpha \rightarrow \{0, 1\}$, then balanced fibers force

$$\#\{c : D(r(1, c)) = 1\} + \#\{c : D(r(0, c)) = 0\} = |\text{Coin}|.$$

Thus a marginal output decoder is correct on exactly half of the two source-side coin trials; Lean’s strict form

`not_finiteCoinRecodingFiberBalanced_of_outputDecoder_correctCount_gt_card`

blocks any better-than-half aggregate decoder. If a repair instead preserves the source by carrying the coin as side information, the source-product obstruction returns, scaled by $|\text{Coin}|$. This still does not refute every possible approximate decorrelation strategy; it does prove that the manuscript cannot simply weaken exact conditional independence to approximate conditional independence on a nondegenerate same-source surface, including these whole-output deterministic and arbitrary finite-coin randomized recoding subcases, without supplying a real quantitative small-product, coin-fiber balance, or normalization estimate.

Current blocker status. This is now a high-confidence local blocker for the deterministic Boolean conditioning repair pattern and its immediate arbitrary-codomain deterministic extension: copied bits, complemented bits, all nonconstant deterministic Boolean recodings, and every deterministic recoding of the same bit into a decidable codomain have exactly the same empty/full degeneracy boundary under the finite-count fiber audit. It is not yet a global $> 90\%$ blocker for the whole PNP route. A future repair could still avoid the theorem by proving that the post-conditioned variable uses residual information not determined by the conditioned source bit, by working with a randomized residual object, or by supplying a quantitatively justified approximate theorem whose allowed error actually dominates the finite-count product defect on the relevant conditioned surface. To upgrade this from a local crux to a route-killing crux, the remaining task is to connect the manuscript's uniqueness-conditioned variables to this exact deterministic same-source recoding surface, or to extend the theorem to the richer post-conditioned object actually used by the proof attempt.

It should therefore not be presented as a reason for the author to abandon the PNP project. Its proper role is diagnostic: it says that a proof continuing along this nearby line must supply a real post-conditioning decorrelation bridge, not merely a different deterministic Boolean recoding of the same conditioned bit. A stop-grade crux would need a broader route-class theorem showing that the manuscript's natural repairs still fall inside a blocked formal family, or an equally broad counterexample to the required conditioning bridge itself.

9 Obstruction II: The Full Local Rule Class Is Enormous

Let n be the number of visible Boolean features after switching. Then the unconstrained class of local predictors is simply

$$\{0, 1\}^{\{0,1\}^n},$$

the set of all Boolean functions on n bits. Its size is

$$2^{2^n}.$$

Therefore any *uniform* code space that represents all such rules injectively must have at least 2^n bits.

This is formalized in `CompressionObstruction.lean`.

Theorem 9.1 (Counting bound for local rules). *Let $LocalRule\ n$ denote the type of Boolean functions on n visible bits, and $BitCode\ s$ the type of s -bit codewords. If*

$$s < 2^n,$$

then there is no injective map

$$LocalRule\ n \rightarrow BitCode\ s.$$

This is the theorem `no_injective_bitCode_of_lt`.

Theorem 9.2 (Binary-log neighborhood corollary). *If $d \geq 2$ and*

$$s < 2^m,$$

then there is no injective map

$$LocalRule(d^{\log_2 m + 1}) \rightarrow BitCode\ s.$$

This is the theorem `no_uniform_code_below_expInput_for_binaryLogNeighborhood`.

Theorem 9.3 (Size lower bound on the full class). *For $d \geq 2$,*

$$2^{2^m} \leq \left| \text{LocalRule}\left(d^{\lceil \log_2 m \rceil + 1}\right) \right|.$$

This is the theorem `expExpInput_le_card_localRule_binaryLogNeighborhood`.

9.1 Interpretation

Again, this does not say that the switched rule class *is* the full local rule class. It says that without a theorem proving otherwise, one cannot pass from “local on n visible bits” to “describable by only $\text{polylog}(m)$ bits.”

That gap is exactly where the current program remains vulnerable.

10 What These Results Break

The present Lean work targets the following inference schema:

$$\text{short decoder} \implies \text{switched local predictor} \implies \text{tiny ACC}^0/\text{ERM hypothesis}.$$

The first arrow is the paper’s switching claim. The second arrow is where the present obstruction lands.

Remark 10.1. If the switched predictor is only known to be some arbitrary function of a log-radius neighborhood, then:

1. the neighborhood already has polynomial size in the ambient parameter, and
2. the class of all such local rules is far too large to admit a polylogarithmic uniform code.

Therefore a successful proof attempt must supply one of the following:

1. a theorem that the switched predictors belong to a very special subclass (for example, a family with explicit circuit/description bound), or
2. a direct compression algorithm showing that the relevant local rules can be encoded in $\text{polylog}(m)$ bits for reasons stronger than locality.

Without such a theorem, the proof’s route from locality to tiny hypothesis class is unsupported.

11 Obstruction III: Zero-Margin Symmetrization Cannot Concentrate

The symmetrization route in the paper makes a stronger claim than the two counting obstructions above. It says, roughly, that after taking a polylogarithmic number of promise-preserving sign flips and then taking a majority vote, the resulting surrogate label agrees with the Bayes rule on the local σ -field for all but a negligible fraction of blocks.

There is a direct obstruction here when the conditional surrogate mean lands at exactly $\frac{1}{2}$.

11.1 Lean counting lemma

The Lean module `SymmetrizationObstruction.lean` formalizes the following fact.

Theorem 11.1 (Odd-majority is exactly balanced on unbiased bits). *Let s be odd. Among the 2^s bit-vectors of length s , exactly half have strict majority **true** and half have strict majority **false**.*

The key formal statements are:

- `majority_compl`: complementing all bits flips the majority bit when s is odd;
- `card_majority_true_eq_card_majority_false`: the `majority-true` and `majority-false` classes have equal cardinality;
- `majority_tie_break_not_high_probability`: if a Bayes rule breaks a $p = \frac{1}{2}$ tie toward **true**, then majority on s unbiased samples matches that rule on only half the sample space.

11.2 Application to the Current Symmetrization Wrapper

Now consider the paper’s symmetrized surrogate label for a fixed block j and bit i . Take the simplest possible decoder P : the decoder that always outputs 0.

Under a sign flip σ , the paper’s own back-mapping rule gives

$$Y_{j,i}^{(r)} = \langle a_{j,i}, \sigma^{(r)} \rangle$$

over $\mathbb{F}_2$, because the decoder output is 0 and the back-map xors in the VV shift.

If $a_{j,i} \neq 0$, then $\langle a_{j,i}, \sigma \rangle$ is an unbiased bit under uniform random σ . Hence the sequence

$$Y_{j,i}^{(1)}, \dots, Y_{j,i}^{(s)}$$

is a fair sample of surrogate bits, and for odd s the majority vote is exactly balanced.

But the paper’s Bayes rule is written with threshold $1[p \geq \frac{1}{2}]$, which at $p = \frac{1}{2}$ chooses the value 1. Therefore, in this zero-margin case, the majority vote matches the stated Bayes rule on only half the seed choices, not with probability $1 - m^{-\Omega(1)}$.

11.3 Consequence

This is materially stronger than the earlier “local does not imply simple” objection. It says:

The concentration-to-Bayes step in the symmetrization argument is false unless an additional margin theorem is proved.

In other words, one must show that for the relevant blocks and bits,

$$\left| p_{j,i} - \frac{1}{2} \right|$$

is bounded below by more than the sampling noise scale. Without such a margin, majority voting cannot recover the Bayes rule uniformly, even for the constant-zero decoder.

12 Obstruction IIIb: The Exact Post-Switch Input Is Not T_i -Invariant

Before moving to a charitable repair, the manuscript's own exact definitions already create a direct problem.

Section 3.2 defines the promise-preserving involution

$$T_i(F^h, A, b) = (F^{\tau_i h}, A, b \oplus a_i),$$

while Section 3.3 defines the post-switch local input

$$u_i(\Phi) = (z(F^h), a_i, b).$$

Appendix A.5 then says this same involution preserves $u = (z, a_i, b)$. The new Lean module `PostSwitchInputObstruction.lean` formalizes the exact contradiction point.

12.1 Lean theorem

Theorem 12.1 (Full-input invariance holds only on zero VV columns). *Let $u = (z, a, b)$ be the exact post-switch local input, and let T act on it by*

$$T(z, a, b) = (z, a, b \oplus a).$$

Then

$$T(u) = u \quad \Longleftrightarrow \quad a = 0.$$

Equivalently, if the VV column a is nonzero, then the involution necessarily changes the right-hand side coordinate b .

The key formal statements are:

- `vvToggle_eq_self_iff_zero`: $b \oplus a = b$ iff $a = 0$;
- `tiInputMap_eq_self_iff_zeroColumn`: the exact local input is T_i -invariant iff the VV column is zero;
- `invariantProjection_tiInputMap`: the reduced projection (z, a) is always preserved;
- `tiInputMap_changes_b_of_nonzeroColumn`: if $a \neq 0$, then b really changes.

12.2 Why this matters

This is not a soft complaint about omitted detail. It is a direct incompatibility between the paper's own formulas:

1. the exact local input keeps b ,
2. the involution toggles b by a_i ,
3. yet the calibration appendix says the involution preserves $u = (z, a_i, b)$.

That can only be true on the special case $a_i = 0$. So the calibration argument cannot be right as written on the full local input.

12.3 Consequence

This forces the proof into a fork:

1. either keep the full input (z, a_i, b) , in which case the involution is not feature preserving and a new calibration theorem is needed;
2. or drop the changing coordinate b , in which case one lands in the invariant-feature regime attacked next.

13 Obstruction IIIc: The Exact Manuscript Fork Already Lands in the Symmetry Budget

The previous section exposed the formula-level inconsistency in the manuscript as written. The new Lean module `PostSwitchForkObstruction.lean` shows that the most charitable exact repair is already constrained by the later symmetry-budget files.

13.1 Lean theorems

Theorem 13.1 (The exact T_i -action is an involution and b resolves exactly the nonzero columns). *On the exact local input $u = (z, a, b)$ with*

$$T(z, a, b) = (z, a, b \oplus a),$$

the map T is an involution. Moreover,

$$b(Tu) \neq b(u) \iff a \neq 0.$$

Equivalently, the b -coordinate distinguishes an involution pair exactly on the nonzero-column slice.

Theorem 13.2 (The exact fork). *Let the target bit flip under the exact involution T , and let the weighting be T -invariant. Then:*

1. *every soft score depending only on the reduced invariant projection (z, a) has zero weighted signed correlation with the target;*
2. *every classifier using $((z, a), b)$ has success advantage over chance bounded by the total weight of the nonzero-column slice.*

The key formal statements are:

- `tiInputMap_involutive`: the manuscript's exact local update really is an involution;
- `resolvedBySideChannel_tiInputMap_b_iff_nonzeroColumn`: the b -side channel resolves an orbit pair exactly when $a \neq 0$;
- `weighted_signedScore_sum_eq_zero_on_invariantProjection`: the reduced exact input (z, a) has zero signed signal under symmetry-respecting weights;
- `doubledAdvantage_invariantProjection_with_b_le_nonzeroColumnMass`: any classifier on $((z, a), b)$ pays for its domination gap with nonzero-column mass;
- `pos_nonzeroColumnMass_of_pos_doubledAdvantage_invariantProjection_with_b`: any positive b -side-channel advantage forces positive nonzero-column mass;

- `exists_b_resolving_input_of_pos_doubledAdvantage_invariantProjection_with_b`: any positive b -side-channel advantage exposes a positive-weight input where $b(Tu) \neq b(u)$;
- `exists_nonzeroColumn_of_pos_doubledAdvantage_invariantProjection_with_b`: the same advantage exposes a positive-weight input with $a \neq 0$;
- `not_exactInputInvariantWeightedSupport_of_pos_doubledAdvantage_invariantProjection_with_b`: such positive advantage refutes the manuscript-style exact-input-invariant support premise;
- `pos_nonzeroColumnMass_of_total_lt_two_mul_weightedCorrectMass_invariantProjection_with_b`: the same nonzero-column obligation now also holds on the strict half-accuracy surface $\text{total} < 2 \cdot \text{correct}$;
- `not_total_lt_two_mul_weightedCorrectMass_invariantProjection_with_b_of_exactInputInvariantWeightedSupport_of_total_lt_two_mul_weightedCorrectMass_invariantProjection_with_b`: exact-input-invariant support blocks the strict half-accuracy route just as decisively as it blocks positive doubled advantage.

13.2 Why this matters

This turns the manuscript fork into a direct quantitative obligation.

If one drops b , then the surviving exact input is already inside the invariant-feature regime: no symmetry-respecting signed signal remains. If one keeps b , then the proof is no longer allowed to say merely that “a small side channel helps.” It must prove that the resulting domination gap is supported by a sufficiently large nonzero-column slice in the actual weighted domain. More sharply, the strengthened Lean API now extracts an actual positive-weight witness on that slice, so an aggregate domination claim must identify where the local input really changes under $b \mapsto b \oplus a_i$. The same API closes the immediate manuscript escape: if the same weighted support is still required to be exact-input invariant, then it has zero nonzero-column mass, so any positive b -side-channel advantage contradicts that invariance premise outright.

So this exact bridge does not merely identify a local inconsistency. It shows that the natural repair lands immediately in the later neutrality and resolution-budget obstructions.

14 Obstruction IV: Involution-Invariant Feature Collapse Forces Exact Neutrality

Once the exact full-input inconsistency is noticed, the most natural charitable repair is to drop the involution-sensitive coordinate b and keep only the involution-fixed part of the local view, say a feature map of the form

$$u' = (z, a_i).$$

This repair looks attractive because it restores feature invariance. The new Lean module `OrbitNeutralityObstruction.lean` shows why this move cannot, by itself, support a domination theorem.

14.1 Lean theorem

Theorem 14.1 (Feature-preserving involutions force exact half accuracy). *Let α be a finite sample space, let $\tau : \alpha \rightarrow \alpha$ be an involution, let $u : \alpha \rightarrow U$ be a feature map, and let $y : \alpha \rightarrow \{0, 1\}$ be the*

target bit. Assume

$$u(\tau x) = u(x) \quad \text{and} \quad y(\tau x) = 1 - y(x)$$

for every $x \in \alpha$. Then for every classifier $h : U \rightarrow \{0, 1\}$,

$$2 \cdot \#\{x : h(u(x)) = y(x)\} = |\alpha|.$$

Equivalently, the average accuracy of $h \circ u$ is exactly $1/2$.

The key formal statements are:

- `correct_under_pair_iff_incorrect`: correctness at x is equivalent to incorrectness at τx ;
- `card_correct_eq_card_incorrect`: the correct and incorrect classes have equal cardinality;
- `two_mul_card_correct_eq_card`: exactly half the finite sample space is classified correctly.

14.2 Why this hits the charitable calibration patch

Suppose the repair to the calibration bug is:

1. remove the involution-sensitive coordinate b from the local input;
2. prove neutrality by pairing each instance with its T_i -flipped partner;
3. then learn or evaluate a classifier using only the reduced feature map u' .

If the paired instances really preserve u' and flip the target bit, then the theorem above applies immediately. Every feature-only classifier on u' has exact $1/2$ average accuracy. So this route cannot yield a strict domination statement, a high-probability Bayes recovery statement, or any per-block success advantage above chance.

14.3 Consequence

This is a stronger objection than “the paper needs more detail.” It says:

If the calibration fix works by dropping the coordinate changed by the involution and then classifying from the invariant remainder alone, that fix destroys any chance of beating $1/2$ accuracy on the paired finite domain.

So a viable rescue must do something more subtle than mere feature collapse. For example, it would have to:

1. keep some controlled non-invariant information,
2. switch from hard classification to a softer statistic whose expectation matters,
3. or show that the actual learning/evaluation domain is not the balanced finite orbit space modeled by the theorem above.

15 Obstruction IVb: Invariant Feature Fibers Have Zero Margin

The most natural reply to Obstruction IV is:

Fine, hard classification on the invariant features fails. But one can still estimate a *soft score* or conditional mean on those features and only use high-margin feature values.

This is the right stronger target. The new Lean module `FiberNeutralityObstruction.lean` blocks that repair under the same involution hypotheses.

15.1 Lean theorem

Theorem 15.1 (Every invariant feature fiber is exactly balanced). *Let α be a finite sample space, let $\tau : \alpha \rightarrow \alpha$ be an involution, let $u : \alpha \rightarrow U$ be a feature map, and let $y : \alpha \rightarrow \{0, 1\}$ be the target bit. Assume*

$$u(\tau x) = u(x) \quad \text{and} \quad y(\tau x) = 1 - y(x)$$

for every $x \in \alpha$. Then for every retained feature value $v \in U$,

$$2 \cdot \#\{x : u(x) = v \text{ and } y(x) = 1\} = \#\{x : u(x) = v\}.$$

Equivalently, conditioned on any retained feature value v , the target bit has exact mean $1/2$.

The key formal statements are:

- `featureFiberTrueEquivFalse`: on each fiber $u^{-1}(v)$, the involution pairs the `true` points with the `false` points;
- `card_featureFiberTrue_eq_card_featureFiberFalse`: the two parts of each fiber have equal cardinality;
- `two_mul_card_featureFiberTrue_eq_card_featureFiber`: the `true` points occupy exactly half of each fiber.

15.2 Why this hits the soft-score repair

This is stronger than the hard-classifier obstruction. It says the retained feature map does not merely fail to support a good classifier; it fails to support *any nonzero conditional margin*. On every retained feature value v , the exact local average is already

$$\mathbb{E}[y \mid u = v] = \frac{1}{2}.$$

So if the repair route is:

1. drop the involution-sensitive coordinate,
2. keep only the invariant summary u' ,
3. estimate a soft score or conditional mean from u' ,
4. then use only “high-margin” u' -values,

it fails immediately on the balanced finite orbit model. There are no high-margin retained feature values at all.

15.3 Consequence

This narrows the remaining rescue space again. A successful patch now has to do more than “replace hard labels by soft scores.” It must either:

1. retain some controlled non-invariant information,
2. show that the actual averaging domain is not fiberwise balanced under the involution,
3. or use a structure theorem stronger than feature-invariant orbit pairing.

16 Obstruction IVc: Symmetry-Respecting Weights Still Have Zero Margin

Another natural reply is:

The real evaluation or training average is not uniform on the finite orbit space. Once one uses the correct non-uniform weighting, the invariant features may regain a useful soft margin.

The new Lean module `WeightedFiberNeutralityObstruction.lean` blocks that repair whenever the weighting itself respects the same involution symmetry.

16.1 Lean theorem

Theorem 16.1 (Orbit-symmetric weights preserve zero fiber margin). *Let α be a finite sample space, let $\tau : \alpha \rightarrow \alpha$ be an involution, let $u : \alpha \rightarrow U$ be a retained feature map, let $y : \alpha \rightarrow \{0, 1\}$ be the target bit, and let $w : \alpha \rightarrow M$ be a weight into any additive commutative monoid. Assume*

$$u(\tau x) = u(x), \quad y(\tau x) = 1 - y(x), \quad w(\tau x) = w(x)$$

for every $x \in \alpha$. Then for every retained feature value v ,

$$\sum_{x: u(x)=v, y(x)=1} w(x) = \sum_{x: u(x)=v, y(x)=0} w(x).$$

The key formal statements are:

- `weightedFeatureFiberTrueMass`: the weighted mass of the `true` part of one feature fiber;
- `weightedFeatureFiberFalseMass`: the weighted mass of the `false` part;
- `weightedFeatureFiberTrueMass_eq_weightedFeatureFiberFalseMass`: the exact balance theorem under involution-invariant weights.

16.2 Why this closes the non-uniform averaging reply

This matters because it removes a very common escape hatch. One can no longer say:

The balanced finite orbit space was just a counting toy model. The real proof uses a more careful empirical or distributional weighting, so the invariant local statistic may still have a nonzero expectation gap.

Not if that weighting respects the same sign-flip involution. The theorem shows that every such weighting still gives exact zero conditional margin on every retained feature fiber. In particular, normalizing an involution-invariant finite weight into a probability distribution still leaves

$$\mathbb{E}[y \mid u = v] = \frac{1}{2}$$

for every retained feature value v with nonzero mass.

16.3 Consequence

So the remaining escape routes are narrower again. To revive a margin on the retained local input, one must now do something stronger than:

1. pass from hard labels to soft scores,
2. pass from uniform counts to non-uniform but symmetry-respecting weights,
3. or restrict attention to invariant-feature fibers alone.

What survives must either break the involution symmetry in the weighting/domain, retain some controlled non-invariant information, or prove a much stronger ensemble-specific structural theorem.

17 Obstruction IVd: Residual Symmetric Slices Still Stay At Chance

The next natural reply is:

Fine. We keep a little non-invariant information, so the exact sign-flip symmetry is broken. We do not need to resolve every orbit, only enough of them.

This is the right next theorem to ask for. The new Lean module `ResidualSymmetryObstruction.lean` makes the burden precise: every residual slice on which the retained information still fails to distinguish the involution partners remains exact-chance under symmetry-respecting weights.

17.1 Lean theorem

Theorem 17.1 (Any unresolved symmetric slice remains balanced). *Let α be a finite sample space, let $\tau : \alpha \rightarrow \alpha$ be an involution, let $p \subseteq \alpha$ be a τ -stable slice, let $u : \alpha \rightarrow U$ be the retained local features, let $y : \alpha \rightarrow \{0, 1\}$ be the target bit, let $w : \alpha \rightarrow M$ be a weight into an additive commutative monoid, and let $h : U \rightarrow \{0, 1\}$ be any classifier. Assume on the slice p that*

$$u(\tau x) = u(x), \quad y(\tau x) = 1 - y(x), \quad w(\tau x) = w(x).$$

Then the total weighted mass of correctly classified points in p equals the total weighted mass of incorrectly classified points in p .

The key formal statements are:

- `weightedCorrectMass`: total weighted correct mass for a feature-only classifier;
- `weightedIncorrectMass`: total weighted incorrect mass;
- `weightedCorrectMass_eq_weightedIncorrectMass`: the global weighted half-accuracy theorem under exact involution symmetry;
- `weightedCorrectMass_eq_weightedIncorrectMass_on_slice`: the residual-slice version obtained by restricting to a τ -stable subtype.

17.2 Why this hits the “controlled non-invariant info” reply

This is the right quantitative obstruction. A small amount of non-invariant information can only help on the part of the domain where it actually breaks the orbit symmetry. Every remaining slice on which the feature map is still involution-invariant contributes only chance performance.

So the paper can no longer get by with a vague statement of the form:

We keep a little symmetry-breaking local information, and that should be enough to restore a useful predictor.

To make that route work, one now needs a real theorem that the unresolved symmetric slice carries negligible weight, or else a theorem that the retained information breaks the involution on almost all of the relevant mass.

17.3 Consequence

This does not yet show that *every* controlled symmetry-breaking repair fails. It does show that such a repair only helps to the extent that it resolves most of the mass. What remains must therefore prove one of:

1. the residual symmetric slice has negligible weight,
2. the retained non-invariant information separates almost all involution pairs,
3. or the actual evaluation domain is not well-modeled by a symmetry-respecting weighted slice at all.

18 Obstruction IVe: Advantage Is Bounded By Symmetry-Broken Mass

The next creative rescue move is the quantitative version of the previous one:

Maybe we only break the symmetry on a relatively small region, but that is enough because the classifier can exploit that region very efficiently and still gain a substantial global advantage.

The new Lean module `AsymmetryBudgetObstruction.lean` blocks that move in the natural finite weighted model.

18.1 Lean theorem

Theorem 18.1 (Asymmetry budget bound). *Let α be a finite sample space with weight $w : \alpha \rightarrow \mathbb{N}$. Let $p \subseteq \alpha$ be a τ -stable slice on which the retained feature map is still invariant under an involution τ , the target bit is flipped by τ , and the weights are τ -invariant. Then for every classifier h ,*

$$2 \cdot \text{CorrectMass}(h) \leq \text{TotalMass} + \text{OutsideMass}(p).$$

Equivalently, the global excess over chance is bounded by the total mass outside the unresolved symmetric slice.

Theorem 18.2 (Strict advantage exposes outside support). *Under the same hypotheses, if*

$$\text{TotalMass} < 2 \cdot \text{CorrectMass}(h),$$

then there is a concrete point x with $w(x) > 0$ and $x \notin p$. Conversely, if every positive-weight point lies in p , then strict advantage over half accuracy is impossible.

The key formal statements are:

- `two_mul_correctSliceMass_eq_sliceMass`: on the unresolved symmetric slice, the correct mass is exactly half of the slice mass;
- `weightedCorrectMass_eq_correctSlice_add_correctOutside`: total correct mass splits into the unresolved slice contribution plus the symmetry-broken contribution;
- `correctOutsideMass_le_outsideMass`: outside the unresolved slice, even perfect classification cannot use more mass than is actually there;
- `two_mul_weightedCorrectMass_le_total_plus_outside`: the final asymmetry-budget inequality;
- `not_total_lt_two_mul_weightedCorrectMass_of_support_subset`: if all positive-weight support remains inside the unresolved slice, strict advantage is blocked;
- `pos_outsideMass_of_total_lt_two_mul_weightedCorrectMass`: strict advantage forces positive outside mass;
- `exists_support_outside_of_total_lt_two_mul_weightedCorrectMass`: strict advantage exposes a positive-weight sample outside the unresolved slice.

18.2 Why this hits the last soft escape

This is stronger than saying “the residual slice is still at chance.” It says the *entire global advantage* is quantitatively budgeted by the part of the domain where symmetry is actually broken.

So one can no longer say:

We only need a little symmetry-breaking local information. Perhaps it affects a small region, but that small region could still be enough to drive the whole lower-bound argument.

Not without a theorem that the symmetry-broken region itself carries substantial mass. The Lean bound shows that if the unresolved symmetric slice has mass close to the total, then the global accuracy remains close to $1/2$. The support witness sharpens the audit: a strict global advantage claim cannot remain purely aggregate, since it must identify at least one positive-mass point where the repair has actually left the unresolved symmetric slice.

18.3 Consequence

At this point, the remaining route is no longer merely “add some asymmetry.” It must prove a real mass-transfer statement:

1. either almost all relevant mass leaves the involution-symmetric slice after the repair,
2. or the weighted domain relevant to the proof is not symmetry-respecting in the sense used above,
3. or the real predictor uses stronger ensemble-specific information than the current local symmetry analysis captures.

19 Obstruction IVf: Invariant Soft Scores Have Zero Signed Signal

Another creative move is:

Perhaps the proof should not be phrased as conditional means on feature fibers at all. Maybe one can compute a richer soft score from the invariant local summary and then feed that score into a more delicate downstream argument.

This is a real strengthening of the repair attempt. The new Lean module `InvariantScoreObstruction.lean` blocks the core signal claim under the same involution symmetry.

19.1 Lean theorem

Theorem 19.1 (Invariant soft scores have zero signed correlation). *Let α be a finite sample space, let $\tau : \alpha \rightarrow \alpha$ be an involution, let $u : \alpha \rightarrow U$ be a retained feature map, let $y : \alpha \rightarrow \{0, 1\}$ be the target bit, let $w : \alpha \rightarrow \mathbb{N}$ be a τ -invariant weight, and let $\text{score} : U \rightarrow \mathbb{Z}$ be any soft score computed only from the retained features. Assume*

$$u(\tau x) = u(x), \quad y(\tau x) = 1 - y(x), \quad w(\tau x) = w(x)$$

for every $x \in \alpha$. Writing

$$\text{sgnTarget}(y) = \begin{cases} 1 & y = 1, \\ -1 & y = 0, \end{cases}$$

one has

$$\sum_{x \in \alpha} w(x) \text{score}(u(x)) \text{sgnTarget}(y(x)) = 0.$$

The key formal statements are:

- `targetSign`: the $\{\pm 1\}$ -encoding of a Boolean target;
- `targetSign_flip`: flipping the Boolean target negates that signed encoding;
- `weighted_signedScore_sum_eq_zero`: the weighted zero-correlation theorem for any invariant soft score.

19.2 Why this hits the richer invariant-scoring repair

This closes a subtler loophole than the earlier fiber-balance statements. Those showed that the conditional mean on each invariant feature value is exactly $1/2$, and therefore that hard classifiers or direct mean estimators on those features cannot beat chance. The new theorem says more: even before one thresholds or calibrates anything, *the score itself* has zero signed signal against the target.

So one can no longer say:

We are not really using a classifier on the invariant features. We only need a good enough numerical score from those features, and then a later nonlinear step can extract the useful signal.

Not under the same involution symmetry. Any score depending only on the invariant retained features is paired with its involution partner at equal weight and opposite signed target. The positive and negative contributions cancel exactly.

19.3 Consequence

Taken together with the previous sections, this leaves very little room for an invariant-feature rescue. The remaining route must now prove something outside the invariant scoring model:

1. either the actual score retains genuinely non-invariant information,
2. or the relevant domain/weighting is not symmetry-respecting,
3. or the ensemble has a stronger special structure that invalidates the involution pairing hypotheses on the real predictor family.

20 Obstruction IVg: Invariant-Score Signal Comes Only From Weight Asymmetry

The next and now very natural reply is:

Fine. Perhaps the retained score inputs stay involution-invariant, but the real training or evaluation weighting is not exactly symmetric. Then an invariant soft score may still pick up a useful signed signal.

This is the last serious invariant-score escape hatch. The new Lean module `WeightAsymmetryObstruction.lean` makes that route exact: any invariant-score signal comes entirely from the antisymmetric part of the weighting.

20.1 Lean theorem

Theorem 20.1 (Only antisymmetric weight mass contributes to invariant-score signal). *Let α be a finite sample space, let $\tau : \alpha \rightarrow \alpha$ be an involution, let $u : \alpha \rightarrow U$ be a retained feature map, let $y : \alpha \rightarrow \{0, 1\}$ be the target bit, let $w : \alpha \rightarrow \mathbb{N}$ be an arbitrary weight, and let $\text{score} : U \rightarrow \mathbb{Z}$ be any soft score on the retained features. Assume*

$$u(\tau x) = u(x), \quad y(\tau x) = 1 - y(x)$$

for every $x \in \alpha$. Then

$$2 \sum_{x \in \alpha} w(x) \text{score}(u(x)) \text{sgnTarget}(y(x)) = \sum_{x \in \alpha} (w(x) - w(\tau x)) \text{score}(u(x)) \text{sgnTarget}(y(x)).$$

Theorem 20.2 (Nonzero invariant-score signal exposes asymmetric scored support). *Under the same hypotheses, suppose the invariant signed-score sum is nonzero:*

$$\sum_{x \in \alpha} w(x) \text{score}(u(x)) \text{sgnTarget}(y(x)) \neq 0.$$

Then there is a concrete point x such that

$$w(x) \neq w(\tau x) \quad \text{and} \quad \text{score}(u(x)) \neq 0.$$

Conversely, if $w(\tau x) = w(x)$ at every point with nonzero score, the signed-score sum is zero.

The key formal statements are:

- `signedScoreContribution`: one point’s contribution to the invariant signed-score sum;
- `antisymmetricWeightContribution`: the corresponding contribution of the weight-difference term $w(x) - w(\tau x)$;
- `two_mul_signedScore_sum_eq_antisymmetricWeight_sum`: the exact decomposition identity;
- `signedScore_sum_eq_zero_of_score_support_weight_symmetric`: zero signal when all nonzero-score points have symmetric weights;
- `exists_weight_asymmetric_score_point_of_signedScore_sum_ne_zero`: nonzero signal exposes a nonzero-score point with asymmetric weight.

20.2 Why this narrows the weighting rescue

This is stronger than the earlier exact-symmetry obstruction. It says not just that symmetric weights give zero signal, but that *all* invariant-score signal is sourced only by the antisymmetric part of the weighting. The involution-invariant part cancels exactly.

So one can no longer say:

The true weighting is probably a little asymmetric, and that may be enough.

Not without quantifying that asymmetry. The witness theorem rules out an even weaker repair where the weights are asymmetric only on score-zero points: such asymmetry contributes nothing. The proof now needs a real bound of the form:

the antisymmetric weight budget is large enough on nonzero-score mass in the relevant domain to support the claimed advantage.

20.3 Consequence

This pushes the remaining route into a much narrower quantitative corner. To save invariant local scoring, one must now prove one of:

1. the actual proof-relevant weighting has a substantial antisymmetric component on nonzero-score support,
2. the score retains genuinely non-invariant information, or
3. the real ensemble/predictor pair is not governed by the involution model above.

21 Obstruction IVh: Side Channels Help Only On Orbit-Resolving Mass

At this point the natural pivot is:

Fine. We stop asking invariant features alone to do the whole job. We keep a controlled involution-sensitive side channel v , and classify on the pair (u, v) .

This is the right next charitable target. The new Lean module `SideChannelResolutionObstruction.lean` packages the earlier symmetry budget results into the exact theorem that this move needs to face.

21.1 Lean theorem

Theorem 21.1 (Side-channel advantage is bounded by orbit-resolving mass). *Let α be a finite sample space, let $\tau : \alpha \rightarrow \alpha$ be an involution, let $u : \alpha \rightarrow U$ be a retained invariant feature map, let $v : \alpha \rightarrow V$ be an arbitrary side channel, let $y : \alpha \rightarrow \{0, 1\}$ be the target bit, let $w : \alpha \rightarrow \mathbb{N}$ be a τ -invariant weight, and let $h : U \times V \rightarrow \{0, 1\}$ be any classifier. Assume*

$$u(\tau x) = u(x), \quad y(\tau x) = 1 - y(x), \quad w(\tau x) = w(x)$$

for every $x \in \alpha$. Define the resolved mass to be the total weight of the points for which the side channel separates the involution pair:

$$\text{ResolvedMass}(v) = \sum_{x: v(\tau x) \neq v(x)} w(x).$$

Then

$$2 \cdot \text{CorrectMass}(h \circ (u, v)) \leq \text{TotalMass} + \text{ResolvedMass}(v).$$

Equivalently, the entire global excess over chance is bounded by the weight of the domain where the side channel actually resolves the orbit pairing.

The key formal statements are:

- **unresolvedBySideChannel**: the slice on which $v(\tau x) = v(x)$;
- **resolvedMass**: the weight outside that unresolved slice;
- **two_mul_weightedCorrectMass_pair_le_total_plus_resolvedMass**: the final resolution-budget inequality for classifiers on (u, v) .

21.2 Why this hits the controlled-side-channel rescue

This is stronger than the earlier residual-symmetry prose, because it says exactly what the side channel must buy. Keeping some involution-sensitive information does *not* by itself create a large advantage. The benefit is quantitatively capped by the mass where that side channel really separates involution partners.

So one can no longer say:

We only need a small side channel. Once it is present, a clever classifier on (u, v) may still recover a substantial domination advantage.

Not without a theorem that the side channel resolves involution pairs on a substantial fraction of the relevant mass. The unresolved slice remains chance, and the total excess over chance is budgeted by what lies outside it.

21.3 Consequence

This leaves the controlled-side-channel route in a much sharper form. To save it, one must now prove one of:

1. the retained side channel separates involution pairs on most of the proof-relevant mass,
2. the weighting itself has a large antisymmetric component that can be exploited,
3. or the real predictor is not well-modeled by this involution-paired finite setup.

22 Obstruction IVi: Any Domination Gap Forces Matching Resolution

The previous section bounded success by resolved mass. The converse reading is now just as important:

if the proof claims a real domination gap over chance from a retained side channel, that claim already commits it to a quantitatively comparable orbit-resolution theorem.

The new Lean module `ResolutionDemandObstruction.lean` packages that commitment as a direct corollary.

22.1 Lean theorem

Theorem 22.1 (Doubled advantage is bounded by resolved mass). *For a weighted classifier h , define the doubled advantage*

$$\text{DoubledAdv}(h) = 2 \cdot \text{CorrectMass}(h) - \text{TotalMass}.$$

Under the same involution hypotheses as in Obstruction IVh, one has

$$\text{DoubledAdv}(h \circ (u, v)) \leq \text{ResolvedMass}(v).$$

The key formal statements are:

- `doubledAdvantage`: twice the weighted success gap over chance;
- `doubledAdvantage_pair_le_resolvedMass`: the final converse bound;
- `resolvedMass_ge_of_le_doubledAdvantage_pair`: a claimed doubled gap δ forces at least δ units of orbit-resolving mass;
- `resolvedMass_ge_of_total_plus_delta_le_two_mul_weightedCorrectMass_pair`: the same burden stated directly at the half-accuracy surface;
- `not_pos_doubledAdvantage_pair_of_resolvedMass_eq_zero`: zero orbit-resolving side-channel mass blocks every positive doubled advantage;
- `resolvedMass_pos_of_pos_doubledAdvantage_pair`: any positive doubled advantage witnesses positive orbit-resolving mass;
- `resolvedMass_pos_of_total_lt_two_mul_weightedCorrectMass_pair`: strict success above half accuracy witnesses positive orbit-resolving mass without passing through the subtraction-defined gap;
- `resolvedMass_eq_zero_of_supportwise_unresolved`: if $v(\tau x) = v(x)$ at every positive-weight point, the resolved mass is zero;
- `exists_resolving_point_of_pos_resolvedMass`: positive resolved mass exposes a positive-weight point x with $v(\tau x) \neq v(x)$;
- `exists_resolving_point_of_pos_doubledAdvantage_pair`: any positive doubled advantage exposes such a point directly;

- `exists_resolving_point_of_total_lt_two_mul_weightedCorrectMass_pair`: any strict half-accuracy advantage exposes such a point directly.

The newest deterministic witness form is important for the residual-side-information repair. It is no longer enough to prove only an aggregate side-channel bound. A claimed positive advantage must identify a proof-relevant support point where the residual information really changes under the involution. If every positive-weight point is supportwise unresolved, Lean now derives the zero-resolved-mass blocker directly.

The same demand is now specialized to the manuscript’s exact post-switch fork. The theorem

`nonzeroColumnMass_ge_of_le_doubledAdvantage_invariantProjection_with_b`

says that any claimed doubled advantage from classifying on $((z, a_i), b)$ forces matching mass on nonzero a_i -columns. Its zero-mass corollary

`not_pos_doubledAdvantage_invariantProjection_with_b_of_zero_nonzeroColumnMass`

blocks the residual b -side-channel bridge whenever the proof-relevant weighted support contains no nonzero a_i -column mass. The support-witness form is now explicit:

`exists_b_resolving_input_of_pos_doubledAdvantage_invariantProjection_with_b,`

`exists_nonzeroColumn_of_pos_doubledAdvantage_invariantProjection_with_b.`

Thus a positive b -side-channel claim must produce a concrete positive-weight post-switch input whose b -coordinate changes under T_i , equivalently a positive-weight input on a nonzero a_i -column.

The latest refinement closes the immediate escape hatch under the manuscript’s exact local-input invariance premise. Lean defines

`exactInputInvariantWeightedSupport w`

to mean that every positive-weight post-switch input is fixed by the full-input involution T_i , not merely by the reduced projection. Then

`nonzeroColumnMass_eq_zero_of_exactInputInvariantWeightedSupport`

proves that this premise forces zero mass on nonzero a_i -columns, because every nonzero a_i -column changes the full input (z, a_i, b) under $b \mapsto b \oplus a_i$. Conversely,

`not_exactInputInvariantWeightedSupport_of_pos_nonzeroColumnMass`

proves that supplying positive nonzero-column mass contradicts exact-input-invariant weighted support. Combining this with the advantage bound gives

`not_pos_doubledAdvantage_and_exactInputInvariantWeightedSupport.`

Thus this repair is no longer left at “prove there is enough nonzero-column mass.” Under the same exact-input invariance premise needed by the local T_i -symmetry argument, positive b -side-channel advantage and the required support condition are formally inconsistent.

The randomized-residual escape now has the same quantitative boundary. The module `RandomizedResidualObst` models a randomized residual as a deterministic side channel on the product of the original finite sample space with a finite coin space. The theorem

`doubledAdvantage_randomizedResidual_le_resolvedMass`

proves that randomized-residual advantage is bounded by the joint mass where the coin-indexed residual separates involution partners, and

`randomizedResidualResolvedMass_ge_of_le_doubledAdvantage`

turns any claimed doubled gap δ into at least δ units of joint resolving mass. Two corollaries pin the non-escape cases:

`not_pos_doubledAdvantage_randomizedResidual_of_coinwise_unresolved`

blocks coinwise-unresolved randomized residuals, while

`not_pos_doubledAdvantage_randomizedResidual_of_fixed_source_support`

blocks arbitrary randomized residuals when all positive source weight is already fixed by the involution. The support-sensitive strengthening is

`not_pos_doubledAdvantage_randomizedResidual_of_supportwise_unresolved.`

It only requires unresolvedness on source/coin pairs with positive joint weight. Its contrapositive form is now also theorem-level:

`exists_resolving_coin_of_pos_doubledAdvantage_randomizedResidual.`

Thus randomization is not a free repair of the residual side-information gap; positive finite-coin advantage must exhibit a real positive-weight source/coin pair where the residual changes across the involution. The newest strengthening removes the last possible “only in the mixture” reading of that witness. From positive randomized resolving mass Lean now extracts a deterministic positive-weight coin slice with positive ordinary side-channel resolving mass:

`exists_positive_coin_resolvedMass_of_pos_randomizedResidualResolvedMass.`

From positive randomized advantage it gets the same conclusion directly:

`exists_positive_coin_resolvedMass_of_pos_doubledAdvantage_randomizedResidual.`

So a finite-coin randomized residual repair cannot defer all symmetry breaking to an aggregate average over coins. One actual coin value of positive coin-weight already carries a standard deterministic side-channel obstruction. The new strengthening shows that Lean can package that deterministic slice all the way up to the ordinary residual-side-information obstruction packet:

`exists_positive_coin_obstruction_package_of_pos_doubledAdvantage_randomizedResidual`

and

`exists_positive_coin_obstruction_package_of_total_lt_two_mul_weightedCorrectMass_randomizedRes.`

So even before specializing to the manuscript’s exact post-switch surface, the finite-coin randomized-residual route no longer stands apart from the ordinary residual-side lane: strict half-accuracy advantage already forces a positive-weight deterministic coin slice carrying the same non-determination, collision, resolving-point, and source-only-failure witnesses.

The post-switch specialization now removes that remaining randomized repair under the exact support premise already used above. The module `PostSwitchRandomizedResidualObstruction.lean` proves

`randomizedResidualResolvedMass_tiInputMap_eq_zero_of_exactInputInvariantWeightedSupport.`

So exact-input-invariant weighted support on (z, a_i, b) leaves zero joint resolving mass for every finite-coin randomized residual. Equivalently,

`not_exactInputInvariantWeightedSupport_of_pos_randomizedResidualResolvedMass_tiInputMap`

says that supplying positive randomized joint resolution contradicts the support premise. The combined theorem

`not_pos_doubledAdvantage_randomizedResidual_and_exactInputInvariantWeightedSupport`

blocks positive finite-coin randomized residual advantage on the exact post-switch surface whenever the route keeps exact local-input invariance of its positive-weight support. The supportwise post-switch version,

`not_pos_doubledAdvantage_randomizedResidual_invariantProjection_of_supportwise_unresolved,`

removes the need to assume fixed support globally: it is enough that every positive-weight input/coin pair has the same residual value after the local-input switch. Conversely,

`exists_resolving_coin_of_pos_doubledAdvantage_randomizedResidual_invariantProjection`

says that any positive finite-coin randomized-residual advantage on the exact post-switch surface must expose a positive-weight input/coin pair resolving that switch. The deterministic-slice form is now specialized too:

`exists_positive_coin_resolvedMass_of_pos_doubledAdvantage_randomizedResidual_invariantProjection`

Thus on the exact post-switch surface, any positive finite-coin randomized advantage must identify a positive-weight coin whose ordinary residual side channel has positive mass resolving the local-input switch. The same boundary now holds on the strict half-accuracy surface. Lean proves

`not_total_lt_two_mul_weightedCorrectMass_randomizedResidual_invariantProjection_of_exactInputInvariantWeightedSupport`

and

`not_total_lt_two_mul_weightedCorrectMass_randomizedResidual_and_exactInputInvariantWeightedSupport`

so exact-input-invariant support blocks not only positive doubled advantage but also any claim that a finite-coin randomized residual beats half accuracy on the exact post-switch surface. Conversely,

`exists_positive_coin_resolvedMass_of_total_lt_two_mul_weightedCorrectMass_randomizedResidual_invariantProjection`

and

`not_exactInputInvariantWeightedSupport_of_total_lt_two_mul_weightedCorrectMass_randomizedResidual_invariantProjection`

show that even this weaker-looking route already exposes a positive-weight deterministic coin slice with ordinary resolving mass and therefore directly refutes the manuscript’s exact-support premise. The collapse is now stronger:

`exists_positive_coin_obstruction_package_of_total_lt_two_mul_weightedCorrectMass_randomizedRes`

shows that the same strict-half post-switch witness already carries the full ordinary invariant-projection residual obstruction package, not merely positive resolving mass. So the post-switch randomized-residual repair no longer survives by retreating from positive doubled advantage to the strict half-accuracy formulation.

`CruxSynthesis.lean` now records this as the narrow `PNPRandomizedResidualSubrepairClass` subledger, backed by the theorem `V13RandomizedResidualSubcoverage`. The synthesis layer no longer leaves these finite-coin witnesses only in the local obstruction files: it explicitly records the ordinary source/coin product-space budget, the supportwise-unresolved zero-mass and no-advantage blockers, the deterministic positive-weight coin-slice witness, and the exact post-switch analogues. This remains deliberately narrow. The broad `PNPRepairClass.randomizedResidual` entry is still uncovered in the current packet; the new synthesis theorem only says that any finite-coin randomized-residual repair must surface one of those explicit witnesses or fail the present Lean boundary.

22.2 Why this matters

This is the strongest current version of the side-channel blocker. It no longer says only that resolved mass is *sufficient* for advantage. It says a claimed advantage is itself evidence that substantial resolution must already be present.

So one can no longer say:

Perhaps the predictor gets a modest but important edge from a very small side channel, and only later amplifies that edge through clever structure.

Not unless that modest edge is matched by actual orbit-resolving mass. The theorem makes the burden explicit: any nontrivial domination statement already entails a nontrivial local resolution statement.

22.3 Consequence

This is close to the form needed to challenge the route on its own terms. To save the current style of proof, one now needs a theorem of the form:

the switched predictor’s retained side channel, deterministic or randomized, resolves involution pairs on enough relevant mass to support the claimed lower-bound-driving advantage.

Without that, the claimed domination gap has no quantitative room to exist.

23 Obstruction V: Short Programs Still Cover the Full Log-Width Local Rule Class

Another natural reply is:

The switched predictor comes from a short global program, so even if arbitrary local rules are huge, only a much smaller subclass can actually arise from polynomial-time decoders.

This is not true without an additional restriction beyond mere global shortness.

23.1 Lean theorem

The new module `ShortProgramObstruction.lean` formalizes the basic truth-table fact.

Theorem 23.1 (Truth tables realize all local rules). *For every n , there is a surjective decoder*

$$\{0, 1\}^{2^n} \twoheadrightarrow \{f : \{0, 1\}^n \rightarrow \{0, 1\}\}.$$

Equivalently, every Boolean rule on n visible bits can be hard-wired by a truth table of length 2^n .

The key formal statements are:

- `truthTableDecode_surjective`: every local rule is realized by a 2^n -bit truth table;
- `truthTableFamily_realizes_all`: the realized class of the truth-table family is the full local-rule space;
- `card_realized_truthTableFamily`: this realized class has exactly cardinality 2^{2^n} .

23.2 Binary-log consequence

At binary-log width the truth-table size is already linear in the ambient parameter:

$$2^{\lfloor \log_2 m \rfloor + 1} \leq 2m \quad (m \neq 0).$$

This is formalized as `truthTableBits_binaryLogWidth_le_twoMul`.

So if the switched interface really has only $n = \lfloor \log_2 m \rfloor + 1$ visible bits, then a merely linear-size code budget already suffices to realize the *entire* local-rule class on that interface.

23.3 Consequence

This directly blocks the short-program rescue in its naive form:

“The global decoder is polynomial-size” does not imply “the induced local rule lies in a tiny subclass.”

On the contrary, polynomial-size descriptions are already large enough to hard-wire every Boolean rule on $O(\log m)$ visible bits. So if one wants a true small subclass theorem, it must use something much stronger than global shortness alone: symmetry, algebraic normal form, a tree interpreter with bounded message alphabet, or some other genuine structural restriction.

24 Obstruction VI: Exact Metagraph Quotients Have Interface-State Explosion

The strongest remaining charitable rescue is roughly this:

The switched neighborhood first collapses to a canonical metagraph or message-passing quotient, and the witness bit is computed by a fixed interpreter on that quotient.

This is a serious idea. It is also constrained by a generic exactness barrier.

24.1 Lean theorem

The module `MetagraphObstruction.lean` models a quotient state by its exact interface behavior on k visible boundary bits. The semantic object is then a boundary transducer

$$(\{0, 1\}^k \rightarrow \{0, 1\}^k).$$

Theorem 24.1 (Boundary transducers are exponentially numerous). *There are exactly*

$$2^{k2^k}$$

boundary transducers on k visible bits.

The key formal statements are:

- `card_boundaryMap`: exact k -bit interface semantics have cardinality 2^{k2^k} ;
- `no_surjective_bitCode_to_boundaryMap_of_lt`: no s -bit code can cover the full interface-semantic space when $s < k2^k$;
- `boundaryMap_card_le_of_surjective_interpret`: any finite state space whose interpreter covers all exact boundary transducers has at least 2^{k2^k} states;
- `not_surjective_quotientCode_boundaryMap_of_stateCard_lt`: factoring an arbitrary code through a smaller quotient state space cannot repair this obstruction;
- `boundaryMap_stateCard_lowerBound_binaryLogWidth`: at $k = \lfloor \log_2 m \rfloor + 1$, exact interface coverage forces at least 2^m quotient states;
- `no_exact_metagraphQuotient_below_inputSize_binaryLogWidth`: at $k = \lfloor \log_2 m \rfloor + 1$, there is no exact quotient with fewer than m code bits.

The newest extension closes a small but important loophole in the first version of this obstruction. One could try to let a large code first choose a finite metagraph state, and then use a fixed interpreter from states to exact boundary behavior. Surjectivity of the composite already makes the interpreter surjective, so the state space itself must have the full boundary-map cardinality. At binary-log boundary width, any exact universal quotient-state repair therefore requires at least 2^m states, even before accounting for how those states are encoded.

24.2 Why this matters

If a metagraph/message quotient is meant to be *exact*, then it must preserve enough state to reconstruct the gadget's interface behavior under gluing. The theorem above says that, in the absence of an additional structural restriction, the exact interface-semantic space is already far too large for a tiny finite-state algebra.

This does not yet show that the current ensemble realizes the full transducer class. What it does show is that a small exact quotient theorem cannot follow from tree-likeness or locality alone. To survive, the metagraph route needs a new theorem that the actual switched/VV gadgets lie in a drastically smaller semantic subclass.

24.3 Consequence

So the metagraph rescue is now narrowed to something like:

not “there is an exact small quotient of all local interfaces,” but “the specific random masked+isolated interfaces collapse to a much smaller algebra for ensemble-specific reasons.”

That is a legitimate possible theorem. It is also substantially stronger than anything currently written in the paper, and it is the next point where a decisive break may occur.

25 Obstruction VII: Tree-Likeness Alone Does Not Yield a Small Message Class

Another natural repair is:

On locally tree-like neighborhoods, the witness bit is computed by a small message-passing or dynamic-program interpreter on the tree.

This too needs a stronger premise than mere tree structure.

25.1 Lean theorem

The new module `TreeMessageObstruction.lean` defines a fixed perfect binary tree architecture of depth n , where internal nodes merely branch on the next input bit and only the leaves carry data.

Theorem 25.1 (Perfect binary trees realize all local rules). *For every Boolean rule*

$$f : \{0, 1\}^n \rightarrow \{0, 1\},$$

there is a depth- n perfect binary decision tree whose evaluation equals f .

The key formal statements are:

- `eval_encodeDecisionTree`: every local rule is exactly realized by a leaf-labeled depth- n perfect tree;
- `evalDecisionTree_injective` and `evalDecisionTree_bijective`: for this fixed perfect architecture, extensional equality of semantics is exactly equality of the leaf labels;
- `evalDecisionTree_surjective`: the semantics map from such trees onto `LocalRule n` is surjective;
- `no_surjective_bitCode_to_decisionTreeSemantics_of_lt`: therefore no s -bit exact code can cover all depth- n tree semantics when $s < 2^n$;
- `localRule_card_le_of_surjective_treeMessageInterpreter`: any finite tree-message state space whose interpreter covers all exact depth- n semantics has at least 2^{2^n} states;
- `not_surjective_quotientCode_treeMessageInterpreter_of_stateCard_lt`: factoring a code through a smaller finite tree-message state cannot repair the obstruction;
- `treeMessage_stateCard_lowerBound_binaryLogDepth`: at depth $n = \lfloor \log_2 m \rfloor + 1$, exact coverage forces at least 2^m message states.

25.2 Why this matters

This is a direct answer to the naive tree-message rescue:

tree-likeness by itself does *not* imply semantic simplicity.

Even with an extremely rigid tree architecture, the semantic class is already the full Boolean function space on the visible inputs. The newest Lean strengthening also removes the “many trees may collapse to few semantics” escape for this architecture: evaluation is injective, so the semantics are exactly the leaf truth table. If a repair instead proposes a finite tree-message state space plus a fixed interpreter to exact depth- n trees, surjectivity onto all exact semantics forces that state space to have the full local-rule cardinality. So if one wants a genuine tree-message theorem, it must use special properties of the actual SAT+VV messages, not just the fact that the underlying graph is a tree.

25.3 Consequence

The surviving charitable tree route is now much narrower:

not “all tree-like local neighborhoods admit a tiny interpreter,” but “the specific masked+isolated tree messages belong to a sharply smaller algebra.”

That is again a real possible theorem. It is also much stronger than the current text, and it is precisely the kind of statement that now has to be made explicit to keep this proof approach alive.

26 What These Results Do Not Yet Prove

The current Lean modules do *not* prove that the full argument fails. In particular, they do not yet address:

1. whether the masking and isolation ensemble enforces extra algebraic structure on local rules;
2. whether the switched predictors might collapse to a subclass analogous to low-depth message passing or tree computations;
3. whether the neutrality and sparsification lemmas are themselves correct in the precise form used in the paper;
4. whether a different, stronger switching theorem could bypass the present obstruction.

So the current status is best described as:

The proof attempt is not yet broken in full, but one of its central complexity-compression transitions has not been justified, and the default combinatorics point in the opposite direction.

27 Stop-Grade Synthesis Ledger

The newest meta-synthesis module, `CruxSynthesis.lean`, makes the status discipline formal. It defines a coarse type of live repair classes and represents a crux packet by the repair classes it covers. A packet is *stop-grade* only if it covers every named repair class in that ledger:

$$\text{PNPCruxPacket.StopGrade packet} \quad \equiv \quad \forall r : \text{PNPRepairClass}, \text{packet.covers } r.$$

The current accumulated PNP packet covers the deterministic same-source conditioning repairs, the v13 repeated-pivot, forced-step, and bad-cell fixed-field repairs, field refinement of a bad cell, invariant-score repairs, bounded side-channel repairs, short-global-decoder repairs, CD evidence normalization as an exact residual-atom boundary, trace capture as the full finite trace-fiber/cardinality/image-budget boundary, the atomic evidence budget as the full finite exactness boundary, the same-source finite-count approximation subcase, and the abstract clocked finite-uniform kernel behind the later *Kpoly* counting step. Lean records this exact covered list as `currentPNPCoveredRepairClasses`.

The v13 entries in this ledger are now tied directly to the v13 route modules rather than recorded as bare Boolean placeholders. `CruxSynthesis.lean` imports `PNPv13RouteSynthesis.lean`, `PNPv13ForcedStepObs` and `PNPv13FieldRefinement.lean`, defines the coverage propositions

`V13RepeatedPositiveFieldedPivotCoverage`, `V13ForcedPositiveFieldedStepCoverage`, `V13FieldedBadCellCoverage`, `V13FieldRefinementBadCellCoverage`, `V13EvidenceNormalizationCoverage`, `V13TraceCaptureFactorizationCoverage`, `V13AtomicEvidenceBudgetCoverage`,

and proves

`currentPNPLocalCruxPacket_theoremBacked_v13_status`.

That theorem says exactly that those seven v13 ledger entries are covered while the current packet is still not stop-grade. This is a route-interface claim: it does not assert that a SAT/Cook–Levin construction, or the manuscript’s actual switched family, has been formalized inside the ledger.

Inside that synthesis layer, the atomic-budget entry is no longer just the original one-sided no-double-counting statement. The proposition `V13AtomicEvidenceBudgetCoverage` now packages the dual under-budget omission witness, the exactness-side conjunction `ExactnessSideConditions`, the equality-surface reductions from exact budget balance to either structural side condition, and the cancellation anchor showing that exact scalar equality plus `NoDoubleCounting` $\leftrightarrow$ `AllPositiveCostAtomsCharged` still does not repair the budget step.

The trace-capture entry in that same synthesis layer has now been promoted in parallel. `V13TraceCaptureFactorizationCoverage` no longer stops at deterministic/randomized fiber-constancy equivalences. It also packages the injectivity criterion `CapturesAllBoolObservers` $\leftrightarrow$ `Function.Injective trace`, the finite trace-alphabet lower bound $|\text{relevant states}| \leq |\text{Trace}|$, and the Boolean decoder-space lower bound $2^{|\text{relevant states}|} \leq 2^{|\text{Trace}|}$. So the synthesis packet now carries not only the pointwise collapsed-fiber conflict but also the finite cardinality and finite image-budget obstructions for the same route class.

The refinement route itself has now been strengthened one layer further. `PNPv13FieldRefinementRouteObstruction` lifts the earlier prefix-level refinement theorem to the whole fielded switching list: if a coarse field already has a bad cell after some explicit fielded prefix, then replacing that step’s field by any genuine refinement still leaves a localized bad cell at the same position. Lean then blocks both `V13FieldSwitchingInstantiatedFrom` and `V13FieldSwitchingFailureMatchingFrom` for the refined suffix by routing through `V13FieldedBadCellFrom`. The Boolean-square regression fixes a concrete two-step example where a one-cell second cut is refined to the second-coordinate field and the whole route still fails.

`PNPv13EvidenceNormalization.lean` supplies the fourth entry. It abstracts a CD evidence atom by four predicates: target relevance, neutrality, safe-buffer absorption, and hidden-gauge absorption. Lean proves

`normalizesNonNeutral_iff_not_hasResidualAtom`,

so the manuscript’s evidence-normalization theorem is exactly the claim that there is no target-relevant non-neutral atom outside both safe-buffer and hidden-gauge classes. The positive-score weakening is also exact:

`positiveAtomsCovered_iff_not_positiveResidualScore.`

The three-atom toy model `v13ToyEvidenceSurface` has one safe atom, one gauge atom, and one positive-score residual atom. The synthesis anchor `v13EvidenceNormalizationCoverage_anchor_toyResidual_b` therefore records the current crux without overstating it: any repair must prove that the actual CD construction has no such residual positive atom, not merely rename residual evidence as safe or gauge.

`PNPv13TraceCapture.lean` supplies the next v13 ledger boundary. It abstracts a trace map on target-relevant states and proves

`observerTraceDecodable_iff_constantOnTraceFibers.`

Thus replacing an adaptive observer by a finite CD trace is exactly a quotient factorization theorem: the observer’s output must be constant on every target-relevant trace fiber. The file also proves the all-Boolean-observer endpoint

`capturesAllBoolObservers_iff_not_relevantTraceCollision.`

So a trace that collapses two target-relevant states cannot capture every Boolean downstream observer. The toy surface `v13CollapsedTraceSurface` collapses two relevant states to one trace while `v13ToyTraceObserver` separates them, and the synthesis anchor `v13TraceCaptureFactorizationCoverage_anchor` records the corresponding non-decodability and universal-Boolean-capture failure. This does not prove that the manuscript’s actual CD traces collide; it pins the exact theorem any trace-capture repair must supply.

The next strengthening turns that qualitative boundary into a finite-alphabet lower bound. The module `PNPv13TraceCaptureCardinalityObstruction.lean` defines the restricted map

`traceOnRelevant : {s | targetRelevant s} → Trace`

and proves

`capturesAllBoolObservers_iff_traceOnRelevant_injective.`

Thus universal Boolean trace capture is not merely absence of one explicit collision witness; it is injectivity of the proposed trace alphabet on the entire target-relevant support. Consequently any finite trace alphabet must satisfy

$$|\{s \mid \text{targetRelevant } s\}| \leq |\text{Trace}|,$$

formalized as

`relevantStateCard_le_traceCard_of_capturesAllBoolObservers`

and contrapositively

`not_capturesAllBoolObservers_of_traceCard_lt_relevantStateCard.`

The regression file records both a positive canary and finite blockers: identity traces on all relevant Boolean states do capture every Boolean observer, but one trace for two relevant states and two

traces for four relevant states are already impossible. So a manuscript repair that proposes finite CD traces must now prove a target-relevant injective coding theorem, not just an abstract factorization slogan.

The next file places this same obstruction on the standard finite-cover route used elsewhere in the project. The module `PNPv13TraceCaptureImageBudgetObstruction.lean` defines `traceDecoderFamily`, whose indices are Boolean decoders $\text{Trace} \rightarrow \text{Bool}$ and whose inputs are the target-relevant states themselves, represented as the subtype $\{s \mid \text{targetRelevant } s\}$. Universal Boolean trace capture then makes this indexed family surjective onto the full Boolean classifier space on the relevant support, via

`traceDecoderFamily_surjective_of_capturesAllBoolObservers.`

But the family of all trace decoders has size only $2^{|\text{Trace}|}$, so it carries an explicit finite predictor-image cover of that same size. Feeding this into the project’s standard finite-image lower bound yields

$$2^{|\{s \mid \text{targetRelevant } s\}|} \leq 2^{|\text{Trace}|},$$

formalized as

`boolClassifierCard_le_traceClassifierCard_of_capturesAllBoolObservers`

and contrapositively

`not_capturesAllBoolObservers_of_traceClassifierCard_lt.`

So the manuscript’s trace-capture route is now blocked not only pointwise by an injectivity obligation, but also globally in the same finite predictor-image language that the later ‘Kpoly’ bridge already uses. The regression file records the positive identity-trace canary and the same two toy blockers in the image-budget form: one trace cannot realize all four Boolean classifiers on two relevant states, and two traces cannot realize all sixteen Boolean classifiers on four relevant states.

`PNPv13RandomCoinTraceCapture.lean` tightens the same trace-capture entry at the manuscript’s random-coin reduction point. A coin-indexed observer $A(s, c)$ is captured by a trace/coin decoder exactly when every deterministic coin slice $s \mapsto A(s, c)$ is trace-decodable:

`randomObserverTraceDecodable_iff_forall_coin_observerTraceDecodable.`

With an inhabited output type this is equivalently coinwise constancy on target-relevant trace fibers, via

`randomObserverTraceDecodable_iff_constantOnTraceFibers.`

Thus random coins do not repair a quotient collision; they only index a family of deterministic quotient obligations. The collapsed two-state trace now has `v13ToyRandomTraceObserver`, and the synthesis anchor `v13TraceCaptureFactorizationCoverage_anchor_collapsedRandomTrace_blocks` records that one conflicting coin slice already blocks randomized trace decodability.

`PNPv13AtomicEvidenceBudget.lean` supplies the next v13 ledger boundary. It abstracts coordinates charging evidence atoms with natural-number costs. Lean defines the coordinate-summed cost by grouping by atom multiplicity: an atom charged by m coordinates contributes m times its cost. The central theorem

`coordinateSummedCost_le_atomBudget_of_noDoubleCounting`

proves that the once-per-atom budget bounds the coordinate-summed cost only under the explicit no-double-counting premise. The converse diagnostic

`doubleCountedAtom_of_atomBudget_lt_coordinateSummedCost`

proves that any strict over-budget coordinate sum exposes a positive-cost atom with multiplicity at least two. The toy surface `v13DoubleCountedBudgetSurface` has two coordinates charging one shared unit-cost atom, so `atomBudget = 1` while `coordinateSummedCost = 2`. The synthesis anchor `v13AtomicEvidenceBudgetCoverage_anchor_doubleCountedAtom_blocks` therefore fixes the exact repair theorem: a manuscript budget bound must prove no positive-cost evidence atom is double-counted, or else pay the multiplicity.

`PNPv13AtomicEvidenceBudgetExactnessCharacterization.lean` adds the missing dual obstruction. It defines `AllPositiveCostAtomsCharged` and `MissedPositiveCostAtom` and proves

`atomBudget_le_coordinateSummedCost_of_allPositiveCostAtomsCharged,`

so charging every positive-cost atom at least once forces the once-per-atom budget to be a lower bound on the coordinate-summed cost. The converse diagnostic

`missedPositiveCostAtom_of_coordinateSummedCost_lt_atomBudget`

shows that any strict under-budget coordinate sum exposes a positive-cost atom with multiplicity zero. Finally,

`coordinateSummedCost_eq_atomBudget_of_noDoubleCounting_of_allPositiveCostAtomsCharged`

proves the exact reuse criterion: a manuscript that wants to identify the coordinate-summed cost with the once-per-atom budget must prove both that no positive-cost atom is double-counted and that no positive-cost atom is omitted. The toy surface `v13MissedPositiveBudgetSurface` charges one unit-cost atom and misses another, so `coordinateSummedCost = 1` while `atomBudget = 2`. This blocks the creative repair where the author avoids multiplicity inflation only by silently leaving some positive-cost evidence atoms unpaid.

`PNPv13AtomicEvidenceBudgetCancellationObstruction.lean` blocks the next escape hatch. The toy surface `v13CancellationBudgetSurface` lets both coordinates charge atom 0 while atom 1 is never charged, with both atoms having unit cost. Lean proves

`coordinateSummedCost = atomBudget = 2`

on that surface, while simultaneously proving `DoubleCountedAtom` and `MissedPositiveCostAtom`. The theorems

`not_forall_coordinateSummedCost_eq_atomBudget_implies_noDoubleCounting,`

`not_forall_coordinateSummedCost_eq_atomBudget_implies_allPositiveCostAtomsCharged,`
and

`not_forall_coordinateSummedCost_eq_atomBudget_implies_exactness_side_conditions`

make the route consequence explicit: exact aggregate budget balance does not entail either structural side condition, nor their conjunction. So the manuscript cannot replace per-atom accounting theorems by a single scalar equality and claim the budget step is repaired by net cancellation.

PNPv13AtomicEvidenceBudgetEqualitySharpness.lean closes the next repair escape that remains after the cancellation countermodel. The cancellation toy surface shows that exact equality alone does not imply either structural side condition, but the new file proves that once either side condition is supplied, the other is forced by equality. Concretely,

`coordinateSummedCost_lt_atomBudget_of_noDoubleCounting_of_missedPositiveCostAtom`

shows that under no-double-counting, any missed positive-cost atom makes the coordinate-summed cost strictly too small, while

`atomBudget_lt_coordinateSummedCost_of_allPositiveCostAtomsCharged_of_doubleCountedAtom`

shows that if every positive-cost atom is charged, then any double-counted positive-cost atom makes the coordinate-summed cost strictly too large. Therefore the equality-surface theorems

`allPositiveCostAtomsCharged_of_coordinateSummedCost_eq_atomBudget_of_noDoubleCounting`

and

`noDoubleCounting_of_coordinateSummedCost_eq_atomBudget_of_allPositiveCostAtomsCharged`

prove that on the exact-budget surface, the two structural obligations are equivalent. This does not rescue the manuscript's route, because the same cancellation surface still satisfies exact equality while violating both conditions; instead it sharpens the crux by isolating the precise missing content of any equality-based repair. The author now needs a real theorem for at least one side condition, not just an aggregate scalar identity.

PNPv13AtomicEvidenceBudgetExactnessReduction.lean makes that last sentence formal and route-facing. It packages the real budget-step obligation as

`ExactnessSideConditions := NoDoubleCounting ∧ AllPositiveCostAtomsCharged`

and proves that this conjunction is exactly what suffices for exact budget reuse. On the equality surface, Lean then proves

`ExactnessSideConditions ↔ NoDoubleCounting`

and also

`ExactnessSideConditions ↔ AllPositiveCostAtomsCharged.`

So once a manuscript proves either structural side condition, it has proved the whole exactness packet. But the same file also proves the sharper obstruction

`not_forall_coordinateSummedCost_eq_atomBudget_and_sideConditionEquivalence_implies_exactnessSi`

The cancellation surface is again the witness: it satisfies exact scalar equality and the equality-surface equivalence `NoDoubleCounting ↔ AllPositiveCostAtomsCharged`, yet still fails `ExactnessSideConditions`. This blocks the creative repair where one cites only the induced equivalence of the two side conditions and silently treats that as if one of them had been proved.

PNPv13ACCEIEnvelope.lean adds the finite skeleton for the manuscript's ACCEI/PNLD envelope step without asserting the analytic envelope hypotheses. The Markov-pruning theorem

`acceiBadCount_mul_succ_threshold_le_of_sum_le`

shows that, for a finite coordinate set, a total envelope budget bounds the number of coordinates whose envelope exceeds a threshold. The strengthened finite pruning packet records the complement and division forms

`acceiBadCount_eq_card_sub_acceiGoodCount,      acceiBadCount_le_budget_div_succ_threshold_of_sum`

and the large-good-set lower bound

`card_sub_budget_div_succ_threshold_le_acceiGoodCount_of_sum_le.`

Thus the finite consequence of an averaged ACCEI budget is exactly a Markov loss of at most $\text{budget}/(\text{threshold} + 1)$ bad coordinates. Lean also proves

`exists_accei_good_of_budget_div_succ_threshold_lt_card,`

so at least one coordinate survives only when that finite loss is strictly smaller than the switched-coordinate set. This pins the repair obligation: the manuscript cannot infer an arbitrarily large pruned coordinate set from a global average envelope without paying this explicit loss term or proving a stronger envelope hypothesis. The sharpness packet makes this obstruction concrete. The envelope

`oneGoodOneBadACCEIEnvelope`

has one coordinate at 0 and one coordinate at $\text{threshold} + 1$. Lean proves

`oneGoodOneBadACCEIEnvelope_markov_badCount_sharp,      oneGoodOneBadACCEIEnvelope_good_lower_bound`

so both the bad-count Markov inequality and the good-coordinate lower bound are met with equality. It also proves

`oneGoodOneBadACCEIEnvelope_not_all_good,`

showing that a total budget of $\text{threshold} + 1$ does not imply all coordinates survive pruning. The sequential theorem

`finiteHistory_product_bound_of_sequentialRateFrom`

generalizes the earlier half-step tower product: if every next event has finite one-step count rate at most $\text{numerator}/\text{denominator}$, then the whole history satisfies the corresponding product bound. Its contrapositive

`not_sequentialRateAdmissibleFrom_of_product_bound_violation`

is the Lean-side obstruction: any proposed ACCEI/PNLD pruning route that violates the finite product bound cannot have the required one-step rate certificate. The stronger localization theorem

`exists_failed_prefixRateStep_at_append_cons_of_product_bound_violation`

turns such a violation into an explicit preceding history, next event, and remaining suffix where the claimed one-step rate inequality fails. Thus a repair cannot hide behind a global product statement; it must supply every local conditional rate cut. The synthesis ledger records this as `V13ACCEIEnvelopeProductCoverage`; this is deliberately only the finite counting skeleton, not a proof that the manuscript's stopped-observer ACCEI envelope exists or has the claimed budget.

The same module now gives the finite-coin stack its own conservative route-coverage map, rather than letting the surrounding prose imply more than the Lean theorems prove. Lean names the

covered finite-coin subrepair classes as `PNPFiniteCoinSubrepairClass` and records the current covered list as `currentPNPFiniteCoinCoveredSubrepairs`. The covered items are:

`sameSourceFiniteCoinMarginalRecoding`
`predicateErasure`
`decoderHalfAccuracy`
`exactScaledFiberImbalance`
`subCardinalityToleranceQuantization`
`sourceLowerBoundToleranceQuantization`
`retainedCoinTolerancePointwiseCollapse`
`retainedSideToleranceSideFiberBalance`
`sourceNonconstantUniformFiberZeroBlock`
`sourceNonconstantZeroIffBalanced`
`deterministicOutputPostprocessing`
`injectivePostprocessingErasureEquivalence`
`observedLeftInversePostprocessingErasureEquivalence`
`observedInjectivePostprocessingErasureEquivalence`
`postprocessingObservedOutputCollisionObligation`
`postprocessingObservedOppositeBiasCollisionObligation`
`postprocessingSignedBiasCancellationCertificate`
`postprocessedSideToleranceObservedFiberBalance`
`postprocessedSideObservedFiberImbalanceLowerBound`
`postprocessedSidePredicateErasure`
`postprocessedSideDecoderHalfAccuracy`
`postprocessedSideSourceRecoveryBlock`
`postprocessedSideCollisionObligation`
`postprocessedSidePointwiseCollisionObligation`
`postprocessedSidePointwiseCollisionInsufficient`
`postprocessedSideOneSidedCoinMapInsufficient`
`postprocessedSideInjectiveWitnessMapEquivalence`
`postprocessedSideCoinMatchingObligation`
`postprocessedSideCoinMatchingCardinalityBlock`
`postprocessedSideCoinMatchingPredicateBlock`
`postprocessedSidePredicateMatchingEquivalence`
`postprocessedSideApproxIndependenceCoinMatchingEquivalence`
`finitePreimagePostprocessingBlowup`

same-source finite-coin marginal recoding
 balanced fibers erase every decidable output
 balanced fibers force every Boolean decoder
 each output-fiber defect is $|\text{Coin}|$ times in
 tolerance below $|\text{Coin}|$ collapses to exact
 tolerance below a source/coin block collapse
 retained-coin tolerance below a source/coin
 retained-side tolerance below a source/coin
 source-side nondegeneracy blocks uniform
 source-side nondegenerate zero tolerance
 exact erasure is stable under deterministic
 lossless output maps cannot create finite
 support-recoverable postprocessing cannot
 support-injective postprocessing cannot
 postprocessed repair of imbalance must
 a balancing collision must merge opposite
 postprocessed repair requires zero signed
 postprocessed retained-side tolerance col
 postprocessed observed-fiber imbalance f
 postprocessed downstream predicates ha
 postprocessed downstream decoders are
 postprocessed downstream source recove
 postprocessed independence forces true/
 every observed coin value is witnessed on
 pointwise support overlap need not prov
 one-sided witness maps need injectivity/
 injective witness maps are exactly coin m
 observed finite multiplicities support a p
 one observed multiplicity mismatch bloc
 one downstream predicate mismatch blo
 all downstream predicates neutral iff coi
 below threshold, approximate independe
 finite-output postprocessing changes τ to

The theorem `currentPNPFiniteCoinCoveredSubrepairs_exact` pins this list on the Lean side.

Several anchor theorems tie the list back to actual mathematics rather than bookkeeping alone:

```

finiteCoinCoverage_anchor_predicateErasure, finiteCoinCoverage_anchor
finiteCoinCoverage_anchor_exactScaledFiberImbalance, finiteCoinCoverage_anchor
finiteCoinCoverage_anchor_sourceLowerBoundToleranceQu
finiteCoinCoverage_anchor_retainedCoinTolerancePointw
finiteCoinCoverage_anchor_retainedSideToleranceSideFi
finiteCoinCoverage_anchor_deterministicOutputPostprocessing, finiteCoinCoverage_anchor
finiteCoinCoverage_anchor_observedLeftInversePostprocessingE
finiteCoinCoverage_anchor_observedInjectivePostprocessingEr
finiteCoinCoverage_anchor_postprocessingObservedOutputColl
finiteCoinCoverage_anchor_postprocessingObservedOppositeBiasC
finiteCoinCoverage_anchor_postprocessingSignedBiasCancellat
finiteCoinCoverage_anchor_postprocessedSideToleranceObserv
finiteCoinCoverage_anchor_postprocessedSideObservedFiberImb
finiteCoinCoverage_anchor_postprocessedSidePredicate
finiteCoinCoverage_anchor_postprocessedSideDecoderHal
finiteCoinCoverage_anchor_postprocessedSideSourceRecor
finiteCoinCoverage_anchor_postprocessedSideCollision
finiteCoinCoverage_anchor_postprocessedSidePointwiseColli
finiteCoinCoverage_anchor_postprocessedSidePointwiseCollis
finiteCoinCoverage_anchor_postprocessedSideOneSidedCoinMa
finiteCoinCoverage_anchor_postprocessedSideInjectiveWitnes
finiteCoinCoverage_anchor_postprocessedSideCoinMatchin
finiteCoinCoverage_anchor_postprocessedSideCoinMatchingCa
finiteCoinCoverage_anchor_postprocessedSideCoinMatchingP
finiteCoinCoverage_anchor_postprocessedSidePredicateMatch
finiteCoinCoverage_anchor_postprocessedSideApproxIndependenceCo
finiteCoinCoverage_anchor_finitePreimagePostprocess
finiteCoinCoverage_anchor_coinwiseTrueFiber_sourceNonconstant_zeroBlock, finiteCoinCoverage_anchor
finiteCoinCoverage_anchor_sourceNonconstantZeroIffl

```

The same synthesis file now also records a narrow randomized-residual subledger `PNPRandomizedResidualSubrep`. This is intentionally not a claim that broad randomized residual repairs are closed. It records the finite-coin product-space facts now proved by `RandomizedResidualObstruction.lean` and

PostSwitchRandomizedResidualObstruction.lean:

productSideChannelBudget	finite coins are an ordinary p
supportwiseUnresolvedZeroMass	supportwise unresolved resid
supportwiseUnresolvedNoAdvantage	supportwise unresolved resid
positiveAdvantageResolutionWitness	positive advantage yields a p
positiveAdvantageDeterministicCoinSliceWitness	positive advantage yields a d
strictHalfDeterministicCoinSliceWitness	strict half-accuracy advantag
strictHalfDeterministicCoinSliceResidualObstructionPackage	strict half-accuracy advantag
postSwitchSupportwiseUnresolvedNoAdvantage	the exact post-switch surface
postSwitchExactSupportNoStrictHalfAdvantage	exact post-switch support bl
postSwitchPositiveAdvantageResolutionWitness	positive post-switch advanta
postSwitchPositiveAdvantageDeterministicCoinSliceWitness	positive post-switch advanta
postSwitchStrictHalfDeterministicCoinSliceWitness	strict half-accuracy post-sw
postSwitchStrictHalfDeterministicCoinSliceResidualObstructionPackage	the exact post-switch strict-l

The route anchors

```

        randomizedResidualCoverage_anchor_supportwise_unresolved_no_positive_advantage
        randomizedResidualCoverage_anchor_positive_advantage_witnesses_resolving_coin
        randomizedResidualCoverage_anchor_positive_advantage_witnesses_deterministic_coin
        randomizedResidualCoverage_anchor_strict_half_advantage_witnesses_deterministic_coin
        randomizedResidualCoverage_anchor_strict_half_advantage_coin_slice_forces_residual_obstruction
        randomizedResidualCoverage_anchor_postSwitch_exact_support_no_strict_half_advantage
        randomizedResidualCoverage_anchor_postSwitch_positive_advantage_witnesses_resolving_coin
        randomizedResidualCoverage_anchor_postSwitch_positive_advantage_witnesses_deterministic_coin
        randomizedResidualCoverage_anchor_postSwitch_strict_half_advantage_witnesses_deterministic_coin
        randomizedResidualCoverage_anchor_postSwitch_strict_half_advantage_coin_slice_forces_residual_obstruction

```

tie that subledger to the actual obstruction theorems.

This matters because the synthesis layer no longer records the randomized post-switch lane only at the positive-doubled-advantage threshold. Even after weakening to the strict half-accuracy formulation, the same finite-coin route is now Lean-visible as already forcing a deterministic ordinary residual slice, and even that slice is no longer a separate escape hatch: Lean immediately upgrades it to the full residual-side obstruction package, both generically and on the exact post-switch support surface.

This is deliberately a subledger. The theorem `finiteCoinStackUncoveredBroadRepairClasses_exact` states that the broad classes not closed by the finite-coin stack are exactly the current synthesis gaps. In particular, the finite-coin stack is recorded as coverage inside `sameSourceFiniteCountApproximation`, not as a proof of `randomizedResidual`, `residualSideInformation`, `approximateDecorrelation`, or `kpolyCompressionBridge`.

It also records the named repair classes still outside the packet:

<code>residualSideInformation</code>	post-conditioned data not determined by the source bit,
<code>randomizedResidual</code>	randomized post-conditioning residual objects,
<code>approximateDecorrelation</code>	broad quantitative replacement for exact finite-count independence,
<code>kpolyCompressionBridge</code>	the remaining manuscript-specific exact-visible / clocked realization bridge.

The residual-side-information gap now has a first narrow collision packet in `ResidualSideInformationObstruct`. It defines `SideInfoDeterminedBy` for residual data that factors through a source summary, `SideInfoCollisionOver` for two worlds with the same source summary and different residual values, `PositiveWeightSideInfoCollisionOver` for the same collision with a positive-weight supported endpoint, and `SourceOnlyPredicateCapturesSideEq` for source-only equality tests on a residual value. Lean proves

```
not_sideInfoDeterminedBy_of_same_base_ne_side,
not_sourceOnlyPredicateCapturesSideEq_left_of_same_base_ne_side,
not_sourceOnlyBooleanClassifier_of_same_base_ne_target.
```

Thus a same-source residual collision cannot be rebranded as source-only data, even for the single equality predicate naming one residual value. The concrete two-point witness `residualSideInfoToyBase/residual` pins this failure without any probabilistic or asymptotic assumptions:

```
residualSideInfoToy_collision,      residualSideInfoToy_not_determinedBy_base,
residualSideInfoToy_not_sourceOnlyPredicate_false,      residualSideInfoToy_not_sourceOnlyBoolean
```

`CruxSynthesis.lean` records this as the narrow `PNPResidualSideInformationSubrepairClass` subledger, with route anchors named `residualSideInformationCoverage_anchor_*`. The newest strengthening packages these anchors as the theorem-backed packet `V13ResidualSideInformationSubcoverage`. This remains deliberately narrow: it does not claim broad coverage of `PNPRepairClass.residualSideInformation`. What it does record is the stronger invariant-base audit boundary now available in Lean. Positive doubled advantage or strict half-accuracy advantage must expose a supported residual collision, a supported residual equality test that is not source-only, and a positive-support prediction separation witness; conversely, supported source-only residual predicates or supportwise source-only classifier behavior block both advantage formulations. Any repair that keeps varying residual data must either prove that the residual is source-determined or expose a genuine side channel to the resolving-mass theory.

The follow-up packet `ResidualSideInformationResolutionCost.lean` removes one tempting interpretation of the “source-determined” escape. If the base feature is itself invariant under the manuscript involution and the residual side information factors through that base feature, then Lean proves

```
side_eq_under_involution_of_determinedBy_invariant_base,
resolvedMass_eq_zero_of_determinedBy_invariant_base,
not_pos_doubledAdvantage_pair_of_determinedBy_invariant_base,
not_total_lt_two_mul_weightedCorrectMass_pair_of_determinedBy_invariant_base.
```

Thus such residuals are not merely source-only in a functional sense; they have zero orbit-resolving mass and cannot beat chance in either the doubled-advantage or strict half-accuracy formulations. The contrapositive boundary is also formalized:

```
not_sideInfoDeterminedBy_of_pos_doubledAdvantage_invariant_base,
not_sideInfoDeterminedBy_of_total_lt_two_mul_weightedCorrectMass_invariant_base.
```

The 2026-05-05 residual-side packet strengthens this negative boundary into a positive witness obligation. Lean now proves

```
exists_positive_same_base_ne_side_of_pos_doubledAdvantage_invariant_base,
exists_positive_same_base_ne_side_of_total_lt_two_mul_weightedCorrectMass_invariant_base.
```


The supported-collision packet then packages the same witness at the residual subledger surface:

```
positiveWeightSideInfoCollisionOverBase_of_pos_doubledAdvantage_invariant_base,
positiveWeightSideInfoCollisionOverBase_of_total_lt_two_mul_weightedCorrectMass_invariant_base,
exists_positive_not_sourceOnlyPredicateCapturesSideEq_of_pos_doubledAdvantage_invariant_base,
exists_positive_not_sourceOnlyPredicateCapturesSideEq_of_total_lt_two_mul_weightedCorrectMass_invariant_base
```

The contrapositive no-advantage form is also now formalized:

```
not_pos_doubledAdvantage_pair_of_supported_sourceOnlyPredicateCapturesSideEq_invariant_base,
not_total_lt_two_mul_weightedCorrectMass_pair_of_supported_sourceOnlyPredicateCapturesSideEq_invariant_base
```

Thus a successful residual-side-information repair over an invariant base must not merely prove that the residual is not source-derived. Any claimed advantage must expose a positive-weight point x where the base still agrees across the involution, $\text{base}(\tau x) = \text{base}(x)$, but the residual side information changes, $\text{side}(\tau x) \neq \text{side}(x)$. Equivalently, the repair must provide an actual same-source residual collision on support, and a supported residual equality predicate that cannot be source-only, or else move to a different, explicitly symmetry-breaking base. If all positive-weight residual equality tests are already source-only, Lean now blocks both positive doubled advantage and strict half-accuracy advantage. The next repair layer is classifier-level rather than value-level: the classifier may syntactically mention $(\text{base}, \text{side})$, but its realized positive-support predictions might still factor through the invariant base. This is now blocked as well. The generic supportwise lemma

```
not_total_lt_two_mul_weightedCorrectMass_of_supportwise_prediction_invariant_base,
not_pos_doubledAdvantage_of_supportwise_prediction_invariant_base
```

shows that any positive-support prediction invariant under the involution stays at chance. The residual specialization introduces `SupportwiseSourceOnlyPairClassifier` and proves

```
prediction_eq_under_involution_of_supportwiseSourceOnlyPairClassifier_invariant_base,
not_pos_doubledAdvantage_pair_of_supportwiseSourceOnlyPairClassifier_invariant_base,
not_total_lt_two_mul_weightedCorrectMass_pair_of_supportwiseSourceOnlyPairClassifier_invariant_base
```

Thus a residual-side-information repair over an invariant base cannot survive by saying that only the final classifier, not each residual equality test, is source-only on support. If the realized predictions factor through the base where the weight is positive, Lean blocks both advantage formulations. This still does not close arbitrary proof-relevant side channels; it makes the invariant-base residual repair boundary exact up to genuinely involution-sensitive supported predictions. The new 2026-05-08 packet turns that classifier-level boundary into a positive audit obligation. At the generic level, Lean now proves

```
exists_positive_prediction_ne_of_pos_doubledAdvantage,
exists_positive_prediction_ne_of_total_lt_two_mul_weightedCorrectMass,
```

so any supportwise involution-invariant classifier prediction immediately contradicts either advantage formulation. The invariant-base residual specialization packages this as

```
exists_positive_prediction_ne_of_pos_doubledAdvantage_pair_invariant_base,
exists_positive_prediction_ne_of_total_lt_two_mul_weightedCorrectMass_pair_invariant_base.
```

Hence a successful residual-side-information repair over an involution-invariant base must now do more than exhibit a supported residual value variation. It must exhibit a positive-weight point x where the final classifier actually separates the involution pair,

$$h(\text{base}(\tau x), \text{side}(\tau x)) \neq h(\text{base}(x), \text{side}(x)).$$

Residual variation without supported prediction separation is no longer enough to claim that the side channel is genuinely proof-relevant to the classifier. Update note, 2026-05-08: the active invariant-base residual-side-information boundary is no longer merely “not source-determined” or even “not source-only on support”. The exact remaining audit obligation is the prediction-separation witness above. The broad `PNPRepairClass.residualSideInformation` entry in `CruxSynthesis.lean` is still not stop-grade covered, but any repair that stays over an involution-invariant base must now surface explicit supported classifier-level separation.

Update note, 2026-05-14: the synthesis layer now packages these invariant-base residual obligations into one route-facing theorem rather than leaving them as separate repair chores. Lean proves

```
residualSideInformationCoverage_anchor_positive_advantage_forces_supported_obstruction_package
residualSideInformationCoverage_anchor_strict_half_advantage_forces_supported_obstruction_package
```

So any positive doubled advantage, or any strict half-accuracy advantage, from `(base, side)` over an involution-invariant base must now simultaneously pay the full residual-side-information bill: the residual is not source-determined, the realized positive-support classifier is not source-only, there is a supported same-base residual variation witness, there is a supported residual equality test that is not source-only, and there is supported prediction separation. These are no longer five independent places where a manuscript repair can hope to wiggle through one at a time; in Lean they now arrive as one coupled obstruction package.

Update note, 2026-05-14 (later): the residual-side-information lane now states the quantitative bridge that the route summary had already been appealing to. Lean proves

```
residualSideInformationCoverage_anchor_positive_advantage_forces_positive_resolvedMass,
residualSideInformationCoverage_anchor_strict_half_advantage_forces_positive_resolvedMass,
residualSideInformationCoverage_anchor_positive_resolvedMass_witnesses_same_base_residual_collision
```

So an invariant-base residual repair no longer gets to claim only that some supported residual witness exists. Any positive doubled advantage or strict half-accuracy advantage from `(base, side)` must already pay strictly positive orbit-resolving mass for `side`, and that quantitative obligation in turn forces a positive-weight same-base residual collision. This is the direct synthesis-layer reduction from residual-side advantage claims to the generic side-channel resolution-demand obstruction.

Update note, 2026-05-14 (later still): the same quantitative condition is now formalized as a classifier-free residual obstruction package. In `ResidualSideInformationResolutionCost.lean`, Lean proves

```
pos_resolvedMass_obstruction_package_invariant_base.
```

So once an invariant-base residual side channel carries positive orbit-resolving mass, the manuscript no longer gets to defer the residual audit to later classifier choices. Before any final Boolean predictor is discussed, positive resolved mass already forces all of the following: the residual is not source-determined, there is a positive-weight same-base residual collision, and there is a supported residual equality predicate that is not source-only. The synthesis file now re-exports this as

```
residualSideInformationCoverage_anchor_positive_resolvedMass_forces_supported_obstruction_package
```

This further tightens the route: residual-side-information advantage first has to pay positive resolved mass, and positive resolved mass already pays a pure residual obstruction package even before prediction-separation obligations are considered.

Update note, 2026-05-14 (exact post-switch witness): this pure residual packet is now instantiated on the manuscript’s exact visible post-switch surface itself. In `PostSwitchResidualWitness.lean`, `Lean` proves

```
pos_resolvedMass_activeOrbit,
exactPostSwitch_pos_resolvedMass_forces_pure_residual_obstruction_package,
exactPostSwitch_activeOrbit_realizes_pure_residual_obstruction_package.
```

So the active two-point exact post-switch orbit already carries positive ordinary resolved mass for the retained side bit, and that quantitative fact alone already forces the pure residual obstruction package on the manuscript’s own local visible surface. Before any classifier-level prediction-separation claim is used, `Lean` now extracts from this exact post-switch witness that the retained side channel is not source-determined over the invariant projection, that there is a positive-weight same-base residual collision, and that there is a supported residual equality predicate that is not source-only.

Update note, 2026-05-14 (synthesis ledger integration): this exact post-switch active-orbit packet is now re-exported through `CruxSynthesis.lean` itself. The theorem

```
residualSideInformationCoverage_anchor_exactPostSwitch_activeOrbit_pure_residual_package
```

packages the same concrete active-orbit witness at the synthesis layer, and the residual-side subrepair ledger now records it as the explicit covered case

```
exactPostSwitchConcretePureResidualWitness.
```

So the theorem-backed route summary no longer stops at the abstract statement that positive resolved mass forces a pure residual obstruction package. It now also includes the manuscript’s own exact post-switch visible surface as a concrete theorem-backed instance of that package.

Update note, 2026-05-14 (exact post-switch prediction witness): the same exact post-switch active orbit is now also recorded at the synthesis layer as a concrete classifier witness. The local theorem

```
exactPostSwitch_activeOrbit_realizes_residual_prediction_witness
```

is re-exported in `CruxSynthesis.lean` as

```
residualSideInformationCoverage_anchor_exactPostSwitch_activeOrbit_prediction_witness.
```

So the theorem-backed route summary now includes not only the pure residual package on the manuscript’s exact visible surface, but also the concrete classifier-level fact that this active orbit preserves the invariant projection, has involution-symmetric weight, flips the target on positive support, achieves positive doubled advantage, and separates a positive-weight involution pair at the final classifier output.

Update note, 2026-05-15 (active-orbit collapse to the exact fork blocker): the same concrete witness no longer sits only at the level of “there exists a positive residual prediction route.” In `PostSwitchResidualWitness.lean` the new theorem

```
exactPostSwitch_activeOrbit_forces_fork_obstruction
```

packages three facts at once:

1. the active orbit beats half accuracy in the strict form `weightedTotalMass < 2*weightedCorrectMass`;
2. its support already carries positive mass on the nonzero-column slice;
3. its weight function is therefore not exact-input invariant.

So the manuscript’s own concrete residual witness no longer leaves a separate “residual side information” escape route on the exact post-switch surface. Lean now sends that witness directly into the already formalized fork obstruction surface: any attempt to keep the same active-orbit witness while also insisting on exact-input-invariant support is immediately contradictory, and this remains true on the strict half-accuracy formulation, not only on the positive doubled-advantage one.

Update note, 2026-05-17 (synthesis-layer residual/fork collapse): this exact surface collapse is now recorded inside the residual-side synthesis ledger itself, rather than only in the local witness file. In `CruxSynthesis.lean`, Lean proves

`residualSideInformationCoverage_anchor_exactPostSwitch_prediction_witness_forces_fork_obstruction`.

The residual-side subrepair ledger also now records the new covered case

`exactPostSwitchPredictionWitnessForcesForkObstruction`.

So on the manuscript’s concrete exact visible post-switch surface, the classifier-level residual prediction witness is no longer tracked as a standalone repair lane. At the synthesis interface it is explicitly coupled to the same fork-obstruction package: strict half-accuracy success, positive nonzero-column mass, and failure of exact-input-invariant support arrive together on the same active two-point orbit.

Update note, 2026-05-17 (generic exact post-switch b -route collapse): the same fork package is now also stated at the full exact post-switch $((z, a_i), b)$ route, not only on the concrete active orbit. Lean now re-exports

`postSwitchForkCoverage_anchor_strict_half_advantage_fork_obstruction_package`.

So for any exact post-switch classifier on $(\text{invariantProjection}, b)$, strict half-accuracy advantage already forces three simultaneous obligations: there is a positive-weight input where the retained b side channel changes under the involution, there is a positive-weight nonzero a_i column, and the support cannot be exact-input invariant. This removes the last gap between the local fork-obstruction file and the route-synthesis layer for the strict half-accuracy exact b -route.

Update note, 2026-05-17 (finite-coin randomized residual is not an independent strict-half route): the same synthesis layer now also re-exports

`randomizedResidualCoverage_anchor_strict_half_advantage_coin_slice_forces_residual_obstruction`.

and

`randomizedResidualCoverage_anchor_postSwitch_strict_half_advantage_coin_slice_forces_residual_o`.

So on both the generic finite-coin product-space surface and the exact post-switch invariant-projection surface, strict half-accuracy randomized residual advantage no longer merely forces a deterministic positive-weight coin slice. That slice already carries the ordinary residual-side obstruction package: failure of determination by the invariant base, positive collision over that base, an explicit positive-weight resolving point, and failure of the corresponding source-only side-equality capture. Thus the

finite-coin randomized residual repair is no longer an independent synthesis lane at the strict-half threshold; Lean collapses it directly into the residual-side route.

The broad `approximateDecorrelation` class remains outside the stop-grade packet. However, `ApproximateDecorrelationObstruction.lean` now blocks the immediate same-source coupled, anti-coupled, and arbitrary deterministic distinguishing-fiber finite-count subcases: any claimed tolerance below $|C \cap E| |C \cap \neg E|$ contradicts the proved cross-product defect identity, and any claimed tolerance certificate for such an informative fiber therefore entails the explicit obligation $|C \cap E| |C \cap \neg E| \leq \tau$. With lower bounds $L_1 \leq |C \cap E|$ and $L_0 \leq |C \cap \neg E|$, Lean further reduces this to $L_1 L_0 \leq \tau$, and directly blocks any claimed tolerance $\tau < L_1 L_0$. The same module now also exposes the whole-output predicate `CountIndependentWithinToleranceOutput`; for every nonconstant deterministic recoding $r : \{0, 1\} \rightarrow \alpha$, approximate independence of all output fibers still forces the same product/tolerance obligation because the $r(1)$ -fiber distinguishes the source bit. For finite-coin randomized recodings $r : \{0, 1\} \rightarrow \text{Coin} \rightarrow \alpha$, the same product/tolerance obligation first holds whenever some output fiber uniformly distinguishes the source side across all coins, and the product-space obligation factors to $(L_1 |\text{Coin}|)(L_0 |\text{Coin}|) \leq \tau$ from source-side lower bounds L_1, L_0 . The newer mixed-coin identities remove that uniformity assumption: for every output fiber y , the forward and backward defects are exactly

$$|C \cap E| |C \cap \neg E| |\text{Coin}| (T_y - F_y) \quad \text{and} \quad |C \cap E| |C \cap \neg E| |\text{Coin}| (F_y - T_y),$$

where $T_y = \#\{c : r(1, c) = y\}$ and $F_y = \#\{c : r(0, c) = y\}$. Therefore a finite-coin same-source randomized repair must either balance all output-fiber coin counts or pay the scaled imbalance in the tolerance; the lower-bound variants let this be checked from source-side estimates L_1, L_0 without knowing the exact conditioned counts. This is now the arbitrary finite-coin same-source recoding subcase of the randomized residual repair space. The balanced subcase is now also characterized: balanced fibers give exact zero-tolerance independence for the marginal output, but if the random coin is retained and any coin value separates $r(1, c)$ from $r(0, c)$, the output $(r(E, c), c)$ has a fiber with defect $|C \cap E| |C \cap \neg E| |\text{Coin}|$. Hence a balanced same-source finite-coin repair must either be a genuine erasure at the output-fiber level, or account for retained side information under the same finite-count audit. The retained-side theorem generalizes the retained-coin result: any side channel whose fiber isolates a true-side output from false-side matches has a forward defect, and any side channel whose fiber isolates a false-side output from true-side matches has a backward defect. In both cases the defect is $|C \cap E| |C \cap \neg E| |\text{Coin}|$ times the size of the selected side/output fiber. The postprocessed-side theorem removes the remaining syntactic escape: even after applying an arbitrary observation $p : \alpha \times \text{Side} \rightarrow \beta$, any observed true-side fiber isolated from all false-side observations has the same forward defect, and any observed false-side fiber isolated from all true-side observations has the same backward defect, scaled by the selected observed-fiber count. Thus a repair cannot keep source-bearing side information and evade the finite-count audit merely by hashing or compressing the retained pair. The output-predicate count theorem makes “genuine erasure” precise: balanced fibers force every decidable output property to have equal true-side and false-side coin counts. The previous output-only decoder theorem is the perfect-recovery special case; the count theorem also rules out any strict predicate-level advantage in either direction. The Boolean decoder corollary strengthens this into an exact half-accuracy law: true-side correct decodes plus false-side correct decodes equal $|\text{Coin}|$, so no balanced marginal decoder can beat random-guess aggregate accuracy over the two source sides. For the uniform true-side/false-side finite-coin fibers, the zero-tolerance version is now phrased over the original source counts: a nonempty finite coin space lifts positive source-side true and false conditioned fibers to the product space, so the corresponding zero-tolerance output-independence claim is impossible without asking the manuscript author to restate nondegeneracy on $\Omega \times \text{Coin}$.

Update note, 2026-05-14 (postprocessed-side skew threshold counterexample): the earlier three-coin postprocessed-side skew audit is now upgraded from a mere support-overlap counterexample to a quantitative approximate-decorrelation blocker. In `ApproximateDecorrelationObstruction.lean`, Lean proves

```

    postprocessedSidePointwiseCollisionSkew_true_count_true = 2,
    postprocessedSidePointwiseCollisionSkew_false_count_true = 1,
postprocessedSidePointwiseCollisionSkew_not_countIndependentWithinToleranceOutput_of_lt_three.

```

So the same skew witness still has two-sided pointwise collisions, and it still admits the earlier one-sided preserving true-to-false witness map, but every approximate-independence certificate with tolerance $\tau < 3$ is formally refuted. The package theorem

```

postprocessedSidePointwiseCollisionSkew_collision_and_oneSidedMap_counterexample_of_lt_three

```

records all three facts together. This sharpens the route-level obstruction: neither pointwise support overlap nor one-sided witness choice is enough to repair postprocessed retained-side dependence below threshold; the manuscript still has to solve the exact multiplicity-level coin-matching obligation. The same file now proves the lower bound is *sharp*. Lean computes the two relevant fiber defects exactly:

```

    postprocessedSidePointwiseCollisionSkew_true_forward_gap_eq_three,
    postprocessedSidePointwiseCollisionSkew_false_backward_gap_eq_three,

```

with the opposite oriented defects equal to 0. From those exact gap values, Lean proves

```

postprocessedSidePointwiseCollisionSkew_countIndependentWithinToleranceOutput_iff_three_le.

```

So this three-coin skew witness is not merely a “fails below some threshold” example: its approximate-decorrelation threshold is formally characterized as exactly $\tau = 3$. The manuscript therefore cannot salvage the route by claiming that support-overlap plus a one-sided preserving map becomes adequate at some smaller hidden tolerance; the obstruction already isolates the precise minimal tolerance needed on this witness.

The theorem

```

not_stopGrade_currentPNPLocalCruxPacket

```

proves that the current packet is not stop-grade, by the explicit uncovered `residualSideInformation` class. Companion theorems separately mark `residualSideInformation`, `randomizedResidual`, `approximateDecorrelation`, and `kpolyCompressionBridge` as uncovered. The finite-coin-specific companion theorems

```

    finiteCoinStack_does_not_cover_residualSideInformation,  finiteCoinStack_does_not_cover_randomizedResidual,
    finiteCoinStack_does_not_cover_approximateDecorrelation,  finiteCoinStack_does_not_cover_kpolyCompressionBridge

```

make the same scoping constraint explicit for the new finite-coin work. By contrast,

```

sameSourceFiniteCountApproximation_covered_currentPNPLocalCruxPacket

```

records the finite-count approximation subcase as covered, and

```

clockedFiniteUniformKernel_covered_currentPNPLocalCruxPacket

```

records that the abstract finite-counting kernel itself is already on the covered side of the ledger.

The current small-image fork is now integrated into the same synthesis layer. `CruxSynthesis.lean` imports `ClockedKpolyActualGapClosure.lean` and records the narrow `PNPKpolySubrepairClass` entries for this fork: exact ZAB ERM payload closure, the bit-vector specialization, equality-transfer to the bit-vector ERM family, the bare exact-visible interface obstruction, the actual-support quotient boundaries, the plug-in lookup/majority endpoint obstructions, and the bounded-sample false-support, default-tie, fallback-side-channel, and structured fallback-family sparse-override boundaries described below. The named theorem

`currentPNPStrongestHonestStatus`

assembles the present honest claim: the theorem-backed `v13` entries are covered, including the residual-atom evidence-normalization boundary and the trace-fiber factorization boundary for observer capture, and the atomic evidence-budget multiplicity boundary; the abstract clocked finite-uniform kernel is covered, the concrete bit-vector exact ZAB ERM family has the clocked payload at budget $r + 2k + 1$, and the bare exact-visible interface cannot imply such payloads for every family below the full visible surface. The same theorem also records that the broad `kpolyCompressionBridge` remains uncovered and that the current packet is not stop-grade. The same synthesis packet now records the concrete SAT-readout repair at the short-global-decoder entry: for Boolean readouts on the one-variable CNF $x \vee \neg x$, arbitrary-output SAT-search correctness is equivalent to single-message readout constancy, while the nonconstant readout x itself has selected-output correctness for both messages and no arbitrary-output message. A separate generic anchor records the manuscript-level version: once the CNF extraction is complete and supported satisfiability is present, supported arbitrary-output SAT-search correctness is equivalent to the semantic single-message promise.

The ledger also records the next marginal target rather than leaving the route choice implicit. It defines `currentPNPNextMarginalTarget` as `residualSideInformation` and proves that this target is still uncovered and still belongs to the finite-coin stack's broad uncovered list. The reason is conservative: postprocessed-side matching and finite-coin randomized residuals now have exact subledgers, while the clocked finite-uniform kernel is already covered. The new residual collision subledger blocks the first source-only recoding escape hatch, and the residual resolution-cost subledger shows that source-determined residuals over an invariant base have zero resolving mass and no advantage. But the broad bridge remains open: a useful retained residual must still either be shown genuinely non-source-derived with positive resolving support, or must explicitly change the invariant-base hypotheses being used by the side-channel budget theorem. The ledger lemma

`not_stopGrade_of_not_covers_currentPNPNextMarginalTarget`

turns that prioritization into a formal audit rule: any packet that still misses the selected next target cannot be called stop-grade. This is deliberately weaker than claiming all residual repairs are broken; it records that the current local crux families, even when useful, do not by themselves close the residual side-information bridge. The same file now also isolates the stronger promoted packet obtained by adding the already formalized theorem-backed residual-side-information blocker as a broad repair-class claim. It defines `currentPNPResidualPromotedPacket`, proves that this packet extends `currentPNPLocalCruxPacket`, proves that it covers `residualSideInformation`, and records that it still leaves exactly `randomizedResidual`, `approximateDecorrelation`, and `kpolyCompressionBridge` uncovered. Accordingly `currentPNPResidualPromotedNextMarginalTarget` is `randomizedResidual`, and `currentPNPResidualPromotedStatus` shows that even this promoted packet is not stop-grade. This does not upgrade the baseline current packet; it records that once the

existing residual obstruction layer is treated as a broad blocker, the next remaining broad escape route is the randomized residual lane rather than residual side information itself. The next broad packet step is now formalized as well. The same module defines `RandomizedResidualCoverage` as the theorem-backed statement that finite-coin randomized residuals stay under the ordinary resolving-mass budget, supportwise-unresolved randomized residuals have zero joint resolving mass and no positive doubled advantage, any surviving positive advantage already yields a deterministic positive-weight coin slice with positive ordinary resolving mass, and any strict half-accuracy advantage already collapses to an ordinary residual-side-information obstruction package on such a deterministic slice. It then defines `currentPNPRandomizedResidualPromotedPacket`, proves that this packet extends `currentPNPResidualPromotedPacket`, and proves that it covers both `residualSideInformation` and `randomizedResidual`. What remains open at that point is exactly `approximateDecorrelation` together with the manuscript-specific `kpolyCompressionBridge`; accordingly `currentPNPRandomizedResidualPromotedNextMarginalTarget` is `approximateDecorrelation`, and `currentPNPRandomizedResidualPromotedStatus` shows that even this stronger packet is still not stop-grade. This is again a promoted extension packet, not a change to the baseline current packet. The next broad packet step is now formalized too. The same module defines `ApproximateDecorrelationCoverage` as the theorem-backed statement that below the natural source/coin threshold, approximate independence of a postprocessed retained-side observation is already equivalent to an exact preserving true/false coin equivalence, while the concrete three-coin skew witness still has two-sided pointwise collisions and a one-sided preserving map but fails approximate independence for every tolerance $\tau < 3$. It then defines `currentPNPApproximateDecorrelationPromotedPacket`, proves that this packet extends `currentPNPRandomizedResidualPromotedPacket` and proves that it covers `residualSideInformation`, `randomizedResidual`, and `approximateDecorrelation`. What remains open at that point is exactly the manuscript-specific `kpolyCompressionBridge`; accordingly `currentPNPApproximateDecorrelationPromotedNextMarginalTarget` is `kpolyCompressionBridge`, and `currentPNPApproximateDecorrelationPromotedStatus` shows that even this stronger packet is still not stop-grade. This is again a promoted extension packet, not a change to the baseline current packet. The final broad packet step is now formalized as well. The same module defines `KpolyCompressionBridgeCoverage` as the theorem-backed statement that the manuscript's remaining product-bound-only and fielded-switching-only exact-visible bridge schemas, and their bundled clocked finite-learning payload variants, are exactly finite-image obligations at the same bit budget; that the nil and Boolean-square one-step clocked bridge schemas are refuted below the exact-surface cardinality threshold; that truly surjective exact-visible families refute the product-bound and fielded-switching clocked bridge schemas below the predictor-cardinality threshold; and that the bundled clocked payload is equivalent both to finite predictor cover and to the existing exact-visible compression target. It then defines `currentPNPKpolyCompressionBridgePromotedPacket`, proves that this packet extends `currentPNPApproximateDecorrelationPromotedPacket`, and proves that it covers `residualSideInformation`, `randomizedResidual`, `approximateDecorrelation`, and `kpolyCompressionBridge`. The theorems `stopGrade_currentPNPKpolyCompressionBridgePromotedPacket` and `currentPNPKpolyCompressionBridgePromotedStatus` therefore record that the current broad PNP synthesis ledger is now stop-grade. This does not claim a full $P \neq NP$ proof; it says only that, at the present synthesis layer, the currently named broad repair classes are all closed by theorem-backed route blockers. The same synthesis layer now also packages the final manuscript-facing exact ERM route as `CanonicalZABERMRouteCoverage`: once a family supplies `CanonicalZABERMRecoveryData`, Lean immediately places it on the exact-visible compression / finite-image / clocked-payload surface already audited by this stop-grade packet. Thus the remaining gap is no longer a missing small-image lemma for that named route; it is the universality bridge showing that the manuscript's actual switched family yields `CanonicalZABERMRecoveryData` or an equivalent code-realization interface.

The same module now also states the exact completion criterion for this ledger. For any packet extending the current local packet,

`stopGrade_iff_covers_currentPNP_gaps_of_extends_current`

proves that the packet is stop-grade if and only if it covers the four named gap classes above. Lean also defines the bookkeeping object `currentPNPGapCompletedPacket`, obtained by adding exactly the current gap list to the current local packet, and proves

`stopGrade_currentPNPGapCompletedPacket.`

It also proves

`currentPNPGapCompletedPacket_covers_currentPNPNextMarginalTarget,`

pinning the selected *Kpoly* target as one of the explicit obligations of any ledger-complete packet. This theorem is only a ledger closure: it does not supply the missing mathematics for residual side information, randomized residuals, approximate decorrelation, or the remaining manuscript-specific *Kpoly* realization bridge. Instead it gives a precise target for any future route-stopping packet. Conversely,

`not_stopGrade_of_not_covers_currentPNP_gaps_of_extends_current`

records that an extension of the current packet is still not stop-grade if it fails to cover the current gap list.

This is intentionally not another local counterexample. Its role is to prevent overclaiming: the current Lean work blocks several same-route repair families, but a fair request for the author to stop would require either covering these remaining repair classes or proving that the manuscript route cannot use them.

28 Next Attack Surface

The next natural formal target is the paper’s implicit compression claim. The first and third interface steps are now isolated in Lean: `VisiblePostSwitchSurface.lean` defines the exact manuscript-visible surface $u = (z, a, b)$ together with its invariant and fork projections, and `ExactSwitchedFamily.lean` packages the exact bit-budget theorem needed to reuse the same-route ERM bound. The remaining mathematical burden is now sharper:

After switching, the witness-bit predictor is not merely local; it is simple enough to lie in a small explicit class.

To make that rigorous, one would want a theorem of the shape:

1. define the exact visible post-switch data carried by the masked Unique-SAT ensemble;
2. define the induced witness-bit function on that data;
3. prove that this function belongs to a sharply bounded class (for example, bounded-depth circuits of size $\text{polylog}(m)$, or a code family of description length $\text{polylog}(m)$);
4. verify that this bound is strong enough to feed the later union-bound and compression steps.

The new module `ClockedKpolyBridge.lean` separates the last item from the still-missing representation theorem. It defines a clocked s -bit decoder family and proves the finite counting kernel:

`exactRecoveryRate_ge_one_sub_uniformMissRatio_pow.`

Thus, once a target is realized by a clocked s -bit family, the existing finite-uniform ERM bound applies with failure term

$$2^s \cdot \rho_X^m.$$

On the Boolean domain it also proves the sample-margin form

`exactRecoveryRate_ge_one_sub_bool_margin_pow,`

which gives recovery rate at least $1 - (1/2)^k$ when the sample length is at least $s + k$. This is not yet the manuscript's *Kpoly* bridge: the open obligation is still to show that the actual switched witness-bit decoder is realized by such a clocked bit-code family with the claimed bit and clock budgets.

The follow-up module `ClockedKpolyTargetInterface.lean` connects this kernel back to the exact visible post-switch surface. Its main theorem says that any already-proved exact visible compression target

`ExactVisibleCompressionTarget G s`

can be repackaged as a clocked s -bit family and therefore yields

`exactRecoveryRate_ge_one_sub_uniformMissRatio_pow_of_exactVisibleCompressionTarget.`

The bridge is now exact in both directions. The module `ClockedKpolyBridgeEquivalence.lean` proves

`exists_clockedExactVisibleRealization_iff_exactVisibleCompressionTarget.`

Thus a clocked realization theorem for the exact post-switch surface is not a separate way around the compression target. It is precisely the same s -bit `ExactVisibleCompressionTarget`, with the clock parameter carried as external bookkeeping. The companion clock-invariance theorem says that changing the external clock bound does not alter which families are realizable by s program bits. Therefore the remaining *Kpoly* bridge has to prove actual small-class structure for the switched predictor family; it cannot be discharged by a new clock wrapper around an otherwise uncompressed truth-table family.

At the synthesis-ledger level, this moves the abstract `clockedFiniteUniformKernel` class to the covered side and leaves only `kpolyCompressionBridge` as the manuscript-specific realization theorem still missing. So the remaining *Kpoly*-side burden is now cleaner:

1. prove a manuscript-faithful exact visible compression target for the actual switched family;
2. then feed that target through the clocked interface to obtain the finite recovery inequality.

What is still not formalized is the first step, namely the actual representation theorem for the manuscript decoder.

The new module `ExactVisibleImageBudget.lean` makes that first step even more concrete. For an indexed predictor family G , Lean proves

$$G.\text{HasBitBudget } s \iff \exists S, (\forall i, G_i \in S) \wedge |S| \leq 2^s.$$

The exact visible and clocked interfaces inherit the same characterization:

`exists_clockedExactVisibleRealization_iff_finitePredictorCover.`

Thus the broad *Kpoly* bridge is now reduced to a finite predictor-image theorem for the actual switched family. This is useful route-level progress, not a closure claim: it does not prove the image is small, it states the exact finite-image fact that any successful small-program repair must supply. The same file also removes an abstraction escape hatch: a finite predictor cover is equivalent to a finite set of representative *indices* whose actual selected predictors cover the whole family. In Lean this is

`finitePredictorCover_iff_finiteIndexRepresentativeCover.`

It is also equivalent to a finite quotient-code presentation:

`finitePredictorCover_iff_finitePredictorQuotient.`

The clocked exact-visible interface now has matching negative forms:

`not_exists_clockedExactVisibleRealization_iff_not_finiteIndexRepresentativeCover`

and

`not_exists_clockedExactVisibleRealization_iff_not_finitePredictorQuotient.`

Thus a proposed finite-image proof can be read operationally either as a finite list of actual switched predictors or as a finite code/decoder quotient, not merely as an arbitrary external set of Boolean functions. The file now also proves the basic closure properties needed to use that formulation compositionally. Finite predictor-image covers, finite representative-index covers, and finite quotient-code presentations are monotone in the allowed budget, pull back across an explicit factorization $G(x) = H(\text{view}(x))$, and push back to the summary family when the view map has a section. These are recorded in Lean as the `finitePredictorCover_of_factorsThrough`, `finitePredictorCover_of_factorsThrough_of_rightInverse`, `finiteIndexRepresentativeCover_of_factor`, `finiteIndexRepresentativeCover_of_factorsThrough_of_rightInverse`, `finitePredictorQuotient_of_fa`, and `finitePredictorQuotient_of_factorsThrough_of_rightInverse` lemmas. This is still a preservation theorem, not a small-image theorem: it keeps any proposed exact-visible compression argument honest about which finite image bound is being transported. Combining that transport with reduced-surface surjectivity gives a sharper pullback obstruction. If an exact family factors through a finite view, the view map has a section, and the reduced family is still surjective onto all Boolean classifiers on that view, then every finite predictor-image cover of the exact family has size at least the full Boolean classifier-space cardinality of the view. The raw (a, b) specialization is now explicit: if G factors through `ABVisibleSurface` k and the reduced family is still fully expressive, then no finite predictor cover of G can have size below

$$2^{|\text{ABVisibleSurface } k|} = 2^{2^k}.$$

The same statement is proved for finite representative-index covers and finite quotient-code presentations. This blocks a concrete finite-image escape hatch: a route cannot claim a small exact-visible image after quotienting to (a, b) unless it proves that the reduced predictor family is far smaller than the full Boolean rule space. The same module now proves the direct obstruction for this finite-image form: if G is still surjective onto all Boolean rules on a finite input surface, then every

finite predictor cover, finite representative-index cover, and finite quotient-code presentation has size at least

$$2^{|\text{surface}|}.$$

For the exact visible surface this yields

`not_exists_clockedExactVisibleRealization_of_surjective_predict_of_lt_predictorCard.`

The newest strengthening removes a weaker escape hatch. Full surjectivity is not required for the counting lower bound: if the switched family already realizes an injectively indexed finite probe family P of classifiers, then every finite predictor cover, representative-index cover, and quotient-code presentation has size at least $|P|$. Lean records this as

`finitePredictorCover_card_le_of_injective_realization,`

together with the representative and quotient variants. The exact visible and clocked negative forms are

`not_exactVisibleCompressionTarget_of_injective_realization_of_lt_card`

and

`not_exists_clockedExactVisibleRealization_of_injective_realization_of_lt_card.`

Thus the route cannot replace a bit-budget proof by a finite-image cover unless it also proves the actual predictor image is a strict subclass not merely of the full Boolean classifier space, but of every large finite probe family it already realizes. The factor-map transport theorem now has the same partial-image form. If an exact family factors through a reduced view with a section, and the reduced family realizes an injectively indexed finite probe family P , then every finite predictor cover, representative-index cover, and quotient-code presentation of the original family still has size at least $|P|$. The generic Lean anchors are

`finitePredictorCover_card_le_of_factorsThrough_of_rightInverse_of_injective_realization`

and the matching representative/quotient variants, with negative forms below $|P|$. Thus a reduced-view repair cannot evade the finite-image audit by avoiding full surjectivity; any large finite class already realized on the reduced view is enough to force the same lower bound after pullback. The concrete (a, b) -reduced route now has the same partial-image specialization. If the exact visible family factors through `abVisibleData` and the reduced (a, b) family realizes an injectively indexed finite probe family P , then the pullback exact family inherits the lower bounds for finite predictor covers, representative-index covers, and quotient-code presentations. The main Lean anchors are

`abVisible_finitePredictorCover_card_le_of_factorsThrough_of_injective_realization,`

`not_exactVisibleCompressionTarget_of_factorsThrough_ab_and_injective_realization_of_lt_card,`

and

`not_exists_clockedExactVisibleRealization_of_factorsThrough_ab_and_injective_realization_of_lt.`

So the (a, b) reduction cannot repair the *Kpoly* bridge by weakening “surjective onto all reduced rules” to an unspecified finite image: every finite probe family already realized on the raw (a, b) surface still gives a hard cardinal lower bound after exact-visible pullback.

The new module `ExactVisibleCompressionObstruction.lean` records the fallback counting barrier on that exact object: if the switched family on $u = (z, a, b)$ is still rich enough to realize *all* Boolean rules on the exact visible surface, then no s -bit code can cover it when $s < |\text{surface}|$. So any same-route rescue must now prove a genuine strict subclass theorem for the *actual* switched predictors.

The new module `ClockedKpolyCompressionObstruction.lean` states the same point in the manuscript’s remaining form. It does not only say that a small exact visible compression target is impossible under full expressivity. It also says that, under the same surjectivity hypothesis, there cannot even exist a clocked s -bit realization family for the switched predictors when

$$s < |\text{ExactVisiblePostSwitchSurface } Z \ k|.$$

So the still-open *Kpoly* bridge really is a strict-subclass obligation on the actual switched family, not an implementation detail about packaging an already-small family into a clocked decoder interface.

`ProductSuccessKpolyBridgeObstruction.lean` now rules out a broader route-level shortcut to that bridge. The `v13` product-success and fixed-field switching facts are event-history statements; by themselves they do not impose any finite-image bound on an arbitrary exact-visible predictor family. Lean formalizes this with the full family

$$\text{fullExactVisibleRuleFamily } Z \ k,$$

whose index is the Boolean rule itself and whose prediction map is surjective onto all exact-visible rules. Therefore any bridge that tried to derive `ExactVisibleCompressionTarget` only from the finite tower product bound is false below the surface-cardinality threshold:

$$\text{not_productBoundOnlyExactVisibleCompressionBridge_nil_of_lt_surfaceCard.}$$

The same is true if the bridge uses only a fixed-field `v13` certificate:

$$\text{not_fieldedSwitchingOnlyExactVisibleCompressionBridge_nil_of_lt_surfaceCard.}$$

The obstruction is not an artifact of the empty list. The Boolean-square one-step model with the one-cell field has a genuine fielded certificate and therefore the product bound, but Lean still proves both product-only and fielded-only bridges false:

$$\text{not_productBoundOnlyExactVisibleCompressionBridge_boolPair_one_step_of_lt_surfaceCard,}$$

$$\text{not_fieldedSwitchingOnlyExactVisibleCompressionBridge_boolPair_one_step_of_lt_surfaceCard.}$$

Thus the remaining *Kpoly* bridge cannot be a consequence of product success alone. It must include a semantic theorem tying the manuscript’s switched event structure to a concrete finite predictor-image cover, representative index cover, or quotient-code presentation of the exact-visible family. The module now makes this semantic boundary exact. It defines

$$\text{V13FieldedProductBoundFrom}$$

for the fielded tower product inequality and proves that, for any fixed exact-visible family G , the implication

$$\text{V13FieldedProductBoundFrom} \rightarrow \text{ExactVisibleCompressionTarget } G \ s$$

is equivalent to

$$G.\text{FinitePredictorCover } (2^s)$$

whenever the product premise is true:

`productBound_true_exactVisibleCompressionBridge_iff_finitePredictorCover.`

The fielded-certificate version is identical:

`fieldedSwitching_true_exactVisibleCompressionBridge_iff_finitePredictorCover.`

Consequently any global product-only or fielded-only bridge immediately yields not only a finite predictor cover, but also a finite representative-index cover and a finite quotient-code presentation for the selected exact-visible family:

`finiteIndexRepresentativeCover_of_productBoundOnlyExactVisibleCompressionBridge,`
`finitePredictorQuotient_of_fieldedSwitchingOnlyExactVisibleCompressionBridge.`

This is the current minimal repair target. A manuscript-side *Kpoly* bridge must prove one of those finite-image objects for the actual switched predictors; product arithmetic and fielded half-success certificates do not reduce that semantic burden. The latest strengthening closes the parallel presentation repair in which the bridge target is not `ExactVisibleCompressionTarget` but the bundled `ClockedKpolyFiniteLearningPayload`. Lean defines the corresponding product-bound-only and fielded-switching-only clocked-payload bridge schemas and proves that they are still exactly demands for finite predictor-image covers:

`productBoundOnlyClockedKpolyFiniteLearningBridge_iff_forall_semanticFiniteImageBridge,`
`fieldedSwitchingOnlyClockedKpolyFiniteLearningBridge_iff_forall_semanticFiniteImageBridge.`

Moreover any such clocked-payload bridge immediately implies the older exact-visible compression bridge, so the same full-visible counterexamples apply. The empty and Boolean-square one-step versions are recorded as

`not_productBoundOnlyClockedKpolyFiniteLearningBridge_nil_of_lt_surfaceCard,`
`not_fieldedSwitchingOnlyClockedKpolyFiniteLearningBridge_boolPair_one_step_of_lt_surfaceCard.`

For already-surjective exact-visible families, the direct refuter uses the Boolean predictor-space threshold:

`not_productBoundOnlyClockedKpolyFiniteLearningBridge_of_true_productBound_of_surjective_predictorCover.`

with the analogous fielded-switching theorem. Thus moving from compression language to payload language does not bypass the finite-image audit.

`ActualSwitchedLocalSupportObstruction.lean` now tests the next natural repair idea: perhaps the manuscript's actual switched-local interface only needs to be small on the block outputs that are really observed. Lean separates that observed-support statement from the full exact-visible *Kpoly* target. The strengthened version constructs an arbitrary finite-support actual-local interface

`supportFullRuleActualSwitchedLocalInterface visibleOf`

whose observed block-output family has truth-table budget bounded by $|\text{Block}|$, hence a predictor cover of size at most $2^{|\text{Block}|}$:

`supportFullRule_outputFamily_finitePredictorCover_card.`

But the selected local-rule family is still exactly the full Boolean family on the exact visible post-switch surface. Therefore, for every

$$s < |\text{ExactVisiblePostSwitchSurface } Z \ k|,$$

Lean proves both

`supportFullRule_not_finitePredictorCover_of_lt_surfaceCard`

and

`supportFullRule_not_clockedKpolyFiniteLearningPayload_of_lt_surfaceCard.`

The previous one-block theorem is retained as the minimal instance. The next possible repair is to add a uniform section from every exact-visible input back to an actual block. Lean proves that this is exactly the missing bridge at this abstraction level:

`predictorFamily_factorsThrough_outputFamily_of_uniformVisibleSection,`

`predictorFamily_finitePredictorCover_of_outputFamily_finitePredictorCover,`

and

`clockedKpolyFiniteLearningPayload_of_outputFamily_hasBitBudget.`

So observed-output compression transfers to the full exact-visible *Kpoly* payload if the actual support reaches every exact-visible input uniformly in the output index. But for the finite-support full-rule construction, this section hypothesis forces the cardinal inequality

$$|\text{ExactVisiblePostSwitchSurface } Z \ k| \leq |\text{Block}|$$

via

`supportFullRule_surfaceCard_le_blockCard_of_uniformVisibleSection.`

Consequently

`supportFullRule_not_hasUniformVisibleSection_of_card_lt_surfaceCard`

blocks the repair whenever the actual observed support is smaller than the full exact-visible surface.

Lean also tests the fallback “reachable-support quotient” repair directly. The observed block-output family is only the pullback of the exact-visible family along the support map:

`supportFullRule_outputFamily_factorsThrough_predictorFamily.`

If `visibleOf` misses any exact-visible input, Lean constructs two distinct full exact-visible rules that agree on every observed block:

`supportFullRule_exists_distinct_rules_same_outputFamily_of_not_surjective.`

Equivalently, exact recovery from observed block outputs forces the support map to be surjective:

`supportFullRule_observedRuleMap_injective_forces_visibleOf_surjective.`

The same obstruction is now stated in the stronger post-processing form. If a candidate repair supplies a decoder

$$\text{decode} : (\text{Block} \rightarrow \text{Bool}) \rightarrow \text{ExactVisibleRule } Z \ k$$

that reconstructs every full exact-visible rule from its observed block trace, then the support map must again be surjective:

`supportFullRule_exactDecoder_forces_visibleOf_surjective.`

The current Lean packet also blocks the softer reply that the proof only needs some downstream invariant of the rule, not the whole rule. Any property $P : \text{ExactVisibleRule } Z \ k \rightarrow \alpha$ decoded from observed block traces must be constant on observed-output fibers:

`supportFullRule_propertyDecoder_constant_on_observedFibers.`

Lean now proves that this condition is also sufficient:

`supportFullRule_exists_propertyDecoder_iff_constant_on_observedFibers.`

Thus a reachable-support quotient is not a free replacement for the full-rule target. Each later invariant must be proved to factor through the observed quotient; otherwise the missing off-support values are semantically available to the full exact-visible target but invisible to the actual block trace. So any property that separates two full rules with the same observed trace cannot be recovered:

`supportFullRule_not_exists_propertyDecoder_of_same_output_ne.`

In particular, if an exact-visible input u_0 is outside the reachable support, even the single bit $r(u_0)$ is not observable from the block trace:

`supportFullRule_not_exists_evalDecoder_of_missed_point.`

The latest point-query boundary is exact, not merely negative. Lean proves that a decoder for the single query $r(u_0)$ exists precisely when u_0 is in the range of the actual support map:

`supportFullRule_exists_evalDecoder_iff_mem_range.`

Consequently, supplying decoders for all exact-visible point queries is equivalent to full reachable-support surjectivity:

`supportFullRule_all_evalDecoders_iff_visibleOf_surjective.`

The same boundary is now available for an arbitrary family of downstream point probes $Q \rightarrow \text{ExactVisiblePostSwitchSurface } Z \ k$. A decoder for the whole query vector exists if and only if every queried point is in the actual reachable support:

`supportFullRule_exists_queryDecoder_iff_forall_mem_range.`

If even one requested probe is outside that support, no such query-vector decoder exists:

`supportFullRule_not_exists_queryDecoder_of_missed_query.`

The latest strengthening handles adaptive downstream probing. Lean now defines `AdaptiveBoolQueryTree`, a finite decision tree whose later exact-visible point queries may depend on earlier rule-bit answers. If every point that can be queried by the tree lies in the reachable support, the whole adaptive computation can be simulated from the observed block trace:

`supportFullRule_exists_adaptiveQueryDecoder_of_allQueriesReachable.`

Conversely, adaptive decoding still factors through exactly the same quotient:

`supportFullRule_exists_adaptiveQueryDecoder_iff_constant_on_observedFibers.`

For crux extraction this gives a compact certificate:

`supportFullRule_not_exists_adaptiveQueryDecoder_of_same_output_eval_ne.`

Any two full rules that agree on every observed block but make the adaptive tree return different outputs refute all observed-trace decoders for that tree. This is the form to use for deeper missed queries, because the witness rules can be chosen to follow the same earlier branch transcript and then differ at the off-support query. In particular, if the root query is off support and the two root branches have different final outputs, no observed-trace decoder exists:

`supportFullRule_not_exists_rootAdaptiveQueryDecoder_of_missed_point_ne.`

Thus adaptive choice of probes does not repair the support gap. It can only use off-support rule bits after a separate theorem proves that those queried bits are actually reachable, or that the tree's final output is invariant on the observed quotient. Therefore, below surface cardinality,

`supportFullRule_not_observedRuleMap_injective_of_card_lt_surfaceCard`

and

`supportFullRule_exists_distinct_rules_same_outputFamily_of_card_lt_surfaceCard`

and now also

`supportFullRule_not_exists_exactDecoder_of_card_lt_surfaceCard`

and

`supportFullRule_exists_unobservable_eval_of_card_lt_surfaceCard`

show that the quotient literally forgets off-support bits of the local rule. Thus observed actual-local support smallness is not enough. A viable manuscript bridge must either prove a finite-image theorem for the selected local rules on the whole exact-visible surface, prove uniform reachability with support at least the surface size, or explicitly replace the full-rule recovery/ERM target by a reachable-support quotient and prove that every later point query or invariant is supported by, or constant on, that weaker quotient target. The plug-in endpoint repair is now tested separately from the observed-support quotient. A natural response to the *Kpoly* gap is that the actual local rule is not an arbitrary full rule, but the result of a finite-alphabet plug-in lookup or plug-in majority procedure. Lean shows that this phrasing does not by itself reduce the image. The literal lookup endpoint in `ActualSwitchedLocalPluginLookupObstruction.lean` is just the full Boolean lookup-table family:

`pluginLookupActualSwitchedLocalInterface_surjective.`

Consequently it has no clocked finite-learning payload below the full truth-table budget, and it cannot be identified with bounded ZAB decision-list constructor data:

`pluginLookupActualSwitchedLocalInterface_not_clockedKpolyFiniteLearningPayload_of_lt_surfaceCard`

`pluginLookupActualSwitchedLocalInterface_not_zabDecisionListData_of_lt_surfaceCard.`

The new file `ActualSwitchedLocalPluginLookupSparseThresholdERMObstruction.lean` pushes the same literal lookup endpoint against the later sparse-threshold route itself. Since unrestricted lookup is still surjective on the full exact-visible rule family, any hypothetical shared exact sparse-threshold ERM packet or recovery packet already inherits the unconditional visible-budget inequality

$$n \leq 2r + 2k + 1$$

on `BitVec n`. The regression file instantiates this at `BitVec 2`, $k = 0$, and extractor width $r = 0$, so Lean forces $2 \leq 1$ and rules out both the exact-family and recovery packets on the manuscript’s literal finite-alphabet lookup endpoint. The counted-majority version is no better without a further structural restriction. `PluginMajorityCounts.ofRule` puts one vote on the desired side at every exact-visible input, so

`pluginMajorityActualSwitchedLocalInterface_surjective`

and the same below-surface payload and ZAB-constructor refuters follow. The follow-up file `ActualSwitchedLocalPluginMajoritySparseThresholdERMRecoveryHeavyRegionCharacterization.lean` closes the same later repair at the counted-majority level. Since unrestricted counted plug-in majority is still surjective on the full exact-visible rule family, Lean now proves that any hypothetical sparse-threshold recovery packet already forces

$$1 - \mu(\text{PMF.lightestPoint } \mu) \leq q.$$

On the skew `BitVec 1` witness $(4/5, 1/5)$, the regression file derives the concrete threshold

$$4/5 \leq q,$$

and therefore rules out the manuscript-style threshold $q = 3/4$ before any sample-budget or injective-branch argument is used. The sample-level version removes the last abstraction: the graph sample of a rule contains one labeled example $(u, r(u))$ for every exact-visible input and plug-in majority recovers r pointwise:

`PluginSampleMajority.predict_graphSample.`

Thus

`pluginSampleMajorityActualSwitchedLocalInterface_surjective`

and the corresponding negative *Kpoly* payload and ZAB-constructor theorems hold. The new file `ActualSwitchedLocalPluginSampleMajoritySparseThresholdERMObstruction.lean` pushes that same endpoint against the later manuscript-facing shared sparse-threshold ERM wrapper. Since unrestricted sample-level plug-in majority is still surjective on the full exact-visible rule family, Lean now proves that it cannot carry

`ActualSwitchedLocalInterface.SharedExactZABSparseThresholdERMData`

or the stronger recovery packet below the same point-block visible-budget threshold. On `BitVec n`, any such identification would force

$$n \leq 2r + 2k + 1.$$

So “finite sample plus majority” is still not enough to enter the already analyzed sparse-threshold ERM route: the manuscript would need a separate theorem showing that its sample-majority endpoint

really lands in that class. The follow-up file `ActualSwitchedLocalPluginSampleMajoritySparseThresholdERMRe` closes the next obvious repair. It no longer argues by visible-budget shortage. Instead it uses the same unrestricted surjectivity to transfer the later recovery-side heavy-region theorem directly to sample-level plug-in majority:

`pluginSampleMajorityActualSwitchedLocalInterface_one_sub_apply_lightestPoint_le_of_nonempty_sh`

So any hypothetical recovery packet on unrestricted sample-level plug-in majority already forces

$$1 - \mu(\text{PMF.lightestPoint } \mu) \leq q.$$

On the same skew `BitVec 1` witness used for the later full-rule and fallback counterexamples, the regression file proves this means

$$4/5 \leq q,$$

and therefore rules out the manuscript-style threshold $q = 3/4$ even before any bounded-code or injective-index argument is invoked. Finally, bounded sample size gives an exact threshold rather than a free compression theorem. In `ActualSwitchedLocalBoundedSampleMajorityObstruction.lean`, Lean proves

`boundedSamplePluginMajorityActualSwitchedLocalInterface_surjective_iff.`

The endpoint is full-rule expressive exactly when the sample bound reaches $|\text{ExactVisiblePostSwitchSurface } Z \ k|$. If the bound is below that surface size, the all-false rule is not realizable because some input is omitted and the tie-breaking majority returns `true`. If the bound reaches the surface size, the graph-sample construction restores full expressivity, and Lean again proves the negative clocked-payload and ZAB-constructor statements.

The bounded side has a sharper invariant than the all-false witness. Lean now defines

`PluginSampleMajority.falseSupport`

and proves

`PluginSampleMajority.falseSupport_predict_card_le_bound,`

because any input omitted from the sample has zero true and false votes and therefore returns the tie-breaking value `true`. At the actual-local endpoint this becomes

`boundedSamplePluginMajorityActualSwitchedLocalInterface_falseSupport_card_le_sampleBound`

and

`boundedSamplePluginMajorityActualSwitchedLocalInterface_not_realizes_rule_of_sampleBound_lt_fa`

So a bounded plug-in majority endpoint can realize only rules whose false support has cardinality at most the sample bound. Therefore “finite plug-in majority” is not a manuscript-side repair by itself. The missing theorem must show either that the intended target rules have small false support under the chosen tie-breaking convention, or that the selected samples/rules are forced into one of the already formalized bounded ZAB/code families.

The same obstruction is now tie-break invariant. Lean defines

`PluginSampleMajority.predictWithDefault`

and

`PluginSampleMajority.nondefaultSupport`

and proves that omitted inputs return the chosen default:

`PluginSampleMajority.predictWithDefault_eq_default_of_not_mem_map fst.`

Consequently

`PluginSampleMajority.nondefaultSupport_predictWithDefault_card_le_bound`

and, at the actual-local endpoint,

`boundedSamplePluginMajorityWithDefaultActualSwitchedLocalInterface_nondefaultSupport_card_le_bound`

Any target whose nondefault support exceeds the sample bound is not realized, and the exact surjectivity threshold is unchanged for every default:

`boundedSamplePluginMajorityWithDefaultActualSwitchedLocalInterface_surjective_iff.`

Thus changing the plug-in majority tie-break from `true` to `false`, or to any explicitly fixed Boolean default, is not a repair.

The remaining obvious tie-break escape is to let the fallback depend on the visible input. Lean now makes this case explicit rather than leaving it as a named repair theorem. It defines

`PluginSampleMajority.predictWithFallback`

and the indexed endpoint

`boundedSamplePluginMajorityWithFallbackActualSwitchedLocalInterface.`

The theorem

`boundedSamplePluginMajorityWithFallbackActualSwitchedLocalInterface_emptySample_predict`

proves that the empty sample already returns the fallback rule exactly, and

`boundedSamplePluginMajorityWithFallbackActualSwitchedLocalInterface_surjective`

therefore proves full exact-visible rule expressivity for every sample bound, including zero. Consequently,

`boundedSamplePluginMajorityWithFallbackActualSwitchedLocalInterface_not_finitePredictorCover_or`

and

`boundedSamplePluginMajorityWithFallbackActualSwitchedLocalInterface_not_clockedKpolyFiniteLearn`

hold below the exact-visible truth-table budget. This closes the fallback repair fork sharply: an input-dependent tie-break is not a bounded-sample majority improvement, but a full lookup table inserted into the index.

Lean also isolates the more disciplined fallback repair where the fallback is not arbitrary, but selected from an explicit family. It defines

`PluginSampleMajority.disagreementSupport,`

`BoundedSampleFallbackFamilyIndex,`

and the endpoint

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface.`

The theorem

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_disagreementSupport_eq_of_eq_off_support.`

proves that the selected predictor can differ from its selected fallback rule on at most `sampleBound` exact-visible inputs. The theorem

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_not_realizes_rule_of.`

therefore blocks realization of any target rule that lies more than `sampleBound` points away from every fallback code. Thus a structured fallback repair has been reduced to a genuine finite Hamming-cover obligation; bounded sampling alone only supplies sparse overrides around the chosen fallback.

This reduction is now exact, not merely one-sided. Lean defines a finite support sample constructor

`PluginSampleMajority.supportSample`

and proves

`PluginSampleMajority.predictWithFallback_supportSample_eq_of_eq_off_support.`

The endpoint theorem

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_realizes_rule_iff_membership.`

then characterizes the predictor image exactly: a target rule is realized iff it belongs to the radius-`sampleBound` Hamming cover around the fallback family. Thus there is no hidden extra expressive channel in the bounded sample part of the construction; the entire repair burden is the finite Hamming-cover burden.

Lean now records the corresponding exact repair criterion. The theorem

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_surjective_iff_fallback_rule_image.`

proves that the structured fallback endpoint is surjective exactly when the fallback-family radius cover is the full exact-visible Boolean rule cube. The pointwise theorem

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_surjective_iff_forall_rule_image.`

unpacks the same condition: every exact-visible rule must be within Hamming distance `sampleBound` of some fallback code. This is the minimal repair theorem for this fork. A proposed structured fallback repair must actually prove that cover; otherwise the exact image theorem supplies a target rule outside the interface image.

The exact repair criterion now has its direct numerical lower-bound form too. The reusable finite-cover theorem

`PluginSampleMajority.boolClassifierSpace_card_le_code_mul_smallSubsets_card_of_radiusCover_eq_rule_image.`

proves that any fallback-family radius cover equal to the whole Boolean rule cube must have

$$2^{|\text{surface}|} \leq |\text{FallbackIndex}| \cdot |\text{smallSubsets}(\text{surface}, \text{sampleBound})|.$$

The endpoint theorem

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_boolClassifierSpace_`

transfers this bound to the structured fallback interface. Thus the repair burden is not merely qualitative coverage: full expressivity forces a concrete cardinality lower bound on fallback codes and sparse override supports.

The Hamming-cover obligation is now counted explicitly. Lean defines

`PluginSampleMajority.smallSubsets`

for supports of size at most the sample radius and

`PluginSampleMajority.fallbackFamilyRadiusCover`

for the union of Hamming balls around the fallback family. The theorem

`PluginSampleMajority.fallbackFamilyRadiusCover_card_le`

proves that this union has size at most

$$|\text{FallbackIndex}| \cdot |\text{smallSubsets}(\text{surface}, \text{sampleBound})|.$$

Consequently the endpoint theorem

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_finitePredictorCover`

gives an explicit predictor-image cover of at most that size, and

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_not_surjective_of_co`

proves non-surjectivity whenever this product is below the full exact-visible truth-table space. This blocks the structured fallback repair unless the fallback family is large enough, together with radius-`sampleBound` sparse overrides, to cover the Boolean cube.

Lean now specializes this burden to fallback families indexed by a fixed bit budget. The endpoint

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface`

uses a universe-lifted `BitCode fallbackBits` index, so the statement still applies to arbitrary finite hidden alphabets. The theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_boolClassifierSpa`

proves that full-rule expressivity of a `fallbackBits`-bit fallback family forces

$$2^{|\text{surface}|} \leq 2^{\text{fallbackBits}} \cdot |\text{smallSubsets}(\text{surface}, \text{sampleBound})|.$$

Conversely,

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_of`

proves non-surjectivity under the strict opposite inequality. This moves the repair obligation from an abstract finite fallback-family size to an explicit code-budget condition: a proposed bit-coded fallback mechanism must pay enough bits, plus enough sampled sparse overrides, to cover the full exact-visible Boolean rule cube.

Lean now expands the sparse-override factor exactly. The theorem

`PluginSampleMajority.smallSubsets_eq_biUnion_powersetCard_range`

partitions the radius- r support family by support cardinality, and

`PluginSampleMajority.card_smallSubsets_eq_sum_range_choose`

proves

$$|\text{smallSubsets}(\text{surface}, r)| = \sum_{i=0}^r \binom{|\text{surface}|}{i}.$$

Consequently,

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_boolClassifierSpa`

specializes the repair burden to

$$2^{|\text{surface}|} \leq 2^{\text{fallbackBits}} \sum_{i=0}^{\text{sampleBound}} \binom{|\text{surface}|}{i}.$$

The companion theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_of`

blocks full expressivity under the strict opposite inequality. This makes the remaining escape route quantitatively explicit: for sublinear sampled override radius, the repair must still justify how a binomial-prefix Hamming ball times the fallback code family covers an exponential truth-table cube.

The exact binomial-prefix count now has its own additive bit-budget obstruction, not merely a coarse polynomial corollary. If a proposed route certifies

$$\sum_{i=0}^{\text{sampleBound}} \binom{|\text{surface}|}{i} \leq 2^{\text{overheadBits}},$$

then

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_fallbackBits_add`

proves

$$|\text{surface}| \leq \text{fallbackBits} + \text{overheadBits}.$$

The matching theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_of`

blocks full-rule expressivity under the same exact envelope assumption and a strict total-bit deficit. This closes the natural repair “use the exact Hamming-ball size instead of the polynomial envelope”:

the exact count may be smaller, but once it is represented as an overhead bit budget, the same truth-table lower bound applies.

The exact support count is also now strict below full radius. Lean proves

`PluginSampleMajority.smallSubsets_ssubset_univ_of_radius_lt_card`

and

`PluginSampleMajority.card_smallSubsets_lt_twoPow_card_of_radius_lt_card,`

so if $r < |\text{surface}|$, then the radius- r Hamming-ball support family is a proper subfamily of all finite supports and has cardinality strictly below $2^{|\text{surface}|}$. Equivalently,

`PluginSampleMajority.sum_range_choose_lt_twoPow_of_lt`

proves

$$\sum_{i=0}^r \binom{n}{i} < 2^n \quad \text{whenever } r < n.$$

The route-level consequence is the zero-fallback blocker

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_zero`

With only one structured fallback rule available, bounded sampled overrides below the visible alphabet size cover a proper Hamming ball, not the full Boolean rule cube.

The same formalization now proves that this boundary is sharp rather than an artifact of the negative counting proof. The generic repair-side theorem

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_surjective_of_nonempty`

shows that any nonempty fallback family becomes full-rule expressive once the sample bound reaches the exact-visible alphabet size. Its bit-coded specialization is

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_surjective_of_sur`

Together with the strict-radius blocker, Lean derives the exact zero-fallback dichotomy

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_zeroFallbackBits_s`

Thus the zero-fallback repair cannot be rescued by a clever tie-breaking choice: with no fallback bits, full expressivity is equivalent to allowing enough sampled overrides to touch every exact-visible input.

The next file, `ActualSwitchedLocalBitFallbackWrapperSparseThresholdERMObstruction.lean`, packages the corresponding manuscript-facing consequence for the later shared sparse-threshold ERM wrapper. If the fallback decoder is already surjective onto all exact-visible Boolean rules, then the empty sample leaves that fallback rule unchanged, so the wrapped endpoint is surjective for every sample bound. Lean therefore transfers the same sparse-threshold visible-budget obstruction directly to this repair path: such a wrapped endpoint cannot carry

`ActualSwitchedLocalInterface.SharedExactZABSparseThresholdERMData`

or the stronger recovery packet below the same point-block threshold, and on `BitVec n` any such identification would still force

$$n \leq 2r + 2k + 1.$$

The regression file `ActualSwitchedLocalBitFallbackWrapperSparseThresholdERMObstructionRegression.lean` checks this on the one-sample wrapped endpoint with the canonical truth-table fallback decoder. Even after adding that full-rule fallback side channel, `BitVec 2, k = 0, r = 0` still refutes the sparse-threshold ERM identification. So “majority plus a full-rule fallback decoder” is not a new sparse-threshold repair; it is just another surjective actual-local endpoint.

The follow-up file `ActualSwitchedLocalBitFallbackWrapperSparseThresholdERMRecoveryHeavyRegionCharacterization.lean` shows that the same fallback side channel also fails at the later q -threshold recovery stage. Once the fallback decoder is surjective, the wrapped endpoint is still surjective, so Lean transfers the recent surjective heavy-region characterization directly through the wrapper. Any hypothetical recovery packet on that endpoint already forces

$$1 - \mu(\text{lightestPoint}) \leq q.$$

This is not another visible-width or truth-table-budget wrapper: it is the same intrinsic lightest-point threshold now stated on the fallback-side repair path itself.

The synthesis layer now cashes those two axes out together on one manuscript endpoint instead of leaving them as separate local facts. In `CruxSynthesis.lean`, the theorem

`kpolyCoverage_anchor_boundedSampleMajorityWithBitFallbackFamily_visibleWidth_and_lightestPoint`

states that any hypothetical recovery packet on the bounded-sample bit-fallback endpoint with the canonical truth-table decoder must satisfy both

$$n \leq 2r + 2k + 1 \quad \text{and} \quad 1 - \mu(\text{lightestPoint}) \leq q.$$

Its companion theorem

`kpolyCoverage_anchor_boundedSampleMajorityWithBitFallbackFamily_not_nonempty_recovery_of_exact`

packages the contrapositive: violating either inequality already rules out the same recovery packet. This is the first explicit two-axis composition here on one exact-visible bounded-sample endpoint, rather than another one-axis refinement.

The regression file `ActualSwitchedLocalBitFallbackWrapperSparseThresholdERMRecoveryHeavyRegionCharacterization.lean` uses the one-sample wrapped endpoint with the canonical truth-table fallback decoder on the skew `BitVec 1` measure $(4/5, 1/5)$. Lean proves the theorem-shape canary $1 - \mu(\text{lightestPoint}) \leq q$, derives the concrete threshold $4/5 \leq q$, and therefore rules out $q = 3/4$ even after granting both sampled overrides and a full-rule fallback decoder. So that fallback side channel does not escape the later recovery obstruction either.

This repair path is now also promoted into the synthesis ledger rather than left only in the local obstruction file. `CruxSynthesis.lean` now records one hundred and sixty-one separate narrow covered entries for this cluster: the sample-level majority visible-budget boundary, the bit-fallback visible-budget boundary, the fallback-side q -threshold `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitVec`, the canonical exact-decoder full-rule boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitVec`, the canonical exact-decoder sparse-threshold ERM visible-budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitVec`, the canonical exact-decoder no-extractor sparse-threshold ERM visible-budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitVec`, the radius-zero exact-decoder finite-cover obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitVec`.

the radius-zero exact-decoder representative-cover obstruction `PNPKpolySubrepairClass.actualLocalBoundedSam`
 the radius-zero exact-decoder quotient obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorit`
 the radius-zero exact-decoder exact-visible compression obstruction `PNPKpolySubrepairClass.actualLocalBounded`
 the radius-zero exact-decoder clocked-realization obstruction `PNPKpolySubrepairClass.actualLocalBoundedSamp`
 the radius-zero exact-decoder clocked-payload obstruction `PNPKpolySubrepairClass.actualLocalBoundedSample`
 the canonical exact-decoder visible-width boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajori`
 the canonical exact-decoder lightest-point boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajor`
 the canonical exact-decoder joint recovery boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajor`
 the canonical exact-decoder no-extractor visible-width boundary `PNPKpolySubrepairClass.actualLocalBoundedS`
 the canonical exact-decoder no-extractor lightest-point boundary `PNPKpolySubrepairClass.actualLocalBoundedS`
 the canonical exact-decoder no-extractor joint recovery boundary `PNPKpolySubrepairClass.actualLocalBoundedS`
 the canonical exact-decoder clocked-payload obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMa`
 the polynomial-envelope additive budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajori`
 the polynomial-envelope surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajority`
 the factored-overhead additive budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajority`
 the factored-overhead surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBi`
 the off-by-one successor-factor additive budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMa`
 the off-by-one successor-factor surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMaj`
 the double-successor-factor additive budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMaj`
 the double-successor-factor surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajori`
 the exact one-sample additive budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityB`
 the exact one-sample surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBit`
 the raw one-sample successor-budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityB`
 the raw one-sample successor-budget surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSamp`
 the zero-sample additive budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFa`
 the zero-sample surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFall`
 the zero-sample exact-decoder boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFall`
 the zero-sample fallback-equivalence boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBi`
 the zero-sample finite predictor-cover boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityB`
 the zero-sample finite representative-cover boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajor`
 the zero-sample finite quotient-code boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBi`
 the zero-sample exact visible compression boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajori`
 the zero-sample clocked realization boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBit`
 the zero-sample clocked-payload boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFa`
 the exact one-sample product boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFall`
 the exact one-sample product surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMaj`
 the exact two-sample quadratic product boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorit`
 the exact two-sample quadratic product surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSa`
 the exact two-sample quadratic-envelope budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSample`
 the exact two-sample quadratic-envelope surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedS`
 the fallback-side finite predictor-cover obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorit`
 the fallback-side finite representative-cover obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMaj`
 the fallback-side finite quotient-code obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajority`
 the fallback-side exact visible compression obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMaj`
 the fallback-side clocked realization obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityF`
 the fallback-side clocked finite-learning payload obstruction `PNPKpolySubrepairClass.actualLocalBoundedSamp`
 the exact fallback-family radius-cover boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityF`
 the fallback-family finite predictor-cover boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorit`

the fallback-family radius-cover surjectivity boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityRadiusCoverSurjectivityBoundary`
 the fallback-family pointwise radius-cover surjectivity boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityPointwiseRadiusCoverSurjectivityBoundary`
 the fallback-family empty-sample realization boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityEmptySampleRealizationBoundary`
 the fallback-family surjective-transport boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajoritySurjectiveTransportBoundary`
 the fallback-family large-disagreement-support obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityLargeDisagreementSupportObstruction`
 the abstract fallback-family support-product boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajoritySupportProductBoundary`
 the abstract fallback-family support-product surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajoritySupportProductSurjectivityObstruction`
 the generic fallback-family full-radius boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityFullRadiusBoundary`
 the generic fallback-family zero-sample fallback-equivalence boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroSampleFallbackEquivalenceBoundary`
 the generic fallback-family zero-sample finite predictor-cover boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroSampleFinitePredictorCoverBoundary`
 the generic fallback-family zero-sample finite representative-cover boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroSampleFiniteRepresentativeCoverBoundary`
 the generic fallback-family zero-sample finite quotient-code boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroSampleFiniteQuotientCodeBoundary`
 the generic fallback-family zero-sample exact visible compression boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroSampleExactVisibleCompressionBoundary`
 the generic fallback-family zero-sample clocked realization boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroSampleClockedRealizationBoundary`
 the generic fallback-family zero-sample clocked-payload boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroSampleClockedPayloadBoundary`
 the bit-fallback exact radius-cover boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFallbackExactRadiusCoverBoundary`
 the bit-fallback finite predictor-cover boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFallbackFinitePredictorCoverBoundary`
 the bit-fallback radius-cover surjectivity boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFallbackRadiusCoverSurjectivityBoundary`
 the bit-fallback pointwise radius-cover surjectivity boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFallbackPointwiseRadiusCoverSurjectivityBoundary`
 the abstract sparse-support product boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFallbackSparseSupportProductBoundary`
 the abstract sparse-support product surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFallbackSparseSupportProductSurjectivityObstruction`
 the generic bit-fallback full-radius boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityBitFallbackGenericFullRadiusBoundary`
 the zero-fallback exact-visible deficit obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroFallbackExactVisibleDeficitObstruction`
 the zero-fallback exact-radius boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroFallbackExactRadiusBoundary`
 the generic sum-of-binomial product boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroFallbackSumOfBinomialProductBoundary`
 the generic sum-of-binomial product surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroFallbackSumOfBinomialProductSurjectivityObstruction`
 the generic sum-of-binomial-envelope budget boundary `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroFallbackSumOfBinomialEnvelopeBudgetBoundary`
 the generic sum-of-binomial-envelope surjectivity obstruction `PNPKpolySubrepairClass.actualLocalBoundedSampleMajorityZeroFallbackSumOfBinomialEnvelopeSurjectivityObstruction`
 the generic surjective-actual-local joint recovery barrier `PNPKpolySubrepairClass.surjectiveActualLocalJointRecoveryBarrier`
 the generic surjective-actual-local visible-width threshold `PNPKpolySubrepairClass.surjectiveActualLocalRecoveryVisibleWidthThreshold`
 the generic surjective-actual-local lightest-point threshold `PNPKpolySubrepairClass.surjectiveActualLocalRecoveryLightestPointThreshold`
 and its no-extractor corollary `PNPKpolySubrepairClass.surjectiveActualLocalNoExtractorLightestPointThreshold`
 plus the broader unconditional half-width no-extractor boundary `PNPKpolySubrepairClass.surjectiveActualLocalUnconditionalHalfWidthNoExtractorBoundary`
 the observed-support noninjective observed-rule-map bridge obstruction `PNPKpolySubrepairClass.supportFullRuleNoninjectiveObservedRuleMapBridgeObstruction`
 the observed-support distinct-rules-same-output bridge obstruction `PNPKpolySubrepairClass.supportFullRuleDistinctRulesSameOutputBridgeObstruction`
 the observed-support adaptive-query collision obstruction `PNPKpolySubrepairClass.supportFullRuleNoAdaptiveQueryCollisionObstruction`
 the observed-support missed-root adaptive-query obstruction `PNPKpolySubrepairClass.supportFullRuleNoRootAdaptiveQueryObstruction`
 the observed-support no-exact-decoder bridge obstruction `PNPKpolySubrepairClass.supportFullRuleNoExactDecoderBridgeObstruction`
 and the observed-support unobservable-evaluation obstruction `PNPKpolySubrepairClass.supportFullRuleUnobservableEvaluationObstruction`
 So the covered Kpoly subledger now does not merely remember the fallback-surjective specializations; it also records both the direct fallback-side finite-cover and clocked-payload failures, the radius-zero exact-decoder no-compression packet, the exact-decoder visible-width threshold, the exact-decoder lightest-point threshold, the exact-decoder visible-width/lightest-point/joint no-extractor obstruction forms, and the underlying generic barrier that any surjective actual-local endpoint must obey. It now also records the generic finite-support/full-rule bridge packet: `PNPKpolySubrepairClass.supportFullRuleObservedSmallNotExactVisibleCoverBoundary` and `PNPKpolySubrepairClass.supportFullRuleNotClockedPayloadBelowSurfaceCard`. It also records the uniform-section surface-cardinality boundary `PNPKpolySubrepairClass.supportFullRuleUniformSectionSurfaceCardinalityBoundary`

and the below-surface no-uniform-section obstruction `PNPKpolySubrepairClass.supportFullRuleNoUniformSection`. It now also records the positive uniform-section lift packet: `PNPKpolySubrepairClass.supportFullRuleUniformSection` and `PNPKpolySubrepairClass.supportFullRuleUniformSectionClockedPayloadBoundary`. So a finite observed block-output support can still coexist with a full exact-visible family that has no below-surface predictor cover, and the same small-support regime still fails every clocked finite-learning payload below surface cardinality. Any would-be uniform visible section already forces the observed support cardinality up to the full exact-visible surface size, so a below-surface support deficit admits no such section at all. And if such a uniform section did exist, the observed block-truth-table cover and the resulting block-cardinality clocked payload would really transfer back to the full exact-visible family. It now also records the generic property-decoder rigidity packet: `PNPKpolySubrepairClass.supportFullRulePropertyDecoderFiberConstancyBoundary`, `PNPKpolySubrepairClass.supportFullRulePropertyDecoderFiberConstancyClockedPayloadBoundary`, and `PNPKpolySubrepairClass.supportFullRuleNoPropertyDecoderSameOutputBoundary`. So any downstream property recovered from observed block-output traces must be constant on observed fibers, a property is decodable exactly when that fiber-constancy holds, and any property separating two full rules with the same observed trace is therefore undecodable. It now also records the reachable-support query-decoder packet: `PNPKpolySubrepairClass.supportFullRuleEvalDecoderRangeBoundary`, `PNPKpolySubrepairClass.supportFullRuleAllEvalDecodersSurjectiveBoundary`, `PNPKpolySubrepairClass.supportFullRuleEvalDecoderRangeClockedPayloadBoundary`, and `PNPKpolySubrepairClass.supportFullRuleNoQueryDecoderMissedQueryBoundary`. So a single exact-visible point query is decodable exactly when that point lies in reachable support, decoding every point query is equivalent to surjectivity of the reachable-support map, a whole query family is decodable exactly when all queried points are reachable, and any family containing one missed query is therefore blocked outright. It now also records the generic observed-quotient factorization and adaptive reachability packet: `PNPKpolySubrepairClass.supportFullRuleOutputFamilyFactorBoundary`, `PNPKpolySubrepairClass.supportFullRuleObservedRuleInjectiveSurjectiveBoundary`, `PNPKpolySubrepairClass.supportFullRuleAdaptiveQueryDecoderReachableBoundary`, and `PNPKpolySubrepairClass.supportFullRuleAdaptiveQueryDecoderReachableClockedPayloadBoundary`. So the observed block-output quotient is only the pullback of the full exact-visible family along reachable support, injectivity of that quotient or the existence of an exact full-rule decoder already forces reachable-support surjectivity, any adaptive exact-visible query tree with all queried points reachable is decodable from observed traces, and such a tree is decodable exactly when its output is constant on observed fibers. It now also records the exact-visible image-lower-bound packet: `PNPKpolySubrepairClass.exactVisibleRepresentativeCoverSurjectivityObstruction`, `PNPKpolySubrepairClass.exactVisibleRepresentativeCoverClockedPayloadObstruction`, `PNPKpolySubrepairClass.exactVisibleClockedRealizationSurjectivityObstruction`, `PNPKpolySubrepairClass.exactVisibleClockedRealizationClockedPayloadObstruction`, and `PNPKpolySubrepairClass.injectiveFiniteProbeClockedRealizationObstruction`. So a fully expressive exact-visible family already blocks not just finite predictor-image covers but also finite representative-index covers, finite quotient-code presentations, and the clocked exact-visible realization target below the full Boolean image size. The payload-side obstruction is now also split out explicitly as `PNPKpolySubrepairClass.exactVisibleClockedFiniteLearningPayloadSurjectivityObstruction` and `PNPKpolySubrepairClass.injectiveFiniteProbeClockedFiniteLearningPayloadObstruction`, so a fully expressive exact-visible family already blocks the bundled clocked finite-learning payload below the full Boolean image size, and any injectively realized finite probe family already transfers the same payload obstruction below the realized probe cardinality. Likewise, any injectively realized finite probe family already transfers the same obstruction to exact-visible compression and clocked realization below the realized probe cardinality. The payload-side lower-bound packet is now also explicit as `PNPKpolySubrepairClass.exactVisibleClockedFiniteLearningPayloadSurjectivityLowerBound`, `PNPKpolySubrepairClass.injectiveFiniteProbeClockedFiniteLearningPayloadLowerBound`, `PNPKpolySubrepairClass.sectionBackedClockedFiniteLearningPayloadSurjectiveObstruction`, `PNPKpolySubrepairClass.sectionBackedInjectiveFiniteProbeClockedFiniteLearningPayloadLowerBound`, and `PNPKpolySubrepairClass.sectionBackedInjectiveFiniteProbeClockedFiniteLearningPayloadObstruction`.

So the bundled clocked finite-learning payload itself now inherits the same positive lower bounds under full exact-visible surjectivity, injectively realized finite probe families, and section-backed pullbacks of both kinds, with the corresponding below-cardinality obstructions recorded explicitly for the two section-backed variants. It now also records the finite-probe representative/quotient lower-bound and section-backed pullback packet: `PNPKpolySubrepairClass.injectiveFiniteRepresentativeIndexCoverLowerBound`, `PNPKpolySubrepairClass.injectiveFinitePredictorQuotientLowerBound`, `PNPKpolySubrepairClass.sectionBackedInjectiveFiniteProbePullbackObstruction`, `PNPKpolySubrepairClass.sectionBackedInjectiveFiniteRepresentativeIndexCoverObstruction`, `PNPKpolySubrepairClass.sectionBackedInjectiveFinitePredictorQuotientLowerBound`, and `PNPKpolySubrepairClass.sectionBackedInjectiveFinitePredictorQuotientObstruction`. So the same injectively realized finite probe family already forces the same cardinality lower bound for representative-index covers and quotient-code presentations, and every section-backed pullback of such a reduced view inherits both the positive lower bounds and the corresponding below-cardinality obstructions for predictor covers, representative-index covers, and quotient codes. It now also records the finite-image equivalence / transport / reduced- (a, b) pullback packet: `PNPKpolySubrepairClass.finiteRepresentativeIndexCoverEquivalence`, `PNPKpolySubrepairClass.finitePredictorCoverBudgetWeakeningBoundary`, `PNPKpolySubrepairClass.finitePredictorQuotientBudgetWeakeningBoundary`, `PNPKpolySubrepairClass.finitePredictorCoverFactorTransportBoundary`, `PNPKpolySubrepairClass.finitePredictorQuotientFactorTransportBoundary`, `PNPKpolySubrepairClass.sectionBackedFiniteRepresentativeIndexCoverSurjectiveObstruction`, `PNPKpolySubrepairClass.sectionBackedFinitePredictorQuotientSurjectiveObstruction`, `PNPKpolySubrepairClass.sectionBackedFiniteRepresentativeIndexCoverSurjectiveLowerBound`, `PNPKpolySubrepairClass.sectionBackedFinitePredictorQuotientSurjectiveLowerBound`, `PNPKpolySubrepairClass.surjectiveFiniteRepresentativeIndexCoverLowerBound`, `PNPKpolySubrepairClass.finiteImageReducedABPullbackObstruction`, `PNPKpolySubrepairClass.finiteImageReducedABPullbackObstruction`, and `PNPKpolySubrepairClass.finiteQuotientReducedABPullbackObstruction`. So finite predictor-image covers, representative-index covers, and quotient-code presentations are already interchangeable at this layer; the predictor-cover / representative-cover / quotient-code budget-weakening moves and factor-transport moves are already internalized separately; section-backed finite summaries already force the same generic below-cardinality obstruction for all three finite-image formulations; the same section-backed surjective summary hypotheses now also yield the matching positive full-Boolean lower bounds in all three formulations; a fully expressive finite family now also carries the same generic full-Boolean lower bound in all three formulations; and the reduced- (a, b) pullback variants are already recognized as the same below-cardinality obstruction whether stated in predictor-cover, injective-probe, or quotient-code form. It now also records the exact-visible compression / realization equivalence packet: `PNPKpolySubrepairClass.exactVisibleCompressionTargetPredictorCoverEquivalence`, `PNPKpolySubrepairClass.clockedExactVisibleRealizationPredictorCoverEquivalence`, `PNPKpolySubrepairClass.clockedFiniteLearningPayloadExactVisibleCompressionEquivalence`, `PNPKpolySubrepairClass.exactVisibleCompressionTargetPredictorQuotientEquivalence`, `PNPKpolySubrepairClass.exactVisibleCompressionTargetRepresentativeIndexCoverEquivalence`, `PNPKpolySubrepairClass.clockedExactVisibleRealizationRepresentativeIndexCoverEquivalence`, `PNPKpolySubrepairClass.notClockedExactVisibleRealizationRepresentativeIndexCoverEquivalence`, and `PNPKpolySubrepairClass.notClockedExactVisibleRealizationPredictorQuotientEquivalence`. So at the exact-visible layer, compression targets, clocked realization targets, and the bundled clocked finite-learning payload are already nothing more than finite predictor-image, quotient-code, or representative-index cover claims at the same bit budget, and the failure of clocked realization is already equivalently the failure of those finite-image schemes. It also records the product-bound /

fielded-switching finite-image bridge packet: `PNPKpolySubrepairClass.productBoundBridgeFiniteImageBoundary`, `PNPKpolySubrepairClass.fieldedSwitchingBridgeFiniteImageBoundary`, `PNPKpolySubrepairClass.productBoundOnlyFiniteImageBoundary`, `PNPKpolySubrepairClass.fieldedSwitchingOnlyFullVisibleObstruction`, `PNPKpolySubrepairClass.productBoundOnlyFullVisibleObstruction`, and `PNPKpolySubrepairClass.fieldedSwitchingOnlySurjectiveVisibleObstruction`. So the manuscript's global product-bound-only and fixed-field-switching-only exact-visible bridge schemas are already nothing more than finite-image-cover demands, and those demands are already refuted below surface cardinality both by the full exact-visible Boolean family and by any already-surjective exact-visible family. It now also records that a cardinally small observed block-output quotient cannot injectively identify the full exact-visible family, already merges two distinct full exact-visible rules to the same observed output trace, cannot exactly decode the full exact-visible family, and must leave at least one exact-visible bit unobservable. It also records the adaptive-query consequence that any observed-output collision separating a tree kills every decoder for that tree, and in particular that missing a queried root point already kills every non-trivial root adaptive-query decoder. And it now records the one-block minimal-instance packet: `PNPKpolySubrepairClass.oneBlockFullRuleObservedSmallNotExactVisibleCoverBoundary`, `PNPKpolySubrepairClass.oneBlockFullRuleNoExactDecoderOneLtSurfaceCardBoundary`, and `PNPKpolySubrepairClass.oneBlockFullRuleUnobservableEvalOneLtSurfaceCardBoundary`. So the one-block observed quotient still has a trivial finite predictor cover while the full exact-visible family stays above every 2^s cover below surface cardinality, already refutes every clocked finite-learning payload there, and once the exact-visible surface has more than one point admits neither an exact full-rule decoder nor total observability of every exact-visible bit. In particular, the manuscript's most permissive bounded-sample bit-fallback endpoint with `exactVisibleRuleDecode` is already full-rule expressive, and on that same endpoint the zero-sample wrapper already carries the full exact-visible no-compression burden, while any hypothetical recovery packet must satisfy both the visible-width ceiling and the lightest-point threshold because both of those obstructions are already enforced at the surjective actual-local layer itself. It also records the exact fallback-family Hamming-cover image theorem, the finite predictor-cover certificate, the exact surjectivity-via-full-cover criterion, the equivalent generic fallback-family pointwise repair criterion, the generic fallback-family empty-sample realization boundary, the generic fallback-surjective transport boundary, the generic fallback-family large-disagreement-support obstruction, the generic zero-sample fallback-equivalence criterion, and now also the four fallback-side no-compression consequences themselves (finite representative-cover, finite quotient-code, exact visible compression, and clocked realization obstructions), together with the equivalent manuscript-facing pointwise repair criterion, before descending to the abstract support-product obstruction, the generic fallback-family full-radius boundary, the abstract sparse-support product obstruction, the generic bit-fallback full-radius and zero-fallback radius boundaries, the radius-zero finite-image and clocked-realization equivalences, and now also the exact pointwise radius-cover criteria themselves as covered ledger entries together with the radius-zero representative-cover and quotient-code transport forms themselves as covered ledger entries, before the generic exact support-count product obstruction and the additive-envelope corollary. So the one-sample and two-sample counting packets are now recognized inside Lean as special cases of an already-ledgered broader Hamming-cover obstruction route, and radius zero is no longer only a surjectivity canary.

Lean also records the coarse polynomial envelope that will usually be easier to compose with asymptotic route statements. The theorem

`PluginSampleMajority.sum_range_choose_le_succ_mul_succ_pow`

proves, for all natural n, r ,

$$\sum_{i=0}^r \binom{n}{i} \leq (r+1)(n+1)^r.$$

The corresponding support-count theorem

$$\text{PluginSampleMajority.card_smallSubsets_le_succ_mul_succ_card_pow}$$

is valid even when the finite input type is empty, because the base is $n+1$. Therefore full expressivity of a bit-coded fallback endpoint implies

$$2^{|\text{surface}|} \leq 2^{\text{fallbackBits}} (\text{sampleBound} + 1) (|\text{surface}| + 1)^{\text{sampleBound}},$$

as formalized by

$$\text{boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_boolClassifierSpa}$$

The strict reverse polynomial inequality gives non-surjectivity via

$$\text{boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_of.}$$

This is not a new analytic lower bound; it is a reusable corollary of the exact finite Hamming-ball count, stated in a shape that makes any “bounded sample plus short fallback code” repair compare directly against the exponential rule cube.

The same corollary is now packaged as an additive bit-budget obstruction. If a proposed route proves that the sparse-override polynomial envelope is bounded by $2^{\text{overheadBits}}$, then full expressivity forces

$$|\text{surface}| \leq \text{fallbackBits} + \text{overheadBits}.$$

This is formalized by

$$\text{boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_fallbackBits_add_c}$$

The strict-deficit form

$$\text{boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_of.}$$

blocks the endpoint when

$$(\text{sampleBound}+1)(|\text{surface}|+1)^{\text{sampleBound}} \leq 2^{\text{overheadBits}} \quad \text{and} \quad \text{fallbackBits}+\text{overheadBits} < |\text{surface}|.$$

Thus a repair cannot hide the sampled override radius in an unspecified “polynomial factor”: it must exhibit enough overhead bits to pay for that factor, and the sum of fallback bits plus overhead bits must still reach the exact-visible alphabet size.

Lean now also closes the common factored-overhead certificate. The arithmetic helper

$$\text{PluginSampleMajority.succ_mul_succ_pow_le_twoPow_add_mul}$$

proves that if $r+1 \leq 2^a$ and $n+1 \leq 2^b$, then

$$(r+1)(n+1)^r \leq 2^{a+br}.$$

Instantiating $r = \text{sampleBound}$ and $n = |\text{surface}|$, the endpoint theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_fallbackBits_add_1`

proves that full expressivity forces

$$|\text{surface}| \leq \text{fallbackBits} + \text{radiusBits} + \text{baseBits} \cdot \text{sampleBound}.$$

The strict-deficit theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_of`

blocks the same endpoint whenever $\text{sampleBound} + 1 \leq 2^{\text{radiusBits}}$, $|\text{surface}| + 1 \leq 2^{\text{baseBits}}$, and the displayed bit sum is still below $|\text{surface}|$. This turns a common informal repair move, budgeting the sample radius and the surface alphabet separately, into an explicit bit inequality that can be checked without leaving the finite interface. These two factor-certificate routes are now also recorded directly in `CruxSynthesis.lean`, so the ledger does not stop at the generic polynomial-factor objection: it now remembers the common separate radius/base-bit budgeting variant too.

Lean now also records the off-by-one variant that a repair is most likely to state directly. The helper

`PluginSampleMajority.succ_le_twoPow_succ_of_le_twoPow`

proves that $\text{sampleBound} \leq 2^{\text{radiusBits}}$ implies $\text{sampleBound} + 1 \leq 2^{\text{radiusBits}+1}$. Consequently

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_fallbackBits_add_1`

proves the lower bound

$$|\text{surface}| \leq \text{fallbackBits} + (\text{radiusBits} + 1) + \text{baseBits} \cdot \text{sampleBound}.$$

The matching theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_of`

gives non-surjectivity under the strict reverse bit inequality. This prevents the endpoint from treating a bit bound on sampleBound as though it were already a bit bound on $\text{sampleBound} + 1$. These successor-factor routes are now also recorded directly in `CruxSynthesis.lean`, so the ledger remembers the raw-value off-by-one variant a manuscript is especially likely to state verbatim. were already a bit bound on the Hamming-ball multiplier $\text{sampleBound} + 1$.

The same successor-bit accounting is now also available when the surface alphabet itself is only bounded as $|\text{surface}| \leq 2^{\text{baseBits}}$. The theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_fallbackBits_add_1`

proves

$$|\text{surface}| \leq \text{fallbackBits} + (\text{radiusBits} + 1) + (\text{baseBits} + 1) \cdot \text{sampleBound}.$$

Its strict-deficit counterpart

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_of`

blocks the endpoint below that bit budget. Thus a repair must pay for both successors in the support-count envelope; neither the sample-radius factor nor the alphabet-base factor can be rounded

down for free. These double-successor routes are now also recorded directly in `CruxSynthesis.lean`, so the ledger keeps the even weaker raw-value surface-cardinality certificate too.

The radius-zero boundary is now formalized as a sharp sanity check. Lean proves

`PluginSampleMajority.smallSubsets_zero` and `PluginSampleMajority.card_smallSubsets_zero`,

so there is exactly one zero-sample sparse override support, namely the empty support. The endpoint theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_fallbackBits_ge_s`

then proves that zero-sample full expressivity forces

$$|\text{surface}| \leq \text{fallbackBits}.$$

Equivalently,

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_zero`

blocks any zero-sample bit-coded fallback route whose fallback code has fewer bits than exact-visible inputs. This is the degenerate case of the Hamming-cover product theorem where the repair has no sampled override budget at all; it recovers the ordinary truth-table code-size barrier inside the structured fallback interface.

The lower bound is also sharp, not merely necessary. Lean now constructs the exact endpoint

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_surjective_zeroSample`

using the canonical full truth-table decoder `exactVisibleRuleDecode`. Given a target exact-visible rule, the surjectivity proof chooses its `exactVisibleRuleEncode` code and the empty bounded sample; the plug-in fallback predictor is then definitionally the decoded truth table. The route-level anchor

`kpolyCoverage_anchor_boundedSampleMajorityWithBitFallbackFamily_surjective_zeroSample_exactVis`

records that the zero-sample structured fallback repair has an exact boundary: below $|\text{surface}|$ fallback bits it cannot be full-rule expressive, and at $|\text{surface}|$ bits it is exactly the unrestricted full truth-table route.

The same observation is now formalized without the radius-zero restriction. Lean proves

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_emptySample_predict`

and

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_surjective_of_fallba`

Thus a structured fallback endpoint is full-rule expressive for every sample bound whenever the fallback family itself is full-rule expressive; the empty sample simply leaves the selected fallback rule unchanged. The bit-coded specialization

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_surjective_exactV`

states this for the canonical full truth-table decoder at $|\text{surface}|$ bits and arbitrary `sampleBound`. Consequently, a bounded-sample fallback repair must prove genuine smallness of the fallback family

itself. Merely bounding the sampled sparse overrides does not matter if the fallback decoder is already the full truth-table family.

Lean now also proves the exact zero-sample converse:

`boundedSamplePluginMajorityWithFallbackFamilyActualSwitchedLocalInterface_zeroSample_surjective`

and its bit-coded specialization

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_zeroSample_surjective`

At radius zero the wrapper contributes no hidden expressivity at all: full-rule expressivity of the endpoint is equivalent to full-rule expressivity of the fallback family or decoder. The route anchors

`kpolyCoverage_anchor_boundedSampleMajorityWithFallbackFamily_zeroSample_surjective_iff_fallback`

and

`kpolyCoverage_anchor_boundedSampleMajorityWithBitFallbackFamily_zeroSample_surjective_iff_falll`

make this available to later synthesis arguments. Lean now packages the full generic structural form in `ActualSwitchedLocalFallbackZeroSampleImage.lean`. For an arbitrary fallback family, the theorems

`FallbackZeroSampleImage.finitePredictorCover_iff_fallbackFamily, FallbackZeroSampleImage.finiteI`

`FallbackZeroSampleImage.finitePredictorQuotient_iff_fallbackFamily, FallbackZeroSampleImage.exac`

and

`FallbackZeroSampleImage.exists_clockedExactVisibleRealization_iff_fallbackFamily`

show that the radius-zero wrapper contributes no hidden image or compression content at all: every finite-cover, representative-cover, finite-quotient, exact-visible-compression, or clocked-realization claim is exactly the corresponding claim for the raw fallback family itself. Combining the same compression equivalence with

`kpolyCoverage_anchor_clockedFiniteLearningPayload_iff_exactVisibleCompressionTarget`

now yields the further route anchor

`kpolyCoverage_anchor_boundedSampleMajorityWithFallbackFamily_zeroSample_clockedPayload_iff_fal`

so radius zero preserves clocked finite-learning payload targets just as rigidly as it preserves image and compression targets. In short: at radius zero the structured fallback wrapper is only a reindexing of the fallback family, not a new small-image mechanism. These zero-sample routes are now also recorded directly in `CruxSynthesis.lean`, so the ledger keeps the degenerate no-override boundary rather than leaving it only as theorem-level scaffolding. The smaller extracted module `ActualSwitchedLocalBitFallbackWrapperObstruction.lean` packages this point as a standalone wrapper boundary: at radius zero the fallback wrapper has no hidden compressive effect at all, so any claimed gain must come from genuinely restricting the fallback decoder rather than from the wrapper. Lean now packages the bit-coded specialization in `ActualSwitchedLocalBitFallbackZeroSampleImage.lean`. The theorem

`BitFallbackZeroSampleImage.finitePredictorCover_iff_fallbackDecoderFamily`

proves that the radius-zero wrapper and the raw fallback decoder family have exactly the same predictor image, not merely the same surjectivity boundary. Consequently,

`BitFallbackZeroSampleImage.exactVisibleCompressionTarget_iff_fallbackDecoderFamily`
and

`BitFallbackZeroSampleImage.exists_clockedExactVisibleRealization_iff_fallbackDecoderFamily`

reduce every zero-sample small-image or clocked-realization claim to the corresponding claim for the fallback decoder family itself. Thus radius zero adds no hidden compression content at all; it is only a reindexing of the same fallback image. Combining the same radius-zero compression equivalence with

`kpolyCoverage_anchor_clockedFiniteLearningPayload_iff_exactVisibleCompressionTarget`
now yields the further route anchor

`kpolyCoverage_anchor_boundedSampleMajorityWithBitFallbackFamily_zeroSample_clockedPayload_iff_`

so radius zero preserves clocked finite-learning payload targets just as rigidly as it preserves finite-cover, quotient, exact-visible-compression, and clocked-realization targets. This structural radius-zero packet is now also promoted into `CruxSynthesis.lean`: the synthesis ledger records the finite-cover, representative-cover, finite-quotient, exact-visible-compression, and clocked-realization equivalence entries, together with the clocked-payload boundary entry, rather than keeping them only in the extracted zero-sample image module. Lean also now packages the complementary exact-decoder obstruction at the same layer. The theorem

`kpolyCoverage_anchor_boundedSampleMajorityWithBitFallbackFamily_not_clockedPayload_of_exactVis`

shows that any exact visible-rule decoder for the bit-fallback family already inherits the standard below-surface cardinality obstruction to clocked finite-learning payloads, and its radius-zero specialization

`kpolyCoverage_anchor_boundedSampleMajorityWithBitFallbackFamily_zeroSample_exactVisibleRuleDeco`

records that the degenerate wrapper does not evade this obstruction either. The follow-on background module `ActualSwitchedLocalBitFallbackZeroSampleProbeObstruction.lean` extends this from full-image equivalence to finite probe transport. Its theorems

`BitFallbackZeroSampleProbeObstruction.finitePredictorCover_card_le_of_fallback_injective_realiz`

and

`BitFallbackZeroSampleProbeObstruction.not_exactVisibleCompressionTarget_of_fallback_injective_`

(together with the representative-cover, quotient, and clocked variants) show that any injectively indexed finite probe family already realized by the fallback decoder survives unchanged through the radius-zero wrapper. This is still background rather than a decisive crux theorem: one must separately prove that the fallback decoder family itself realizes a probe family large enough to contradict the claimed budget, so the transport result alone does not rule out every legal strengthening of the route.

The radius-one boundary is now formalized as the first nontrivial sparse override count. Lean proves

`PluginSampleMajority.smallSubsets_one` and `PluginSampleMajority.card_smallSubsets_one`,

so the allowed supports are exactly the empty support and one singleton support for each exact-visible input, hence

$$|\text{smallSubsets}(\text{surface}, 1)| = |\text{surface}| + 1.$$

The endpoint theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_boolClassifierSpa`

therefore specializes full expressivity at one sample to

$$2^{|\text{surface}|} \leq 2^{\text{fallbackBits}} \cdot (|\text{surface}| + 1).$$

Conversely,

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_on`

blocks the route under the strict opposite inequality. Thus the cheapest nonzero sampled override budget buys only a linear multiplicative factor over the fallback code family; it cannot substitute for the missing exponential truth-table bits.

This one-sample boundary now also has the sharp additive bit-budget form. If the route proves the exact successor envelope

$$|\text{surface}| + 1 \leq 2^{\text{baseBits}},$$

then

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_fallbackBits_add_l`

proves

$$|\text{surface}| \leq \text{fallbackBits} + \text{baseBits}.$$

The matching strict-deficit theorem

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_on`

blocks full-rule expressivity below that budget. If instead the route states only the raw manuscript-style bound

$$|\text{surface}| \leq 2^{\text{baseBits}},$$

Lean derives the successor cost explicitly:

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_fallbackBits_add_l`

proves

$$|\text{surface}| \leq \text{fallbackBits} + (\text{baseBits} + 1),$$

and

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_on`

is the corresponding negative. This closes a small but important escape hatch: the one-sample case should use the exact linear support count, but even there the successor bit on $|\text{surface}| + 1$ cannot be rounded away. These one-sample routes are now also recorded directly in `CruxSynthesis.lean`, so the ledger keeps both the exact linear successor-count certificate and the weaker raw visible-size variant.

The next sparse radius is also now exact. Lean proves

`PluginSampleMajority.card_smallSubsets_two,`

namely

$$|\text{smallSubsets}(\text{surface}, 2)| = 1 + |\text{surface}| + \binom{|\text{surface}|}{2}.$$

Therefore two sampled overrides give only the empty support, singleton supports, and unordered pairs. The endpoint

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_boolClassifierSpa`

specializes surjectivity to the concrete quadratic product bound

$$2^{|\text{surface}|} \leq 2^{\text{fallbackBits}} \cdot \left(1 + |\text{surface}| + \binom{|\text{surface}|}{2} \right),$$

with strict negative

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_tw`

Lean also records the corresponding additive budget certificate: if the exact quadratic envelope fits into `overheadBits` bits, then

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_fallbackBits_add_`

proves

$$|\text{surface}| \leq \text{fallbackBits} + \text{overheadBits},$$

and

`boundedSamplePluginMajorityWithBitFallbackFamilyActualSwitchedLocalInterface_not_surjective_tw`

is the strict-deficit form. Thus allowing two local sampled repairs does not change the nature of the obstruction: a quadratic support multiplier is still far from a free truth-table oracle. These exact one- and two-sample support- count routes are now also recorded directly in `CruxSynthesis.lean`, so the ledger remembers both the concrete product certificates and the exact quadratic-envelope bit-budget form.

The canonical exact $(\text{zfeat}(z), a, b)$ route now records this finite-image burden on the constructive side as well. In `ExactZABTargetInterface.lean`, `ExactZABERMRoute.lean`, `CanonicalZABCandidateInterface.lean`, `CanonicalZABCodeWitness.lean`, `CanonicalZABRecoveryInterface.lean`, and `CanonicalZABERMInterface.lean`, an explicit fixed-order decision-list code assignment, or equality to the corresponding ERM-selected wrapper, yields all three finite-image objects: a predictor cover, representative-index cover, and quotient-code presentation of size at most 2^{r+2k+1} . The identity and projected bit-vector ERM specializations expose the same objects. `CruxSynthesis.lean` now packages this at the synthesis level as `CanonicalZABERMRouteCoverage`: the final manuscript-facing exact ERM route already

sits on the stop-grade exact-visible / finite-image / clocked-payload surface. This is a positive route for the actual canonical family, not a product-success shortcut: the manuscript must still prove that its switched predictors are the family realized by such shared-extractor codes or by the named ERM wrapper. The new module `ExactZABExtractorCollisionObstruction.lean` closes a structural branch of that same route. If the shared extractor $\text{zfeat} : Z \rightarrow \{0,1\}^r$ is noninjective, then every realized exact $(\text{zfeat}(z), a, b)$ decision-list predictor is constant on the fibers of `exactZABVisibleData`. Lean packages this first at the raw realized family level via

`not_surjective_predict_of_not_injective_of_realizedByRawExactZABDecisionListFamily`,
and then lifts it to the named route surfaces

`ExactZABDecisionListTargetData.not_surjective_predict_of_not_injective_zfeat`,
`CanonicalZABDecisionListCandidateData.not_surjective_predict_of_not_injective_zfeat`,
`exactZABDecisionListERMFfamily_not_surjective_of_not_injective_zfeat`,
and the corresponding recovery packages. When Z is finite, the pigeonhole corollary

`not_injective_of_card_gt_bitVec`

shows that every extractor with $|Z| > 2^r$ lands in this forbidden branch. This is genuine crux progress for the named exact $z+a+b$ decision-list route: on the noninjective-extractor branch, the route assumptions do not merely fail to imply compression, they prevent surjectivity on the full exact rule class altogether. What remains live is only the different branch that first proves the extractor is injective enough for the actual manuscript surface. The new module `ExactZABVisibleCollisionObstruction.lean` shows that this was not an accident of the raw decision-list combiner. Lean now proves that every exact route whose predictors are still functions only of the visible summary $(\text{zfeat}(z), a, b)$ inherits the same collision blocker. On the non-shared side this closes the affine-feature, sparse-threshold, and affine-decision-list route surfaces via

`not_surjective_predict_of_not_injective_of_realizedByExactZABAffineFeatureFamily`,
`not_surjective_predict_of_not_injective_of_realizedByExactZABSparseThresholdAffineFamily`,
and

`not_surjective_predict_of_not_injective_of_realizedByExactZABAffineDecisionListFamily`.

On the shared-basis side it closes the corresponding affine-feature, sparse-threshold, and shared decision-list surfaces via

`not_surjective_predict_of_not_injective_of_realizedBySharedExactZABAffineFeatureFamily`,
`not_surjective_predict_of_not_injective_of_realizedBySharedExactZABSparseThresholdAffineFamily`,
and

`not_surjective_predict_of_not_injective_of_realizedBySharedExactZABAffineDecisionListFamily`,
with the ERM-facing wrapper

`sharedExactZABAffineDecisionListERMFfamily_not_surjective_of_not_injective_zfeat`

available directly. So the noninjective-extractor branch is now closed across the broader exact ($\text{zfeat}(z), a, b$) route portfolio, not only for the original raw decision-list presentation. The new module `ExactZABWidthCollisionObstruction.lean` specializes this further to the manuscript-shaped bitvector branch. If $\text{zfeat} : \{0, 1\}^n \rightarrow \{0, 1\}^r$ satisfies $r < n$, then Lean first proves

`not_injective_bitVecExtractor_of_width_lt,`

so the extractor is automatically in the forbidden collision branch before any downstream affine or decision-list reasoning begins. It then lifts that fact to the ERM-facing exact route surfaces

`exactZABDecisionListERMFamilynot_surjective_of_width_lt,`

`exactZABAffineFeatureERMFamilynot_surjective_of_width_lt,`

`exactZABSparseThresholdAffineERMFamilynot_surjective_of_width_lt,`

and on the shared-basis side

`sharedExactZABAffineFeatureERMFamilynot_surjective_of_width_lt,`

`sharedExactZABSparseThresholdAffineERMFamilynot_surjective_of_width_lt,`

`sharedExactZABAffineDecisionListERMFamilynot_surjective_of_width_lt.`

So for bitvector latent data the only live exact-ZAB branch is no longer “perhaps the extractor is merely lossy but still usable”; the extractor must at least avoid width compression outright. The new module `ExactZABInjectiveBranchObstruction.lean` then turns to the remaining live branch: injective extractors with $r \geq n$. It does not claim that every such branch is impossible, but it proves that surjectivity on the current exact/shared exact ERM route surfaces already forces large route parameters. For the raw exact decision-list ERM family,

`exactZABDecisionListERMFamilyn_extractorWidthLowerBound_of_surjective_predict`

shows that surjectivity implies

$$r \geq 2^{n+2k} - (2k + 1),$$

with the route-facing negation exposed as

`exactZABDecisionListERMFamilynot_surjective_of_extractorWidth_lt_truthTableGap.`

On the shared-basis side,

`sharedExactZABAffineFeatureERMFamilyn_featureCountLowerBound_of_surjective_predict`

forces at least $n + 2k$ shared affine features, and

`sharedExactZABAffineDecisionListERMFamilyn_combinerWidthLowerBound_of_surjective_predict`

forces

$$p \geq 2^{n+2k} - 1.$$

So the current injective-extractor escape hatch is no longer “keep zfeat injective but make the downstream witness small” on these ERM surfaces. Any surviving repair would have to change the route class or accept these large parameter requirements. The new module `ActualSwitchedLocalZABDecisionListWidth`

lifts the same live-branch arithmetic to the manuscript-facing local-rule interface `ActualSwitchedLocalInterface.Z`. Abstractly,

`ZABDecisionListData.visibleCardGapLowerBound_of_surjective_predict`

forces any surjective actual-local ZAB route to pay extractor width at least

$$|\text{ExactVisiblePostSwitchSurface } Z \ k| - (2k + 1).$$

On the n -bit input space $Z = \{0, 1\}^n$, the concrete specialization

`ZABDecisionListData.extractorWidthLowerBound_of_surjective_predict`

again yields

$$r \geq 2^{n+2k} - (2k + 1),$$

and

`not_nonempty_zabDecisionListData_of_surjective_predict_of_extractorWidth_lt_truthTableGap`

says there is no actual-local ZAB-data package below that threshold. So the “injective but still small” obstruction is now recorded directly on the manuscript’s local-rule data surface, not only on the ERM wrapper. The follow-up module `SameWidthZABObstruction.lean` closes the smallest remaining injective repair inside that branch. If the extractor stays same-width, $\text{zfeat} : \{0, 1\}^n \rightarrow \{0, 1\}^n$, then once the visible width $n + 2k$ is at least 2, Lean proves

`exactZABDecisionListERMFFamily_not_surjective_of_sameWidth`

for the exact ERM surface and

`ActualSwitchedLocalInterface.not_nonempty_zabDecisionListData_of_surjective_predict_of_sameWidth`

for the manuscript-facing actual-local surface. The concrete full-rule counterexample appears as

`ActualSwitchedLocalInterface.fullRuleActualSwitchedLocalInterface_not_zabDecisionListData_of_s`

So the live injective exact-ZAB branch no longer includes “same width but injective”: any surviving repair must widen zfeat strictly beyond the original n visible bits, not merely prove injectivity. The next route-facing refinement is `ExactZABInjectiveERMRouteCrux.lean`. It converts the widened-branch lower bounds into direct contradictions on the named ERM target surface `fullExactVisibleRuleFamily`. Below the verified thresholds, there is no choice of training samples that makes the raw exact decision-list ERM family, the shared exact affine-feature ERM family, or the shared exact affine decision-list ERM family equal the full exact-visible Boolean rule class. The three Lean statements are

`not_exists_exactZABDecisionListERMFFamily_eq_fullExactVisibleRuleFamily_of_extractorWidth_lt_tr`

`not_exists_sharedExactZABAffineFeatureERMFFamily_eq_fullExactVisibleRuleFamily_of_featureCount_1`

and

`not_exists_sharedExactZABAffineDecisionListERMFFamily_eq_fullExactVisibleRuleFamily_of_combiner`

So the widened injective branch is no longer only a parameter lower-bound warning: on the present ERM route surface, small widened injective extractors and small downstream combiners cannot possibly realize the manuscript’s full exact-visible target family at all. The next strengthening is `ExactZABFullRuleObstruction.lean`. It removes the ERM/sample-choice wrapper entirely. Below the same verified thresholds, Lean proves that the underlying hypothesis classes themselves do not realize the full exact-visible Boolean family:

```
fullExactVisibleRuleFamily_not_realizedByRawExactZABDecisionListFamily_of_extractorWidth_lt_tr
fullExactVisibleRuleFamily_not_realizedBySharedExactZABAffineFeatureFamily_of_featureCount_lt_v
fullExactVisibleRuleFamily_not_realizedBySharedExactZABSparseThresholdAffineFamily_of_linearBu
and
```

```
fullExactVisibleRuleFamily_not_realizedBySharedExactZABAffineDecisionListFamily_of_combinerWid
```

So this repair branch is no longer merely blocked on ERM target selection: below those budgets, changing the sample set or retuning the ERM wrapper inside the same exact-ZAB classes cannot help, because the classes themselves miss the full target family. What remains live is only the genuinely large-parameter branch with widened extractors or downstream budgets beyond these thresholds. The next exact-ZAB strengthening is `ExactZABDecisionListStructuralObstruction.lean`. This one is not a cardinality argument at all. For the raw exact $(\text{zfeat}(z), a, b)$ decision-list class, Lean now proves that every positive-width branch already fails structurally:

```
not_realizedByRawExactZABDecisionListFamily_of_surjective_predict_of_pos_widths.
```

The proof splits on the shared extractor output. If some latent state z_0 already activates a zfeat bit before any a - or b -bit is read, then every predictor in the class is constant on the entire (a, b) -fiber above z_0 . If instead every $\text{zfeat}(z)$ is zero, then the class ignores z entirely. Either way, once $n > 0$ and $k > 0$, the route cannot realize a surjective exact family, so in particular

```
fullExactVisibleRuleFamily_not_realizedByRawExactZABDecisionListFamily_of_pos_widths
```

and the manuscript-facing actual-local wrapper is impossible:

```
ActualSwitchedLocalInterface.fullRuleActualSwitchedLocalInterface_not_zabDecisionListData_of_p
```

This closes the raw exact decision-list branch completely for positive latent and visible widths, independently of the extractor width r . The still-live exact-ZAB repair branches are therefore only the richer shared affine-feature, sparse-threshold, and shared affine decision-list classes. The next exact-ZAB route analysis, `SharedExactZABAffineFeatureCharacterization.lean`, does not close one of those remaining branches, but it identifies its exact mathematical content. Lean now proves that the shared exact affine-feature class realizes the full exact-visible Boolean rule family if and only if the shared affine summary

$$u \mapsto \text{exactZABAffineFeatureSummary}(\text{zfeat}, \text{features}, u)$$

is injective on the exact visible surface. The key statements are

```
injective_summary_of_surjective_predict_of_realizedBySharedExactZABAffineFeatureFamily,
```

`fullExactVisibleRuleFamily_realizedBySharedExactZABAffineFeatureFamily_of_injective_summary`,
and

`fullExactVisibleRuleFamily_realizedBySharedExactZABAffineFeatureFamily_iff_injective_summary`.

So this branch is not a small-class theorem by itself. Once the shared summary collides, surjectivity fails immediately; but if the summary is injective, the arbitrary truth-table combiner recovers every exact-visible Boolean rule. In other words, the route reduces exactly to an injective codebook condition on the shared affine summary, not to a standalone compression argument. The follow-on file `SharedExactZABAffineFeatureSharpness.lean` shows that this injective-summary condition is genuinely satisfiable at the lower-bound threshold on the native bitvector surface. For $Z = \text{BitVec } n$, $\text{zfeat} = \text{id}$, and $\text{features} = \text{canonicalAffineBasis}(n + 2k)$, Lean proves that

$\text{exactZABAffineFeatureSummary}(\text{id}, \text{canonicalAffineBasis}(n + 2k), u) = \text{bitVecZABVisibleData}(u)$,

so the shared summary is injective and therefore

`fullExactVisibleRuleFamily_realizedBySharedExactZABAffineFeatureFamily_id_canonical`.

Hence the previously verified obstruction $p < n + 2k$ is sharp for this branch: at $p = n + 2k$, the shared exact affine-feature route remains fully expressive. This is diagnostic rather than obstructive. It shows that feature count alone does not block the route once the summary is allowed to be an injective codebook for the full visible datum (z, a, b) . The next branch analysis, `SharedExactZABAffineDecisionListCharacterization.lean`, gives the parallel exact description for the shared affine decision-list route. Here the shared extractor and shared affine basis are fixed, and only the downstream fixed-order decision-list code varies. Lean proves that this branch realizes the full exact-visible Boolean rule family if and only if the first-active signature

$$u \mapsto \text{exactZABAffineDecisionListSignature}(\text{zfeat}, \text{features}, u)$$

is injective on the exact visible surface. The key statements are

`injective_signature_of_surjective_predict_of_realizedBySharedExactZABAffineDecisionListFamily`,

`fullExactVisibleRuleFamily_realizedBySharedExactZABAffineDecisionListFamily_of_injective_signature`

and

`fullExactVisibleRuleFamily_realizedBySharedExactZABAffineDecisionListFamily_iff_injective_signature`.

So this branch is not blocked by a small-width argument alone. It reduces exactly to an injective first-active codebook condition on the shared summary. Any future crux for this branch has to attack that injective-signature repair route directly, not merely the decision-list wrapper as such. The follow-on file `SharedExactZABAffineDecisionListStructuralObstruction.lean` does exactly that on the native bitvector exact-visible surface. Lean first proves the raw structural theorem

`not_injective_firstActiveFeature_affineFeatureVector_of_one_lt` :

for any affine feature family on $\text{BitVec } d$, once $d > 1$, the first-active signature

$$x \mapsto \text{firstActiveFeature?}(\text{affineFeatureVector}(\text{features}, x))$$

is never injective. The reason is structural rather than cardinal: either all features are empty, so the signature is constantly `none`, or the earliest nonempty affine parity test is true at at least two distinct points, so that earliest signature class already collides. Pushed back through the visible-data bijection `bitVecZABVisibleData`, this yields

`not_injective_exactZABAffineDecisionListSignature_id_of_one_lt`

for $Z = \text{BitVec } n$, $\text{zfeat} = \text{id}$, and arbitrary shared affine bases on the full visible width $n + 2k$. Hence Lean also proves

`fullExactVisibleRuleFamily_not_realizedBySharedExactZABAffineDecisionListFamily_id_of_one_lt.`

This is genuine crux work for the identity-extractor shared affine decision-list route: once $n + 2k > 1$, no choice of shared affine basis and no decision-list width p can recover the full exact-visible Boolean rule family. What remains live is only the genuinely widened injective-extractor branch, where (z, a, b) is first embedded into a larger ambient bitspace before the shared affine basis is applied. The next file `SharedExactZABAffineDecisionListFiberObstruction.lean` closes that remaining extractor escape hatch for every nonempty latent type with positive visible width. Lean proves

`exactZABAffineDecisionListSignature_fiberPoint,`

showing that after freezing one latent point $z = z_0$, the shared exact affine decision-list signature is exactly the raw first-active affine signature on the $2k$ visible suffix bits (a, b) . Since $k > 0$ implies $2k > 1$, the earlier raw theorem applies on every such fiber, yielding

`not_injective_exactZABAffineDecisionListSignature_of_pos`

for arbitrary extractors $\text{zfeat} : Z \rightarrow \text{BitVec } r$ and arbitrary shared affine bases. Consequently Lean also proves

`fullExactVisibleRuleFamily_not_realizedBySharedExactZABAffineDecisionListFamily_of_pos.`

The route-facing wrappers

`SharedExactZABDecisionListTargetData.not_surjective_predict_of_pos`

and

`SharedExactZABDecisionListRecoveryData.not_surjective_predict_of_pos`

push the same obstruction onto the exact target and recovery interfaces. So the shared exact affine decision-list branch is now closed on every nonempty exact surface with $k > 0$, independently of extractor width r , decision-list width p , or the choice of shared extractor. The recovery side has a separate live quantitative premise. Lean now tests it directly in `BadCodeAgreementObstruction.lean`: if a bad code agrees with the target on every positive-mass input, its agreement mass is exactly 1. Hence any strict $q < 1$ recovery package forces every bad code to disagree with the target somewhere on positive sampling mass:

`BitEncodedClassifierFamily.exists_pos_mass_disagreement_of_badCode_agreementBound_lt_one.`

The exact ZAB recovery interface exposes the same demand as

`ExactZABERMRecoveryData.exists_pos_mass_disagreement_of_badCode.`

For an atomic distribution δ_x , this specializes to a sharper empty-package obstruction: a bad code agreeing with the target at x rules out strict recovery:

`not_ExactZABERMRecoveryData_of_pure_badCode_agrees,`

`not_CanonicalZABERMRecoveryData_of_pure_badCode_agrees.`

The underlying PMF bridge is now explicit in `FiniteIIDAgreement.lean`. Lean proves that agreement mass is the PMF outer measure of the agreement set, so agreement mass 1 forces agreement on the entire positive-mass support:

`agrees_on_support_of_agreementMass_eq_one.`

Conversely, a single positive-mass disagreement gives the strict quantitative boundary

`agreementMass_lt_one_of_pos_mass_disagreement.`

The latest strengthening fills the gap between pure atoms and full support. If a bad code agrees with the target on a finite region R , then the advertised bad-code threshold must be at least the R -mass:

`finset_mass_le_agreementMass_of_agrees_on,      BitEncodedClassifierFamily.regionMass_le_of_badCodeAgreesOn,`

Thus any region with mass above q refutes a uniform agreement bound by q :

`BitEncodedClassifierFamily.not_badCodeAgreementBound_of_agrees_on_region_gt.`

The exact ZAB ERM interface exposes the same quantitative obligation:

`ExactZABERMRecoveryData.regionMass_le_of_badCode_agrees_on,      not_ExactZABERMRecoveryData_of_b`

The current Lean strengthening also localizes the contrapositive. A uniform bad-code threshold q forces every finite region of mass above q to contain a positive-mass disagreement point for each bad code:

`BitEncodedClassifierFamily.exists_pos_mass_disagreement_in_region_of_badCodeAgreementBound_lt_m`

For exact ZAB recovery this becomes

`ExactZABERMRecoveryData.exists_pos_mass_disagreement_in_region_of_badCode.`

So a repair cannot claim a small global bad-code threshold while allowing even a moderately large all-agreement region for one non-target code under the sampling PMF. It must provide localized disagreement, on positive sampling mass, in every region whose actual mass exceeds the advertised threshold. The sample-level form is now explicit as well. In `FiniteIIDAgreement.lean`, an agreement region of mass p contributes at least p^m to the consistent-sample mass:

`regionMass_pow_le_consistentSampleMass_of_agrees_on.`

In `FiniteIIDFamilyBound.lean`, the samples consistent with any fixed bad code form a subevent of the deceptive samples:

`EncodedFamily.consistentSampleMass_le_deceptiveSampleMass_of_badCode,`

and hence

`EncodedFamily.regionMass_pow_le_deceptiveSampleMass_of_badCode_agrees_on.`

The exact ZAB specialization

`exactZAB_regionMass_pow_le_deceptiveSampleMass_of_badCode_agrees_on`

turns every large agreeing region for one bad code into a concrete positive deceptive-sample event under IID sampling. This does not depend on a finite-probe or full-surface counting argument; it is a direct sample-distribution burden. This turns support witnesses into recovery-bound obstructions without adding a new finite-image or probe assumption. Equivalently, a pure-atom agreement bound with $q < 1$ forces every code that matches the target at the atom to be globally equal to the target predictor. This means the finite-image ERM wrappers do not close the route by themselves: the manuscript must prove a support/disagreement theorem for the actual switched bad codes and the actual sampling measure. The synthesis ledger now records this as route coverage rather than leaving it only in the local obstruction file. In `CruxSynthesis.lean`, the `PNPKpolySubrepairClass` list includes product-bound and fielded-switching finite-image boundary entries, the corresponding clocked-payload boundary entries, the large-region bad-code disagreement boundary, and the full-visible and already-surjective Boolean-family obstructions. These are deliberately narrow anchors: they show that the product/fielded shortcut has been exhausted at both the compression and payload interfaces, while the recovery side now has an explicit positive-mass disagreement obligation under the actual sampling measure. The broad `kpolyCompressionBridge` no longer remains an uncovered broad escape class in the synthesis ledger: the later promoted packet packages these finite-image and counterexample anchors into a theorem-backed blocker for the entire current broad bridge route. What still does *not* happen here is an affirmative small-class theorem for the actual switched predictor family. The newest strengthening removes an avoidable presentation dependency from that obstruction. The counterexample need not be exactly `fullExactVisibleRuleFamily`; any exact-visible family whose prediction map is already surjective onto all Boolean rules on the exact surface refutes the product-bound-only and fielded-switching-only bridge schemas below the surface threshold:

`not_productBoundOnlyExactVisibleCompressionBridge_of_true_productBound_of_surjective_predict_of`

`not_fieldedSwitchingOnlyExactVisibleCompressionBridge_of_true_fieldedSwitching_of_surjective_p`

At the finite-image boundary, the same point is recorded as

`not_productBoundSemanticFiniteImageBridge_of_true_productBound_of_surjective_predict_of_lt_pre`

and its fielded-switching analogue. Thus a repair cannot evade the blocker by renaming the index type or by presenting the same fully expressive family through a different exact-visible parameterization.

The new module `ABVisibleCompressionObstruction.lean` closes the most charitable reduced-surface workaround for that burden. Suppose the exact switched family on $u = (z, a, b)$ actually factors through the raw visible projection (a, b) . Lean now proves that any exact visible compression target, or any clocked s -bit realization family for the exact surface, pulls back to the same s -bit budget on the reduced raw family over `ABVisibleSurface` k . Hence, if that reduced family is still surjective onto all Boolean rules on (a, b) , then no such theorem can hold below the reduced truth-table threshold

$$s < |\text{ABVisibleSurface } k| = 2^{2k}.$$

So “the switched decoder ignores z ” is not yet a same-route rescue. It still leaves a separate strict-subclass theorem to prove on the raw (a, b) -surface itself. The same module also now propagates this reduced truth-table lower bound through the charitable invariant-view and fork-view compression interfaces. So replacing the exact surface by one of those intermediate exact views still does not evade the raw (a, b) counting burden.

The module `ABDecisionListObstruction.lean` now turns that last sentence into a concrete strict-subclass theorem for the first raw (a, b) candidate class. The fixed-order raw decision-list decoder has only

$$2k + 1$$

program bits, while the full Boolean rule class on (a, b) needs

$$|\text{ABVisibleSurface } k| = 2^{2k}$$

truth-table bits. Lean proves the sharp positive-width inequality

$$2k + 1 < 2^{2k} \quad (k > 0).$$

Therefore a reduced family that is still surjective onto all Boolean rules on (a, b) cannot be realized by the fixed-order raw decision-list route. The same module lifts the result back to the exact surface: if an exact switched family factors through (a, b) and the reduced family is fully expressive, then it cannot be realized by the raw exact-surface decision-list class. This does not close the full *Kpoly* bridge, but it removes one concrete repair subclass from the live ledger: the author must prove that the actual reduced switched family is much smaller than all raw Boolean rules, not merely that it uses the raw (a, b) coordinates. The new module `ABDecisionListRouteCrux.lean` pushes this one step closer to the named positive route interface. Its theorem

`not_factorsThrough_ab_and_decisionList_of_surjective_predict_of_pos`

combines the positive route certificate from `ABDecisionListRoute.lean` with the reduced truth-table lower bound and shows that, for every positive width, the route assumptions themselves are inconsistent on any branch where the reduced (a, b) family remains fully expressive. The invariant-lift form

`not_invariant_and_decisionList_of_surjective_lift_of_pos`

packages the same contradiction directly at the lifted exact-surface route, and the concrete corollary

`fullExactABRuleFamily_not_exactVisibleCompressionTarget_of_pos`

shows that the canonical full pullback of all raw (a, b) rules cannot meet the $2k+1$ decision-list budget. This is real crux progress for that named repair surface: on its full-expressivity branch the route assumptions no longer merely fail to imply compression, they contradict it. The synthesis ledger now records these as narrow `PNPKpolySubrepairClass` coverage items alongside the clock-wrapper equivalence, the finite-image budget/transport lemmas, the product/fielded semantic-boundary and surjective-family anchors, and the reduced (a, b) finite-image and quotient-code pullback obstructions. Before the final broad promotion these sat underneath an uncovered `kpolyCompressionBridge` banner; now they feed directly into the theorem-backed `KpolyCompressionBridgeCoverage` packet. The latest bounded-sample fallback update extends that same narrow ledger with explicit entries for the canonical `exactVisibleRuleDecode` endpoint: one saying the endpoint is already full-rule expressive for every sample bound, and one saying any sparse-threshold recovery witness on it must satisfy the joint visible-width and lightest-point barrier. The next refinement lifts that

one level higher: the same ledger now records the generic surjective-actual-local joint recovery barrier, the generic surjective-actual-local visible-width threshold, the generic surjective-actual-local lightest-point threshold, and the corresponding “no extractor below threshold” barrier. The current refinement keeps the negative visible-width form as well via the unconditional surjective-actual-local half-width no-extractor boundary, so the exact-decoder fallback entries are explicitly exposed as specializations of a broader route-level obstruction rather than as isolated endpoint facts.

The next reduced-surface module, `SharedABFeatureRoutes.lean`, pushes this from abstract counting to the concrete shared-basis families already used by the route. On the raw (a, b) -surface it now defines the corresponding bit families and proves the direct lower bounds:

- a shared affine-feature realization forces $|\text{ABVisibleSurface } k| \leq 2^r$,
- a shared sparse-threshold realization forces $|\text{ABVisibleSurface } k| \leq 2r$,
- a shared decision-list realization forces $|\text{ABVisibleSurface } k| \leq r + 1$.

Equivalently, if the lifted reduced family on (a, b) is still fully expressive, then none of these shared raw classes can realize it below the same reduced truth-table threshold. So the remaining manuscript-side burden is not merely to state invariance and choose one shared basis; it is to prove that the resulting reduced family is genuinely a strict subclass of all Boolean rules on (a, b) .

The companion module `SharedABFeatureObstruction.lean` packages the same fact as exact-pullback refuters for the shared-basis route. If the exact switched family factors through (a, b) , the reduced raw family is surjective, and the shared combiner budget is below 2^{2k} , then the exact family cannot be realized by the corresponding shared exact class. Lean proves this separately for:

$$2^r, \quad 2r, \quad r + 1$$

shared affine-feature, sparse-threshold, and decision-list budgets. The synthesis ledger records these as additional narrow `PNPKpolySubrepairClass` items. Standing alone, these theorems only rule out full reduced-rule expressivity for named shared classes; they do not prove a manuscript-faithful small-class theorem for the actual switched family. But the broad synthesis ledger no longer leaves `kpolyCompressionBridge` uncovered: those narrow shared-class blockers are now explicitly absorbed into the promoted `KpolyCompressionBridgeCoverage` route blocker. The same shared-feature blocker now has the finite-probe form needed after the new image-budget foundation. Full reduced-rule surjectivity is unnecessary: if the reduced family already realizes an injectively indexed finite probe family P , then a shared affine-feature realization forces

$$|P| \leq 2^{2^r},$$

a shared sparse-threshold realization forces

$$|P| \leq 2^{2r},$$

and a shared decision-list realization forces

$$|P| \leq 2^{r+1}.$$

The exact-pullback versions have the same bounds whenever the exact visible family factors through `abVisibleData`. Lean records the refuters as

`not_realizedBySharedExactABAffineFeatureFamily_of_factorsThrough_ab_and_injective_probe_of_lt_1`

and the matching sparse-threshold and decision-list variants. Thus a shared-basis repair cannot merely avoid the full Boolean rule family; it must prove that the actual reduced predictor image avoids every finite probe family larger than the named combiner image. The new module `SharedABFeatureRouteCrux.lean` pushes this from “named class obstruction” to “named route contradiction.” It combines the positive route interfaces from `SharedABFeatureRoutes.lean` with the reduced truth-table lower bound and proves that, whenever the reduced (a, b) family remains fully expressive, the route assumptions themselves are inconsistent below the corresponding combiner budgets. Lean now records three factored contradictions,

`not_factorsThrough_ab_and_sharedAffineFeature_of_surjective_predict_of_lt_cardFormula,`

with the matching sparse-threshold and decision-list variants, and three invariant-lift contradictions for the same named route surfaces. So on the full-expressivity branch these shared-feature repairs no longer merely fail to supply a missing small-class theorem; they directly contradict the exact-visible compression target forced by their own positive route certificates. What remains uncovered is only a different repair shape that first proves the actual reduced family is a strict subclass of all Boolean rules on (a, b) .

The new module `ABVisibleInvariantSurjectivityObstruction.lean` closes a more basic loophole on the exact surface itself. Suppose Z is nontrivial and the switched predictor is `ABVisibleInvariant`, i.e. it depends only on (a, b) and ignores the exact z -coordinate. Then Lean now proves that the resulting predictor cannot be surjective onto all Boolean rules on `ExactVisiblePostSwitchSurface` Z k . The reason is structural: two distinct exact inputs with the same (a, b) -data must receive the same output, so every z -fiber is collapsed before any coding budget enters the picture. This is strictly stronger than the reduced-surface counting obstructions above. It says that an “ignore z ” repair is not merely a compression claim still waiting for a tighter bit bound; it already changes the target rule class on the exact visible surface. The same module propagates this obstruction through the shared affine-feature, sparse-threshold, decision-list, recovery, and canonical AB candidate interfaces, so the route cannot recover by relabeling the same (a, b) -only family under a more charitable exact-surface wrapper. The newest strengthening removes the remaining ambiguity in that diagnosis. It does not only refute surjectivity. Lean now defines

`ABFiberSeparatingRule`

for any exact-surface Boolean rule that distinguishes two inputs with identical (a, b) -data, and proves

`not_exists_predict_eq_of_abVisibleInvariant_of_abFiberSeparatingRule.`

So an (a, b) -invariant switched family cannot select even one such target rule. The concrete hidden-bit target

`boolZProjectionRule` $(z, a, b) = z$

is registered as a separating rule, yielding

`not_exists_predict_eq_boolZProjectionRule_of_abVisibleInvariant.`

Thus any same-route repair that discards z must prove that the target predictors themselves are constant on every z -fiber over fixed (a, b) ; otherwise the repair misses a named exact-surface target, not merely a large classifier class.

The support-level version now rules out a weaker recovery story as well. If a distribution μ assigns positive mass to two exact inputs u, v with the same (a, b) -data and a target T separates

them, then every (a, b) -invariant predictor must disagree with T at some positive-mass input. The generic theorem is

`exists_pos_mass_disagreement_of_abVisibleInvariant_predict_of_abFiberSeparation,`

and the concrete hidden-bit specialization is

`exists_pos_mass_disagreement_of_abVisibleInvariant_predict_boolZProjectionRule.`

Consequently, a repair cannot weaken exact realization to distributional recovery on any proof-relevant measure that keeps both hidden- z sides of one visible fiber in support. It must either show that the target is fiber-constant on the exact surface or prove that the proof distribution removes one side of every target-separating hidden- z fiber. The crux ledger records this as the narrow subrepair

`abVisibleInvariantSupportDisagreementObstruction.`

The newest bridge composes this support fact with the PMF agreement-mass interface. Under the same positive-mass same-fiber hypotheses, every (a, b) -invariant predictor has agreement mass strictly below 1:

`agreementMass_lt_one_of_abVisibleInvariant_predict_of_abFiberSeparation.`

For the hidden-bit target this becomes

`agreementMass_lt_one_of_abVisibleInvariant_predict_boolZProjectionRule.`

The family-level contrapositive is now named as well:

`not_exists_agreementMass_eq_one_of_abVisibleInvariant_of_abFiberSeparation,`

with the hidden-bit specialization

`not_exists_agreementMass_eq_one_boolZProjectionRule_of_abVisibleInvariant.`

Thus an exact (a, b) -only repair cannot be repackaged as perfect distributional recovery by choosing a different member of the same invariant family on any measure supporting both hidden-bit alternatives in one visible fiber.

The new module `SharedExactZABSparseThresholdSharpness.lean` shows that the shared exact sparse-threshold $(z \text{ feat}(z), a, b)$ branch is genuinely live once the shared basis is allowed to be large. On raw `BitVec` d , Lean constructs the full point-block basis `allAffinePointBlockFeatures` d , together with an explicit mask and threshold witness `pointBlockSparseThresholdCode`, and proves

`rawSharedSparseThresholdPredict_allAffinePointBlockFeatures_eq,`

so every Boolean rule on `BitVec` d is realized by one shared sparse-threshold code on that basis. Pushed back through the exact visible-data bijection for identity extractors, this yields

`fullExactVisibleRuleFamily_realizedBySharedExactZABSparseThresholdAffineFamily_id_allAffinePoi`

for $Z = \text{BitVec } n$ whenever $n + 2k > 0$. So the earlier theorem

`fullExactVisibleRuleFamily_not_realizedBySharedExactZABSparseThresholdAffineFamily_of_linearBu`

is only a small-budget obstruction. It does not close the sparse-threshold repair surface outright. The remaining burden on that branch is to prove that the manuscript’s actual switched family lands in a much smaller sparse-threshold subclass than this fully expressive point-block realization.

The follow-up module `SharedExactZABSparseThresholdInjectiveLiveness.lean` removes the identity-extractor restriction. Lean proves

`exactZABVisibleData_injective_of_injective_zfeat`

and then pulls the same point-block witness back along any injective visible map $(z, a, b) \mapsto (zfeat(z), a, b)$, yielding

`fullExactVisibleRuleFamily_realizedBySharedExactZABSparseThresholdAffineFamily_of_injective_zf`

So on the injective side the shared exact sparse-threshold route is not merely alive at the native `BitVec` surface; it stays fully expressive for every injective extractor. Combined with the previously formalized noninjective-extractor collision obstruction, this means extractor-width counting alone cannot kill the sparse-threshold branch. Any remaining crux for that branch must prove that the manuscript’s actual switched family lies in a strictly smaller sparse-threshold subclass than this ambient injective realization.

The next module `SharedExactZABSparseThresholdInjectiveERMLiveness.lean` upgrades this from ambient class expressivity to the manuscript-shaped ERM route itself. Write

$p = \text{allAffinePointBlockFeatureCount}(r+2k), \quad U = \text{ExactVisiblePostSwitchSurface } Z \ k.$

Under the standard finite counting gap

$$2^{2p}(|U| - 1)^m < |U|^m,$$

Lean proves

`exists_samples_sharedExactZABSparseThresholdAffineERMFamily_eq_fullExactVisibleRuleFamily_of_in`

and the extractor-level corollary

`exists_samples_sharedExactZABSparseThresholdAffineERMFamily_eq_fullExactVisibleRuleFamily_of_in`

So whenever the visible summary $(z, a, b) \mapsto (zfeat(z), a, b)$ is injective, there exists one labeled sample for each exact visible rule such that ERM over the shared exact sparse-threshold point-block family returns that rule exactly, and the resulting indexed ERM family is literally equal to `fullExactVisibleRuleFamily`. This rules out a possible repair story in which the branch remained expressive only as a raw hypothesis class but collapsed once the manuscript’s ERM wrapper was imposed. Any remaining obstruction on the sparse-threshold branch must therefore use extra semantic restrictions on the sampled subclass, not merely the passage from class realizability to ERM realizability.

The new consolidation module `SharedExactZABSparseThresholdERMCharacterization.lean` packages the two sides into one route statement. First Lean proves the converse helper

`injective_zfeat_of_injective_exactZABVisibleData,`

so injectivity of the full visible map $(z, a, b) \mapsto (zfeat(z), a, b)$ is equivalent to injectivity of $zfeat$ itself. Then it proves

`exists_samples_sharedExactZABSparseThresholdAffineERMFamily_eq_fullExactVisibleRuleFamily_iff_in`

and the extractor-level form

`exists_samples_sharedExactZABSparseThresholdAffineERMFamily_eq_fullExactVisibleRuleFamily_iff_`

So, under the same point-block width and finite counting gap, the sparse-threshold ERM route is neither partly dead nor partly alive: it is exactly an injective-extractor route. Noninjective extractors are now formally closed at the ERM equality surface, while injective extractors remain fully live. That sharpens the remaining burden again: any further crux must either prove that the manuscript’s actual extractor is noninjective on the exact surface or prove that the manuscript’s switched family does not in fact land in this point-block shared sparse-threshold ERM class.

The manuscript-facing bridge `ActualSwitchedLocalSharedExactZABSparseThresholdERMData.lean` now packages exactly that second obligation. It defines `ActualSwitchedLocalInterface.SharedExactZABSparseT` which says that one actual switched-local predictor family is literally one shared exact sparse-threshold ERM family on the canonical point-block basis. Lean then proves the generic obstruction

`ActualSwitchedLocalInterface.SharedExactZABSparseThresholdERMData.not_surjective_predict_of_no`

and, at the full-rule equality surface,

`ActualSwitchedLocalInterface.injective_zfeat_of_eq_fullExactVisibleRuleFamily.`

So any actual switched-local sparse-threshold ERM realization of the full exact visible rule family already forces an injective extractor. On the concrete `BitVec n` surface, Lean specializes this to

`ActualSwitchedLocalInterface.fullRuleActualSwitchedLocalInterface_not_nonempty_sharedExactZABSP`

which rules out every width-compressing extractor $\text{BitVec } n \rightarrow \text{BitVec } r$ with $r < n$ for this manuscript-facing route. This does not close the injective sparse-threshold branch, but it removes the last ambiguity about what the actual switched-local packet still has to supply: it must either provide an injective extractor or leave the point-block shared sparse-threshold ERM class.

The follow-up characterization file `ActualSwitchedLocalSharedExactZABSparseThresholdERMCharacterizat` adds the missing liveness direction at the same manuscript-facing level. Under the same point-block width and finite counting gap, Lean first proves the generic injective-liveness theorem

`ActualSwitchedLocalInterface.nonempty_sharedExactZABSparseThresholdERMData_of_injective_zfeat,`

which reindexes the full-rule ERM witness samples along the current predictor map. So on the injective branch the manuscript-facing data wrapper is not special to the full-rule counterexample: once *zfeat* is injective, Lean can realize *any* actual switched-local predictor family as one shared exact sparse-threshold ERM family on the canonical point-block basis, provided the same finite counting-gap packet holds. The file then specializes this generic construction to the full-rule endpoint and proves

`ActualSwitchedLocalInterface.fullRuleActualSwitchedLocalInterface_nonempty_sharedExactZABSpars`

and then packages the full equivalences

`ActualSwitchedLocalInterface.fullRuleActualSwitchedLocalInterface_nonempty_sharedExactZABSpars`

and

`ActualSwitchedLocalInterface.fullRuleActualSwitchedLocalInterface_nonempty_sharedExactZABSpars`

So the full-rule actual switched-local sparse-threshold ERM packet is not just partially constrained: at this abstraction level it is exactly an injective-extractor packet. The noninjective side is closed by the previous bridge file, and the injective side is genuinely live even after asking for the manuscript-facing actual-local data wrapper. Any remaining crux on this branch must therefore attack either the extractor itself or the claim that the actual switched family lands in this point-block shared sparse-threshold ERM class; it can no longer come from a gap between ambient ERM realizability and the manuscript's actual-local packaging.

The next file `ActualSwitchedLocalSharedExactZABSparseThresholdERMWidthCharacterization.lean` removes the last width-level ambiguity on the native `BitVec n` surface. Lean defines an explicit low-bit-preserving embedding $\text{BitVec } n \rightarrow \text{BitVec } r$ for every $n \leq r$, proves

`ActualSwitchedLocalInterface.exists_injective_bitVecExtractor_iff_width_le,`

and then packages the manuscript-facing existential route theorem

`ActualSwitchedLocalInterface.fullRuleActualSwitchedLocalInterface_exists_sharedExactZABSparseT`

So, under the same point-block width and finite counting gap as before, the full-rule actual switched-local sparse-threshold ERM packet exists for some width- r extractor exactly when $n \leq r$. This is still not a crux closure: it proves the surviving branch is precisely the non-compressing injective-extractor branch. Any stronger obstruction must therefore use more than width bookkeeping or the generic actual-local sparse-threshold ERM wrapper.

The next two files sharpen the same packet in a different direction. `BitFamilyUniformRecoveryLowerBound.lean` records the generic reverse counting arithmetic: on any finite input domain with at least two points, the finite recovery gap

$$2^s(|X| - 1)^m < |X|^m$$

already forces $s < m$. `ActualSwitchedLocalSharedExactZABSparseThresholdERMSampleLowerBound.lean` then specializes that fact to the point-block sparse-threshold route. Lean proves that the standard point-block gap hypothesis used by the current liveness packet implies

$$m > 2 \cdot \text{allAffinePointBlockFeatureCount}(r + 2k) = 2^{2(r+2k)+1},$$

and therefore, via the earlier width characterization $n \leq r$, any nontrivial full-rule actual switched-local sparse-threshold ERM packet on `BitVec n` forces

$$m > 2^{2(n+2k)+1}.$$

This is still not a crux closure. It is an honest sample-growth consequence of the current finite counting-gap liveness theorem, not an unconditional impossibility theorem for the injective sparse-threshold branch itself.

The next bridge pushes that consequence off the special full-rule counterexample and onto the bounded-sample plug-in-majority actual-local endpoint. First, `ActualSwitchedLocalSharedExactZABSparseThres` factors the previous argument through the reusable theorem

`ActualSwitchedLocalInterface.visibleWidthBudget_lt_sampleSize_of_nonempty_sharedExactZABSparseT`

which applies to any surjective actual switched-local family carrying the same shared exact sparse-threshold ERM data. Then `ActualSwitchedLocalBoundedSampleSharedExactZABSparseThresholdERMSampleBo` combines that with the earlier characterization of bounded sample-level plug-in majority: if the

sample bound already reaches the exact-visible alphabet size, so that the bounded-sample endpoint is full-rule expressive, and if that same endpoint is also identified with one shared exact sparse-threshold ERM family, then the same point-block counting-gap packet at $m = \text{sampleBound}$ forces

$$\text{sampleBound} > 2^{2(n+2k)+1}.$$

Equivalently, under that same gap hypothesis, no such bounded-sample manuscript-facing packet can exist at or below the visible-width budget. This is still conditional route-diagnostic work rather than a closure of the surviving injective sparse-threshold branch: it only rules out bounded-sample actual-local packets that are both full-rule expressive and demanded to satisfy the current finite counting-gap liveness inequality at that same bound.

The latest strengthening removes even the need to assume that a bounded-sample packet witness was supplied ahead of time. Lean now proves

`ActualSwitchedLocalInterface.boundedSamplePluginMajorityActualSwitchedLocalInterface_visibleWi`

Here the only extra route-side hypothesis is the noncompressing width condition $n \leq r$: Lean inserts the explicit low-bit embedding `bitVecPrefixEmbedding : BitVec n → BitVec r`, uses the new generic injective-liveness theorem to manufacture the manuscript-facing sparse-threshold packet automatically on the bounded-sample endpoint, and then re-runs the same sample-growth argument. So once bounded sample plug-in majority is large enough to be full-rule expressive, the surviving injective-width branch itself already forces $\text{sampleBound} > 2^{2(n+2k)+1}$ under the current counting-gap packet; the theorem no longer depends on separately postulating a bounded-sample ERM equality witness.

The next bounded-sample refinement, `ActualSwitchedLocalBoundedSampleSharedExactZABSparseThreshold` repackages the same counting-gap packet into a genuine forbidden-window statement. On `BitVec n`, any bounded-sample plug-in-majority actual-local packet identified with one shared exact sparse-threshold ERM family must satisfy either

$$\text{sampleBound} < 2^{n+2k}$$

or

$$2^{2(n+2k)+1} < \text{sampleBound}.$$

So the whole intermediate regime

$$2^{n+2k} \leq \text{sampleBound} \leq 2^{2(n+2k)+1}$$

is impossible under that same counting-gap hypothesis. Lean also lifts the same obstruction to the manuscript-facing recovery packet through the `.data` field. This remains route-facing obstruction work rather than branch closure: it strengthens the same finite counting-gap consequence into a forbidden bounded-sample window, but does not touch packets outside that interval.

The smaller follow-up file `ActualSwitchedLocalBoundedSampleSharedExactZABSparseThresholdERMRecover` then lifts the earlier large-sample bridge directly to the manuscript-facing recovery endpoint itself. No new counting argument is used: recovery data already contains the underlying exact-family packet, so once the bounded-sample plug-in-majority endpoint is large enough for surjectivity, the same point-block counting-gap hypothesis at $m = \text{sampleBound}$ still forces

$$\text{sampleBound} > 2^{2(n+2k)+1}.$$

Equivalently, no bounded-sample manuscript-facing recovery packet can exist at or below that visible-width budget under the same counting-gap packet. This is not a new closure theorem, but it

closes a manuscript-facing loophole: the large-sample burden now hits the recovery packet directly, not only the underlying data packet or the later forbidden-window repackaging.

The next file, `ActualSwitchedLocalBoundedSampleSharedExactZABSparseThresholdERMVisibleBudgetObs`, adds the unconditional bounded-sample analogue of the later actual-local visible-budget obstruction. Once the bounded sample bound reaches the exact-visible alphabet size, so that the bounded-sample plug-in-majority endpoint is surjective, any shared exact sparse-threshold ERM packet on that endpoint already forces

$$|\text{ExactVisiblePostSwitchSurface}(Z, k)| \leq 2 \cdot \text{allAffinePointBlockFeatureCount}(r + 2k).$$

On `BitVec` n , this again specializes to

$$n \leq 2r + 2k + 1.$$

So if $2r + 2k + 1 < n$, then the bounded-sample manuscript-facing endpoint cannot carry the shared exact sparse-threshold ERM packet at all, provided only that its sample bound is large enough for surjectivity. Unlike the previous bounded-sample sample-growth theorem, this one uses neither the counting-gap liveness hypothesis nor any recovery-side agreement estimate. It is still not a full closure of the injective branch, but it pushes the unconditional half-width obstruction all the way to the bounded-sample endpoint.

The next file, `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryData.lean`, adds the missing manuscript-facing recovery wrapper for this same point-block route. It defines actual-local recovery data to be the earlier exact-family packet together with one uniform bad-code agreement bound. Lean then shows that such data automatically induce the corresponding exact-visible shared sparse-threshold recovery packet, and therefore close all of the standard consequences at the actual-local layer:

- the exact-visible compression target at budget $2 \cdot \text{allAffinePointBlockFeatureCount}(r + 2k)$,
- a finite predictor cover / representative cover / quotient of size $2^{2 \cdot \text{allAffinePointBlockFeatureCount}(r + 2k)}$,
- the bundled clocked K -poly finite-learning payload at that same budget, and
- the weighted exact-recovery lower bound

$$1 - 2^{2 \cdot \text{allAffinePointBlockFeatureCount}(r + 2k)} q^m \leq \text{bitExactRecoverySampleMass}(\mu, \dots, m).$$

The file also inherits the earlier extractor obstructions: noninjective `zfeat`, and in particular width-compressing `BitVec` $n \rightarrow \text{BitVec } r$ maps with $r < n$, make even this stronger recovery packet impossible on surjective actual-local families. This is again route-facing infrastructure rather than a new crux closure. It shows that once the manuscript can honestly supply the bad-code agreement bound, no extra finite-image or clocked-payload bookkeeping remains to be proved for this branch; the remaining burden stays concentrated on the agreement estimate and the injective extractor regime itself.

The next file, `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryAtomObstruction.lean`, turns that recovery wrapper into a genuine obstruction for overly concentrated recovery laws. Suppose the actual switched-local family is surjective on the exact visible surface, the recovery layer carries a bad-code agreement bound q , and there is one visible point x whose sampling mass already exceeds that bound:

$$\mu\{x\} > q.$$

If the exact visible surface also contains some different point $y \neq x$, then Lean now shows that no `ActualSwitchedLocalInterface.SharedExactZABSparseThresholdERMRecoveryData` packet can exist. The proof is direct: surjectivity supplies one local rule that is constantly false and another that spikes exactly at y ; the shared sparse-threshold target-data packet gives a bad code for the former whose decoded predictor agrees with it at x ; and the earlier region-mass obstruction then contradicts the bad-code agreement bound because the region $\{x\}$ already has mass $> q$.

This has an immediate pure-atom corollary. If $\mu = \delta_x$ is a pure point mass and $q < 1$, then every surjective nontrivial visible surface is already impossible for the shared exact sparse-threshold recovery packet. Lean also lifts the same obstruction to the bounded-sample plug-in-majority endpoint and to the full-rule `BitVec` n endpoint whenever $0 < n + 2k$, since that guarantees the exact visible surface contains at least two distinct points. This is real obstruction work, but it is not full branch closure: it kills atomic or too-heavy recovery laws, not the remaining non-atomic injective recovery regime.

The next file, `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryConcentrationObstruction`, sharpens that atom obstruction into a surface-wide concentration bound. Let μ be the recovery law on the exact visible surface, assume the actual switched-local family is surjective, and suppose the visible surface contains two distinct points. Fix any visible point y . Surjectivity again supplies the singleton spike at y and the constant-false target, so the bad-code agreement packet forces the decoded bad code to disagree with the false target on at most a q -mass region. But outside $\{y\}$ the spike is false, so the whole complement already lies inside that agreement region. Lean therefore proves

$$1 - \mu\{y\} \leq q$$

for every visible point y . On a nontrivial visible surface this implies that not all visible points can carry mass $> 1/2$; choosing one with $\mu\{y\} \leq 1/2$ yields the unconditional recovery-side lower bound

$$\frac{1}{2} \leq q.$$

So no surjective actual-local sparse-threshold recovery packet can exist once $q < 1/2$ and the exact visible surface has at least two points. Lean lifts this obstruction to the bounded-sample plug-in-majority endpoint and to the full-rule `BitVec` n endpoint whenever $0 < n + 2k$, since that condition guarantees a nontrivial exact visible surface. The regression file also records the sharp two-point Bool canary: under the uniform recovery law on `ExactVisiblePostSwitchSurface Bool 0`, any full-rule recovery packet must satisfy $q \geq 1/2$, and in particular $q = 0$ is impossible. This is again genuine obstruction work, but still not branch closure: it rules out the entire $q < 1/2$ recovery regime on nontrivial visible surfaces, while the remaining $q \geq 1/2$ injective regime stays open.

The next file, `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryCardinalityObstruction`, upgrades that two-point obstruction to the full finite surface cardinality. The helper file `FinitePMFPointMassBound` proves that every probability mass function on a finite nonempty type has some point y with

$$\mu\{y\} \leq |S|^{-1}.$$

Combining that averaging fact with the earlier surjective recovery inequality $1 - \mu\{y\} \leq q$ yields the sharper universal lower bound

$$1 - |S|^{-1} \leq q$$

for every surjective actual-local sparse-threshold recovery packet on a finite exact visible surface S . Lean lifts that obstruction to the bounded-sample plug-in-majority endpoint and to the full-rule endpoint. On the full-rule `BitVec` n surface this specializes to

$$1 - 2^{-(n+2k)} \leq q.$$

So every q strictly below the uniform-complement threshold is impossible. The regression file records the concrete uniform **BitVec** $1, k = 1$ canary: any full-rule recovery packet must satisfy $q \geq 1 - 2^{-3} = 7/8$, and in particular $q = 0$ is already impossible at both the full-rule and bounded-sample surjective endpoints. This is again real obstruction work, but still not branch closure: it removes the whole sub-uniform-complement recovery regime while leaving the surviving near-perfect injective regime open.

The follow-up file, **ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryThresholdWidthObst**, turns that finite-threshold lower bound around. Suppose some manuscript route claims a stronger recovery estimate

$$q < 1 - 2^{-p}.$$

Lean then proves that the full-rule **BitVec** n endpoint must satisfy

$$n + 2k < p.$$

Equivalently, whenever $p \leq n + 2k$, no such full-rule recovery packet can exist. The same inverted threshold obstruction lifts to the bounded-sample plug-in-majority endpoint once the sample bound is large enough for surjectivity. So any concrete recovery estimate below the finite uniform-complement floor automatically becomes a direct visible-width blocker. The regression file records the uniform **BitVec** $1, k = 1$ canary in both directions: hypothetical $q = 1/2$ data already forces $1 + 2 < 4$, and setting $p = 3$ rules out both the full-rule and bounded-sample endpoints outright. This is again real obstruction work, though still not branch closure: it converts subthreshold recovery claims into width constraints, while the surviving near-perfect regime with sufficiently large p remains open.

The boundary-refinement file, **ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryThreshold**, then combines that threshold-width inversion with the earlier unconditional surjective ceiling $n \leq 2r + 2k + 1$. If a manuscript route claims the strict recovery estimate

$$q < 1 - 2^{-(2r+4k+\text{slack}+1)},$$

Lean upgrades the full-rule **BitVec** n budget to

$$n \leq 2r + 2k + \text{slack}.$$

The bounded-sample plug-in-majority endpoint satisfies the same sharpened budget once the sample bound is large enough for surjectivity. In particular, setting $\text{slack} = 0$ kills the old boundary case $n = 2r + 2k + 1$ as soon as the route claims $q < 1 - 2^{-(2r+4k+1)}$. The regression file records the concrete uniform canaries: for **BitVec** $1, k = 1$, hypothetical $q = 1/2$ data already forces $1 \leq 2$; for **BitVec** $3, k = 0, r = 1$, the same $q = 1/2$ threshold rules out both the full-rule and bounded-sample surjective endpoints. This is still not branch closure, but it removes the former one-step visible-budget escape hatch whenever the recovery estimate is strong enough.

The next recovery-side file, **ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryRegionObst**, attacks the surviving injective branch from a different angle. The exact-visible wrapper **SharedExactZABSparseThres** already says that any one bad code can agree with the target on at most a q -mass finite region. On the injective branch, however, **SharedExactZABSparseThresholdInjectiveLiveness.lean** proves that the shared exact sparse-threshold family realizes *every* Boolean rule on the visible surface. Lean therefore takes any actual-local target predictor, any proper finite region R , and any visible point $x_0 \notin R$, flips the target only at x_0 , and realizes that altered rule by a shared sparse-threshold code. The resulting bad code still agrees with the target on all of R , so the recovery packet must satisfy

$$\mu(R) \leq q$$

for every proper finite region R . The same obstruction lifts not only to the full-rule endpoint but also to bounded sample plug-in majority, and here the bounded-sample corollary needs no surjectivity hypothesis at all: the generic actual-local theorem works with any chosen bounded-sample index.

This is real obstruction work, but still not branch closure. It does *not* show that the injective branch is impossible for every finite PMF; it shows that the branch can survive only when the advertised threshold q dominates the mass of every proper finite region. That is genuinely stronger than the earlier uniform cardinality floor on nonuniform measures. The regression file `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryRegionObstructionRegression.lean` uses the two-point `BitVec 1, k = 0` visible surface with a skew $4/5$ – $1/5$ PMF: the heavy singleton region already forces

$$4/5 \leq q,$$

and the concrete threshold $q = 3/4$ rules out both the full-rule endpoint and the bounded-sample endpoint at sample bound 2. So the new obstruction is strictly sharper than the old $1 - 1/|U| = 1/2$ uniform-cardinality floor on that same surface, while still leaving the near-perfect injective regime open above the skew region masses.

The next cleanup, `FinitePMFHeavyRegionCharacterization.lean` together with `ActualSwitchedLocalShare` shows that the preceding obstruction is already intrinsic and does not really depend on searching over arbitrary finite regions. On any finite PMF, Lean now proves the exact equivalence

$$\exists R \subsetneq U \text{ finite with } q < \mu(R) \iff \exists y \in U, q < 1 - \mu(y).$$

So the injective recovery obstruction can be repackaged canonically as a light-point criterion: a heavy proper region exists exactly when one visible point has too small a mass. The new recovery theorem reduces the earlier arbitrary-region obstruction to this point-complement form, and the regression file verifies the equivalence concretely on the same skew two-point `BitVec 1, k = 0` PMF. In particular, the genuinely non-surjective bounded-sample endpoint at sample bound 0 is already ruled out by the single inequality

$$3/4 < 1 - 1/5 = 4/5.$$

This is not new branch closure, but it does tighten the earlier narrative: the surviving injective branch is constrained by point-complements, not merely by an existential search over regions.

Lean now sharpens that characterization one step further. The same finite-PMF file proves that the existential point-complement condition is equivalent to a single canonical threshold

$$q < 1 - \mu(\text{lightestPoint}(\mu)),$$

where `lightestPoint( $\mu$ )` is a visible point of minimal mass. So the route obstruction does not just say “some point is too light”; it is already controlled by the worst visible atom of μ . The recovery file packages this directly onto bounded-sample plug-in majority as well: the empty sample index is enough to force

$$1 - \mu(\text{lightestPoint}(\mu)) \leq q$$

on the injective branch, with no surjectivity assumption. On the skew two-point `BitVec 1, k = 0` PMF this canonical threshold is exactly $1 - 1/5 = 4/5$, so the genuinely non-surjective sample-bound-0 endpoint is ruled out already by the single inequality $3/4 < 4/5$.

Lean now makes the structural meaning of that threshold fully explicit. The same finite-PMF file proves that the *entire family* of proper-region mass inequalities is equivalent to this one canonical bound:

$$\forall R \subsetneq U, \mu(R) \leq q \iff 1 - \mu(\text{lightestPoint}(\mu)) \leq q.$$

So the strongest proper-region obstruction is not hidden somewhere in a large search space of finite regions; it is attained by the single extremal region $U \setminus \{\text{lightestPoint}(\mu)\}$. The route-facing recovery file imports this equivalence directly, so the injective manuscript packet is now seen to satisfy the whole proper-region family exactly up to the worst-atom threshold. On the skew two-point `BitVec 1, k = 0` regression this means, concretely, that the claim “every proper region has mass at most $3/4$ ” already fails for the same sharp reason $3/4 < 1 - \mu(\text{lightestPoint}(\mu))$.

The follow-up file `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryInjectiveConcentration` closes the most obvious gap in those recovery thresholds. The earlier concentration and cardinality obstructions only applied on the *surjective* branch, because they obtained bad codes by realizing arbitrary singleton spikes. The injective-region obstruction is already strong enough to recover the same numerics without any surjectivity assumption. Lean simply takes the region $U \setminus \{y\}$: if *zfeat* is injective and one actual-local index *i* is present, then the point-complement bad code agrees with predictor *i* on all of $U \setminus \{y\}$, so every injective recovery packet must satisfy

$$1 - \mu(y) \leq q.$$

Combining this with the finite point-mass pigeonhole bound recovers

$$1 - |U|^{-1} \leq q$$

for *every* injective manuscript-facing recovery packet, even when the actual-local family is very far from surjective. The regression file `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryInjectiveConcentration` makes that distinction explicit with a one-index constant-false actual-local family on `ExactVisiblePostSwitchSurface`: the family is not surjective, yet hypothetical injective recovery data already forces

$$1 - 2^{-3} = 7/8 \leq q,$$

so $q = 3/4$ is impossible there as well. This is real route-facing obstruction work, but still not branch closure: it removes the surjectivity escape hatch for the injective concentration/cardinality floor, while the surviving near-perfect regime remains open.

The next bounded-sample follow-up removes the last surjectivity assumption from the manuscript-facing threshold inversion itself. The older bounded-sample threshold-width file still needed the sample-size hypothesis $|U| \leq \text{sampleBound}$ to make the endpoint surjective before it could conclude that

$$q < 1 - 2^{-p} \implies n + 2k < p.$$

But on the injective branch there is no need to realize every Boolean rule via bounded samples: Lean can simply fix the explicit bounded-sample predictor index $\langle [], \text{by simp} \rangle$, apply the new injective concentration theorem to that one actual-local predictor, and recover the same uniform threshold

$$1 - 2^{-(n+2k)} \leq q$$

with no lower bound on `sampleBound`. The file `ActualSwitchedLocalBoundedSampleSharedExactZABSparseThresholdERMRecoveryInjectiveConcentration` then inverts this bound exactly as before, but now for the genuinely non-surjective bounded-sample endpoint as well. Its regression file `ActualSwitchedLocalBoundedSampleSharedExactZABSparseThresholdERMRecoveryInjectiveConcentration` uses the `BitVec 1, k = 1` surface at sample bound 0: the bounded-sample interface is formally not surjective, yet any hypothetical injective recovery packet still forces $7/8 \leq q$, and the stricter threshold $q = 3/4$ is impossible. This is again real obstruction work, but not branch closure: it closes a bounded-sample technical escape hatch on the injective branch, while the near-perfect regime still survives.

The next file, `ActualSwitchedLocalSharedExactZABSparseThresholdERMBitCode.lean`, separates the weaker consequence that does *not* depend on any recovery estimate. Lean packages the exact-family identification itself as an actual-local bounded-code witness:

`ActualSwitchedLocalInterface.SharedExactZABSparseThresholdERMData.bitCodeData.`

So once the actual switched-local family is exactly the point-block shared exact sparse-threshold ERM family, each local rule is already decoded from one fixed bit family of budget

$$2 \cdot \text{allAffinePointBlockFeatureCount}(r + 2k).$$

Therefore exact-family identification alone, before any agreement bound is proved, already yields:

- the exact-visible compression target at that budget,
- a finite predictor cover / representative cover / quotient of size $2^{2 \cdot \text{allAffinePointBlockFeatureCount}(r+2k)}$, and
- the bundled clocked K -poly finite-learning payload at that same budget.

This sharpens the route decomposition. The recovery-data wrapper is only needed for the weighted exact-recovery lower bound; it is not needed for the finite-image or clocked-payload side. That is still route-facing infrastructure rather than a new blocker, but it removes one more place where the manuscript could hide missing bounded-code bookkeeping behind the recovery argument.

The next file, `ActualSwitchedLocalSharedExactZABSparseThresholdERMVisibleBudgetObstruction.lean`, turns that bounded-code packet into an unconditional surjectivity obstruction. If an actual switched-local family is exactly the point-block shared exact sparse-threshold ERM family and is still surjective on the exact visible surface, then Lean now proves

$$|\text{ExactVisiblePostSwitchSurface}(Z, k)| \leq 2 \cdot \text{allAffinePointBlockFeatureCount}(r + 2k).$$

So the exact visible surface must fit inside the point-block sparse-threshold bit budget before any recovery estimate or finite counting-gap hypothesis is used. On `BitVec n`, this specializes to the explicit width bound

$$n \leq 2r + 2k + 1.$$

Equivalently, if $2r + 2k + 1 < n$, then the full-rule actual switched-local interface cannot carry the shared exact sparse-threshold ERM packet at all. This is weaker than the earlier conditional $r \geq n$ characterization of the surviving injective branch, but it is unconditional: it does not rely on the counting-gap liveness hypothesis and it does not require any recovery-side agreement estimate. So even before the route reaches the harder injective extractor regime, Lean already rules out all extractors that compress below roughly half the visible width. The new route-facing wrapper now makes the quantifier order explicit as well: below the same threshold, Lean proves

$$\neg \exists z \text{feat} : \text{BitVec } n \rightarrow \text{BitVec } r, \text{SharedExactZABSparseThresholdERMData}(\text{fullRuleActualSwitchedLocalInt}$$

So the visible-budget obstruction is no longer only about a bad fixed extractor; the full-rule sparse-threshold ERM route cannot rescue itself by choosing a different extractor later.

The follow-up file `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryVisibleBudgetObstr` closes the obvious manuscript-facing interface gap in that argument. Recovery data already contains

the exact-family data, so the same unconditional half-width obstruction applies directly to the full recovery packet: any surjective manuscript-facing recovery packet on `BitVec n` still forces

$$n \leq 2r + 2k + 1.$$

Lean then records the corresponding endpoint corollaries for both the full-rule actual switched-local interface and the bounded-sample plug-in-majority interface. Thus the unconditional visible-budget obstruction is not merely a data-layer fact; it already kills the manuscript-facing recovery endpoints below the same half-width threshold, with no q -estimate and no finite counting-gap hypothesis. The updated endpoint wrappers again remove the extractor choice itself: below the same threshold, Lean proves that there does not exist any extractor $zfeat : \text{BitVec } n \rightarrow \text{BitVec } r$ supporting the full-rule recovery packet, and likewise none supporting the bounded-sample plug-in-majority recovery packet once the sample bound is large enough for surjectivity.

Lean now records a different manuscript-facing consequence of the same recovery packet: it always yields the bundled clocked K -poly finite-learning payload at point-block budget

$$s = 2 \cdot \text{allAffinePointBlockFeatureCount}(r + 2k).$$

The new file `ActualSwitchedLocalSharedExactZABSParseThresholdERMRecoveryPayloadObstruction.lean` turns that directly into an obstruction theorem. If the actual switched predictor family already realizes an injectively indexed finite probe family of size $> 2^s$, then no manuscript-facing sparse-threshold recovery packet can exist. Lean then sharpens that to a route-native finite-family statement: if the actual predictor map itself is injective on a finite index type, then recovery already forces

$$|\text{Index}| \leq 2^s.$$

So the manuscript cannot hide an oversized actual-local family behind the finite-learning layer merely by avoiding surjectivity onto the full Boolean rule space. The full-rule surjective endpoint is just one corollary; the more important route-facing statements are the general injective-realization version and its injective-index specialization. The latest strengthening now removes the extractor choice under the same payload obstruction. Lean proves that once the actual switched predictor family is too expressive for the payload budget, there does not exist any extractor $zfeat$ supporting the manuscript-facing recovery packet. This is recorded both for the full-rule surjective endpoint and for arbitrary finite-index actual-local families whose predictor map is already injective on too many indices. The regression file makes this distinction concrete on the three-point visible surface `ExactVisiblePostSwitchSurface(Fin 3, 0)`: Lean defines an actual-local family with exactly five indices and five realized predictors, proves that this family is *not* surjective onto the full eight-rule Boolean space, and still rules out recovery because the point-block payload budget there only permits $2^2 = 4$ predictors. The strengthened endpoint now says more: that five-predictor family cannot be rescued by picking some different `Fin 3` $\rightarrow$ `BitVec 0` extractor later. This is real obstruction work, but not branch closure: it closes a manuscript-facing finite-learning loophole beyond the pure width/cardinality wrappers, while the larger injective regime remains open.

Lean now records the same finite-image burden on the quotient-code side directly. The new file `ActualSwitchedLocalSharedExactZABSParseThresholdERMRecoveryQuotientObstruction.lean` starts from the fact that any recovery packet already yields a finite predictor quotient of size at most 2^s at the same point-block budget. From there Lean proves a route-facing exact consequence: every injectively realized finite probe family on the exact visible surface must already fit inside that quotient budget. Equivalently, if the actual predictor map itself is injective on a finite index type, then recovery forces

$$|\text{Index}| \leq 2^s$$

even before one appeals to the clocked finite-learning payload semantics. Thus a quotient-code repair is not a new escape hatch. It owes exactly the same small-image theorem on the actual switched family. The latest strengthening now removes the extractor choice here as well: once an injective realized family or injective predictor index set already exceeds the quotient budget, Lean proves that there does not exist any extractor *zfeat* supporting the manuscript-facing recovery packet. The regression file reuses the same genuinely non-surjective **Fin 5** family on the three-point visible surface and shows that recovery is already impossible at the quotient layer, because five injectively realized predictors cannot be encoded into the available four-code budget; moreover, that family cannot be rescued by picking some different **Fin 3** $\rightarrow$ **BitVec 0** extractor later. This is not branch closure either, but it closes a different manuscript-facing loophole: one cannot hide an oversized realized family behind a proposed small quotient presentation.

Lean now records a different recovery-side obstruction that genuinely uses the agreement threshold q , not the same 2^s image budget again. The new file **ActualSwitchedLocalSharedExactZABSparseThresholdERMRecovery** observes that once the actual switched family is identified with one fixed shared exact sparse-threshold bit family, every other distinct realized predictor is automatically a bad code against the current target. Therefore the recovery hypothesis itself forces a pairwise separation law: for any two distinct realized predictors $g_i \neq g_j$,

$$\text{agreementMass}(\mu, g_i, g_j) \leq q.$$

This is a different kind of burden from the earlier width, payload, or quotient wrappers. It says that recovery cannot coexist with even one realized pair whose overlap under μ is too large. The regression file again reuses the same genuinely non-surjective **Fin 5** family on the three-point visible surface, but now with the pure measure concentrated at one visible point. Two distinct realized predictors in that family agree on full μ -mass there, so Lean rules out recovery already for $q = 3/4$. Thus the route cannot escape merely by keeping the realized family small while still allowing highly overlapping realized predictors under the chosen measure.

Lean now pushes that same q -dependence to a family-wide obstruction. The new file **ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryHeavyRegionCardinalityObstruction** shows that if one visible point x already has mass $\mu(x) > q$, then any two realized predictors agreeing at x must in fact be equal globally. Since the actual outputs are Boolean, evaluation at that one heavy point can take only two values, so any injectively indexed actual switched family supporting the recovery packet must satisfy

$$|\text{Index}| \leq 2.$$

This is not another bit-budget wrapper. It is a measure-side collapse theorem: one heavy atom above the claimed recovery threshold forces the whole realized family through a single visible Boolean coordinate. The regression file again reuses the genuinely non-surjective **Fin 5** family with pure mass at one visible point, and now rules out $q = 3/4$ for the stronger reason that an injective five-predictor family cannot survive a heavy-point threshold below 1.

Lean now generalizes that collapse from one coordinate to any finite heavy region. The new file **ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryHeavyRegionCardinalityObstruction** proves that if a finite visible region R already has mass $\mu(R) > q$, then any two realized predictors agreeing on all of R must already be equal globally. So on any injectively indexed actual switched family, restriction to R is itself injective, and the whole realized family must inject into the Boolean trace space on R :

$$|\text{Index}| \leq 2^{|R|}.$$

This is genuinely stronger than the heavy-point theorem when no individual atom exceeds q but a small region still does. The regression file exhibits exactly that situation on the three-point visible surface with split measure $(1/2, 1/2, 0)$: no single point has mass above $3/4$, so the heavy-point

collapse does not apply, yet the two-point region $\{0, 1\}$ already has mass 1. Lean then reuses the genuinely non-surjective `Fin 5` family and rules out recovery at $q = 3/4$ because an injective five-rule family cannot fit into the four Boolean traces on a two-point region.

Lean now pushes that heavy-region barrier directly onto the manuscript’s full-rule endpoint, removing the extra injective-branch hypothesis that the earlier proper-region file still required. The new file `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoveryFullRuleRegionObstruction.lean` proves that if a full-rule recovery packet existed and one finite visible region R satisfied $|R| < |\text{surface}|$ together with $\mu(R) > q$, then the full Boolean rule family on the whole exact visible surface would have to inject into the Boolean trace space on R . Lean formalizes that as the mass bound

$$\mu(R) \leq q$$

for every proper finite region R , with no injectivity or width assumption on $zfeat$. The regression file reuses the skew `BitVec 1` measure $(4/5, 1/5)$: Lean now derives $4/5 \leq q$ from any hypothetical full-rule recovery packet by the new theorem itself, and therefore rules out $q = 3/4$ on that endpoint without appealing to the older injective-branch construction.

Lean now pushes the same region barrier past the full-rule identity case to arbitrary surjective actual switched families. The new file `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoverySurjective` shows that if the actual predictor family is surjective onto all Boolean rules on the exact visible surface, then every proper finite region still has mass at most q . The proof is cleaner than the old injective-branch argument: Lean chooses two realized predictors directly from surjectivity, namely the constant-false rule and the same rule flipped at one point outside the region, and then applies the heavy-region rigidity theorem to force a contradiction. So the route cannot escape by keeping surjectivity while allowing duplicated indices or other noninjective presentations of the realized family. The latest strengthening now removes the extractor choice on that same branch: once some proper visible region has mass above q , Lean proves that there does not exist any extractor $zfeat$ supporting the manuscript-facing recovery packet on the surjective actual switched family.

The regression file `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoverySurjectiveRegionObstruction` makes that distinction explicit. It defines a duplicated-index actual local interface whose predictor map ignores one Boolean coordinate of the index, so the family is still surjective onto the full exact visible rule class but is no longer injective as an indexed family. On the same skew `BitVec 1` measure, Lean proves that any recovery packet on this duplicated interface already forces $4/5 \leq q$, and therefore rules out $q = 3/4$. This shows the new theorem is not merely another injective-family cardinality wrapper. The new existential endpoint says more: the duplicated surjective family cannot be rescued by switching to some different `BitVec 1` → `BitVec 1` extractor after the heavy proper region has already violated the q -bound.

The follow-up file `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoverySurjectiveHeavyRegionObstruction` packages the exact threshold hidden in that surjective-region obstruction. On the surjective branch, Lean now proves that the whole proper-region mass family collapses to one scalar inequality

$$1 - \mu(\text{lightestPoint}) \leq q.$$

So even without injective indexing, the obstruction is not really “search over all proper regions”; it is already the canonical complement mass of the lightest visible point. The regression file `ActualSwitchedLocalSharedExactZABSparseThresholdERMRecoverySurjectiveHeavyRegionCharacterization` keeps this branch nontrivial by reusing the duplicated-index surjective family. On the same skew `BitVec 1` witness, Lean proves the concrete threshold $4/5 \leq q$ through that lightest-point form and again rules out $q = 3/4$ on a surjective family whose predictor map is not injective.

The new synthesis upgrade in `CruxSynthesis.lean` packages those two surjective actual-local recovery burdens at the route level rather than only as file-local facts. First, below the half-width ceiling $2r + 2k + 1 < n$, Lean now records that no extractor at all can support the manuscript-facing recovery packet on any surjective actual-local endpoint on `BitVec n`. Second, below the intrinsic lightest-point threshold $q < 1 - \mu(\text{lightestPoint})$, Lean likewise records that no extractor can support that packet on any surjective actual-local endpoint over a finite visible surface. The concrete sample-level plug-in-majority route and the bit-coded exact-visible fallback route are now explicit instances of those general synthesis theorems, rather than separate endpoint-specific wrappers.

If such a theorem cannot be proved, then the proof attempt should be regarded as stalled at a real mathematical gap, not a missing editorial detail.

29 Lean Excerpts

Listing 1: Core asymptotic obstruction

```
theorem polylog_isLittleO_logRadiusNeighborhood (k : Nat) {d c : Real}
  (hd : 1 < d) (hc : 0 < c) :
  polylog k =o[atTop] logRadiusNeighborhood d c

theorem logRadiusNeighborhood_not_isBigO_polylog (k : Nat) {d c : Real}
  (hd : 1 < d) (hc : 0 < c) :
  ¬ logRadiusNeighborhood d c =O[atTop] polylog k
```

Listing 2: Core counting obstruction

```
theorem no_injective_bitCode_of_lt {n s : Nat} (hs : s < 2 ^ n) :
  ¬ ∃ f : LocalRule n → BitCode s, Function.Injective f

theorem no_uniform_code_below_expInput_for_binaryLogNeighborhood
  {d m s : Nat} (hd : 2 ≤ d) (hs : s < 2 ^ m) :
  ¬ ∃ f : LocalRule (d ^ (Nat.log 2 m + 1)) → BitCode s,
    Function.Injective f
```

30 Conclusion

The present crux-first formal work yields a clean conditional diagnosis.

1. Logarithmic-radius locality is *not* the same thing as polylogarithmic complexity.
2. The full class of local Boolean rules on that visible data is far too large to fit into a polylogarithmic coding budget.
3. Therefore the program needs a separate theorem proving exceptional compressibility of the particular switched predictors it generates.
4. The v13 $P = NP$ upper-bound decoder also needs a single-message theorem for arbitrary satisfying SAT-search outputs. In Lean this is not just a slogan: correctness for all valid SAT-search outputs is equivalent to the fixed-message readout obligation, and with one satisfying witness it is equivalent to single-message constancy. At the manuscript CNF layer, completeness of the extraction makes supported arbitrary-output SAT-search correctness equivalent to the

semantic single-message promise itself; a separate unrepresented-witness countermodel now shows this completeness premise cannot be dropped. In the generalized obstruction, two satisfying witnesses with unequal readouts already rule out the existence of any message correct for all valid SAT-search outputs. The weaker selected-output contract is now explicitly separated: ambiguous satisfying witnesses can make selected-output correctness true for two different messages while arbitrary-output correctness remains impossible.

5. The v13 product-success step also needs a sequential conditional admissibility theorem; unconditional half-success marginals are insufficient, while the checked sequential conditional interface is strong enough for the two-step finite product bound.
6. A generic finite switching theorem now proves the full tower-product bound from sequential half-admissibility. The open crux is whether the manuscript's switched histories satisfy that admissibility premise on positive-mass prefixes. Empty prefixes are automatically harmless by finite-history monotonicity, while one failed nonempty prefix half-step is enough to block the tower argument. Independently, a failed final tower-product count blocks sequential admissibility and now localizes to an exact failed conditional prefix cut.
7. The v13 instantiation obligation is now fixed-field-level: both the abstract atom-ledger interface and an existential concrete-cell interface are equivalent to the generic prefix half-step. The manuscript must therefore specify the actual finite history field and prove that its fixed fibers decompose the relevant histories and satisfy the cellwise half-success bounds. The decomposition part is automatic for any finite cell map; the substantive obligation is the half-success bound inside each supplied cell, equivalently a proof that every supplied cell has at least as many next-failure prefix points as next-success prefix points. Equivalently, after the field is fixed, each successful prefix point must be injectively matched to an unsuccessful prefix point in the same field cell. This fixed-field premise is now also diagnostic in the negative direction: if the matching or prefix certificate fails, Lean extracts a supplied cell whose next-failure count is strictly smaller than its next-success count. Product-bound violations therefore localize not only to a fielded position but to a concrete bad cell in that position. This fixed-field premise is strictly stronger than global half-success: a field that reveals the successful Boolean coordinate already fails cellwise at the first step. The ordered fielded-switching interface now gives the positive bridge as well: if every supplied field certificate is proved, the tower product bound follows; if one fixed-field prefix certificate fails, the remaining fielded suffix is blocked. More generally, a positive-mass fixed-field cell that is entirely successful already blocks the certificate, so a field that exposes the next success event is admissible only in the vacuous zero-success case. Equivalently, every successful atom in a valid fixed-field certificate must be mixed with at least one unsuccessful prefix point. The same conclusion now holds for any supplied field that determines the next success bit cellwise on the current history support: on a positive-mass success prefix, such a field blocks the whole fielded suffix, and the theorem applies at any later fielded position after the preceding success-event history has been accumulated. This is strictly stronger than the global version: the Boolean-square diagonal model has a second-coordinate field that is not globally success-determining but becomes success-determining after conditioning. In the operational matching formulation, any recursive matching proof on such a positive-mass prefix directly proves that the supplied field is not success-determining on the accumulated history support. Even more concretely, every successful prefix point must have a same-cell failed prefix point; a single successful point whose supplied field cell contains only successes is already enough to rule out the matching and certificate.

8. A repeated exact-success event gives a checked positive-mass obstruction to that atom-ledger obligation: after any positive-mass first occurrence, the second repeated step has conditional success 1, so no abstract, concrete-cell, fixed-field, or recursive same-cell matching v13 instantiation exists for the repeated two-step claim. The Boolean-square instance also violates the global tower-product inequality, blocking both abstract and concrete v13 certificates by the product-bound contrapositive. The fixed-field version is blocked at the same level: when both repeated cuts use the one-cell field, the fielded tower-product bound fails and therefore no recursive same-cell matching proof can exist. This failure is no longer only global: Lean localizes any such failure to a concrete fielded position and then to a bad supplied cell whose failures are fewer than successes. In the one-cell repeated model the second conditioned cut is explicitly the failed same-cell matching/certificate obligation, with the unique bad cell carrying failure count 0 and success count 2. The forced-step generalization blocks the same repair even when the later event is not a literal repeat: on a positive accumulated history, any next success event already implied by that history leaves the count unchanged and cannot be a half-step. In the Boolean-square anchor, two valid coordinate half-cuts force a third globally half-mass diagonal success event, so the third fixed-field switching and same-cell matching obligations fail.

These are currently the sharpest formally checked cruxes we have in Lean.

References

- [1] Anonymous manuscript under study. *P != NP: A Non-Relativizing Proof via Quantale Weakness and Geometric Complexity*. October 2025.
- [2] Mettapedia formal notes. `Mettapedia/Computability/PNP/LocalityObstruction.lean` and `Mettapedia/Computability/PNP/CompressionObstruction.lean`.